

THE ZCASH ARBORETUM

VIII

ZSA Guide

Zcash Shielded Assets: asset identity, issuance, transfer, and burn

Marek
m@rek.onl

Abstract

This volume of *The Zcash Arboretum* describes the OrchardZSA protocol of ZIP 226 and ZIP 227, both with status Draft, as a delta on the Orchard protocol that the *Ironwood Guide* constructs. The protocol extends the Orchard protocol by the issuance, transfer and burn of Custom Assets. The volume constructs, in dependency order, the identity of an Asset; the note, its commitment and its value commitment, extended by the Asset Base; the Action statement and the Action Group; burn and the binding validating key; issuance and the global issuance state; and the status of the transaction carrier. It proves no counterfeiting across Assets, supply integrity, issuance authority and privacy of the Asset type under named assumptions. NU7, specified by ZIP 259 (Draft), introduces no transaction format and includes neither ZIP, so no network upgrade includes the protocol; its transaction format survives only in withdrawn text, and every construction is therefore classified as specified, designed but unspecified, or an open problem. The volume assumes the *Math Guide*; the *Crypto Guide* for signatures (§“Digital signatures”), Pedersen commitments (§“Commitment schemes”, §“The Pedersen commitment”) and random oracles (§“Hash functions and the random oracle model”); the *Halo 2 Guide* for the proof system; and the *Ironwood Guide* for the Orchard protocol. It is non-normative: the Zcash Protocol Specification, cited as the protocol specification with the title of a section, and the Zcash Improvement Proposals are authoritative.

Contents

1	Introduction	5
1.1	Scope and status	5
1.2	Epistemic classes	7
1.3	The OrchardZSA protocol	8
1.4	Requirements on shielded assets	10
2	Asset identity	11
2.1	Issuance keys and the issuer identifier	11
2.2	Asset descriptions and Asset Identifiers	13
2.3	Asset Digests and Asset Bases	14
2.4	The Native Asset Base	18
3	Notes and note commitments	18
3.1	The OrchardZSA note	19
3.2	Note commitments	19
3.3	The note seed and the lead byte	22
3.4	The note plaintext	23
3.5	The per-Asset value commitment	24
4	OrchardZSA Actions and Action Groups	25
4.1	Dummy notes and Split Inputs	25
4.2	Nullifiers of OrchardZSA notes	28
4.3	The OrchardZSA Action statement	30
4.4	The OrchardZSA Action proof	34
4.5	Action Groups	36
5	Burn and the binding signature	37
5.1	Burn sets and OrchardZSA bundles	37
5.2	The binding validating key	38
5.3	Per-Asset balance	39
6	Issuance of Custom Assets	43
6.1	Issue Notes	44
6.2	Issuance Actions and Issuance Bundles	46
6.3	Computation of ρ for Issue Notes	47
6.4	Reference notes	49
6.5	Issuance authorisation	51
7	The global issuance state	53
7.1	The issuance state and its chaining	53
7.2	State transition of a transaction	55
7.3	Finalisation	58

7.4	Verification of an OrchardZSA transaction	59
8	The transaction carrier	61
8.1	Transaction formats	61
8.2	Transaction digests and the signature message	64
8.3	Fees	66
9	Security	67
9.1	Accepted transactions and assumptions	67
9.2	Asset Bases and balance of accepted bundles	69
9.3	Circulating supply	73
9.4	Authority over issuance	75
9.5	Hiding of the Asset type	77
10	Classification of the components	80
10.1	Classes of the constructions and results	81
10.2	Open problems	84

1 Introduction

This section fixes the subject of the volume, its status and its vocabulary before any construction: the sources and their status, and the notation (§1.1); the three epistemic classes, the status of a result proved here and the carrier of a construction (§1.2); the Assets, the OrchardZSA protocol and the premise on the pool that carries its notes (§1.3); and the requirements that a protocol for shielded assets must meet, each paired with the result of this volume that discharges it (§1.4).

1.1 Scope and status

This volume of *The Zcash Arboretum* describes the OrchardZSA protocol, defined in §1.3, as a delta on the Orchard protocol that the *Ironwood Guide* constructs. Every construction of the volume modifies an object of that volume, or adds an object beside it, and the objects of the Orchard protocol are cited by the *Ironwood Guide*’s titles, never re-derived: §“Notes and note commitments”, with Definition “Note” and Definition “Note commitment”; §“The note seed: Ironwood derivations (lead byte 0x03)”; §“Actions, bundles, and dummy notes”, with Definition “Dummy notes”; §“Value commitments and the binding signature”, with §“Value commitments” (Definition “Net value commitment”), §“The balancing value” and §“The binding signature”; §“The Action statement and its proof”, with §“The Action statement” (Definition “Orchard Action statement”); §“Nullifiers”; §“Sinsemilla: hash, commitment, and short forms”; §“GroupHash, domain separation, and nothing-up-my-sleeve generators”; §“The Orchard protocol and its two pools”; §“The transaction format”; §“Value conservation”, with Theorem “Balance”; §“Fields, groups, and encodings”; and §“Requirements on a shielded payment”. The volume further assumes the *Math Guide*; the *Crypto Guide* for digital signatures (§“Digital signatures”), Pedersen commitments (§“Commitment schemes”, §“The Pedersen commitment”) and random oracles (§“Hash functions and the random oracle model”); and the *Halo 2 Guide* for the proof system. No other volume is cited. The volume is non-normative: the Zcash Protocol Specification, cited as the protocol specification with the title of a section, and the Zcash Improvement Proposals are authoritative.

Sources. Two ZIPs specify the protocol. ZIP 227, “Issuance of Zcash Shielded Assets”, specifies the issuance of Custom Assets; ZIP 226, “Transfer and Burn of Zcash Shielded Assets”, specifies their transfer and burn. Both have status Draft and category Consensus (ZIP 226 and ZIP 227, preambles). ZIP 226 is written relative to the protocol with ZIP 2005 applied (ZIP 226, Abstract), and calls ZIP 2005 “Orchard Quantum Recoverability”; the current title of ZIP 2005 is “Ironwood Quantum Recoverability”, and its status is Proposed (ZIP 2005, preamble). Every construction of the OrchardZSA protocol is therefore fixed, where it is fixed at all, by Draft text, and none belongs to the consensus rules of any network upgrade. The class of each construction (§1.2) is stated once, where the construction is made.

Definition 1.1 (ZIP statuses). The status field of a ZIP takes one of the values that ZIP 0, “ZIP Status Field”, fixes; a ZIP with several revisions carries one status per revision. This volume uses five of them.

- *Draft*: the status of every ZIP on submission. ZIP 226, ZIP 227, ZIP 229, ZIP 258 and ZIP 259 have it.

- *Proposed*: typically the stage after Draft, set after consideration, feedback and rough consensus of the community. ZIP 2005 has it.
- *Active*: typically used for Process or Informational ZIPs, set once rough consensus on a Proposed ZIP is reached. Revision 0 of ZIP 317 has it.
- *Final*: the status of a Consensus or Standards ZIP that is both implemented and activated on the Zcash network.
- *Withdrawn*: the owners of the ZIP have removed it from consideration by the community. ZIP 230 and ZIP 246 have it.

A ZIP text is *live* when its status, or the status of the revision concerned, is Draft, Proposed, Active or Final. Every published ZIP that this volume cites is either live or Withdrawn. The *Ironwood Guide*’s Definition “ZIP” names only the statuses Draft and Reserved; Reserved is not used here.

Status with respect to network upgrades. NU6.3 is the active network upgrade, deployed by ZIP 258, and NU7 is specified by ZIP 259 (Draft). The sources of the consensus changes of NU7 are the protocol specification, ZIP 200, Revision 2 of ZIP 207, Revision 3 of ZIP 214, and ZIPs 218, 235, 237, 2003 and 2008; none of these is ZIP 226 or ZIP 227; NU7 introduces no new transaction format, and version-5 and version-6 transactions remain valid (ZIP 259, Abstract and Specification, “NU7 consensus changes”). ZIP 258 does not mention Zcash Shielded Assets (ZIP 258, Specification). ZIP 226, “Deployment”, still names NU7 as the network upgrade that deploys the protocol, which ZIP 259 contradicts, and ZIP 227, “Deployment”, reads “TBD”. The assignment of the OrchardZSA protocol to a network upgrade is therefore an open problem in the sense of Definition 1.2, stated in §1.2.

Notation. The notation is that of the *Ironwood Guide*, whose table in §“Fields, groups, and encodings” maps it to the names of the protocol specification. Table 1 extends that map to the names that ZIP 226 and ZIP 227 use and that the table of the *Ironwood Guide* does not list. The names of the ZIPs appear in that table only. The table lists no symbol that this volume defines: every such symbol, among them the Asset Base, the Asset Identifier and Asset Digest, the OrchardZSA note and value commitments, the split flag, the pause flag, the issuance keys, the global issuance state and the burn set, is defined at its first use. The same convention applies to the generator L^{Orchard} of Construction 4.4, which ZIP 226 writes calligraphically.

Prose uses British spelling, as in finalisation and authorisation. Identifiers and titles taken from the ZIPs keep their own spelling, as in the flag finalize and the title “Issuance Authorization Signature Scheme” of ZIP 227.

Table 1. ZIP notation and series notation. The set $\mathcal{E}^*$ is that of the *Ironwood Guide*’s Definition “Orchard Action statement”; the star encoding is its Definition “Star encoding” and the group hash its Construction “GroupHash on Pallas”; the three generators are those of its §“Value commitments” and §“Nullifiers”. The value $\ell_p = 256$ is fixed in the protocol specification, §“Pallas and Vesta”.

ZIP notation	This volume	Meaning
$\mathbb{P}$	$\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$, also $\mathcal{E}$	the Pallas group
$\mathbb{P}^*$	$\mathcal{E}^* := \mathcal{E} \setminus \{\mathcal{O}\}$	the Pallas group without its identity
$q_{\mathbb{P}}$	p_{Pallas}	the Pallas base-field modulus
$n_{\mathbb{P}}$	p_{Vesta}	the order of the Pallas group
ℓ_p	256	the bit length of the star encoding
LEBSP_k	LE_k	k -bit little-endian encoding
$\text{repr}_{\mathbb{P}}(P)$	P^*	the star encoding of $P \in \mathcal{E}$
$\text{GroupHash}^{\mathbb{P}}$	GroupHash	the group hash into $\mathcal{E}$
$\mathcal{V}^{\text{Orchard}}, \mathcal{R}^{\text{Orchard}}, \mathcal{K}^{\text{Orchard}}$	$\mathcal{V}^{\text{Orchard}}, \mathcal{R}^{\text{Orchard}}, \mathcal{K}^{\text{Orchard}}$	fixed generators of the Orchard protocol

1.2 Epistemic classes

Definition 1.2 (Three epistemic classes). Each construction of this volume is in exactly one of three classes.

- (i) It is *specified* when normative text of the protocol specification or of a live ZIP (Definition 1.1), Draft included, fixes it.
- (ii) It is *designed but unspecified* when no live normative text and no published proof fixes it, but a complete design of it exists, in withdrawn ZIP text, in text removed from a ZIP, in a document outside the ZIPs, or as an argument of a ZIP’s rationale without proof, and that design conflicts with no live text.
- (iii) It is an *open problem* when the protocol needs it and either no text, live, withdrawn or removed, supplies a design of it; or every existing design conflicts with live text, in that a value or position that the design assigns is assigned otherwise by live text; or live texts contradict one another about it.

The *Ironwood Guide*’s Definition “Epistemic classes” uses the first two names for claims about the deployed protocol; here they are redefined for constructions, and the third is added.

Each construction is classified where it is made. Among the designed but unspecified constructions are the OrchardZSA circuit (§4.4), the derivation of the commitment trapdoor of an OrchardZSA note (§3.3) and the fee terms (§8.3). Among the open problems are the pool that carries OrchardZSA notes (Remark 1.7); the live carrier, whose withdrawn version 6 is now the format of ZIP 229, which need not support Zcash Shielded Assets (ZIP 229, Non-requirements); the bit of the pause flag enableZSA, whose position in withdrawn ZIP 230, bit 2 of the flags byte, is that of enableCrossAddress in ZIP 229 (§8.1); and the value of the lead byte, whose removed value 0x03 is the Ironwood lead byte (§3.3).

Definition 1.3 (Status of a result proved in this volume). A theorem, proposition, lemma or corollary of this volume is proved here for the constructions as the ZIPs state them, and carries no class: the classes of Definition 1.2 apply to constructions and to the rationale arguments of the ZIPs. The hypotheses of a result are named in its statement. A hypothesis that concerns a designed but unspecified object, or an open problem (Definition 1.2(iii)), makes the result *conditional* on that object:

- on the OrchardZSA circuit, through Assumption 4.9 and Assumption 4.10 of “The OrchardZSA Action proof” (§4.4);
- on the signed message, through Assumption 6.13 of “Issuance authorisation” (§6.5);
- on an open problem, through a hypothesis that names it: Lemma 7.6, Proposition 6.11(a) and Theorem 9.11.

A result about notes created by issuance, such as the provenance of Asset Bases or supply integrity, holds for every deterministic derivation of the commitment trapdoor rcm from the published fields of such a note. It depends on the designed but unspecified derivation of rcm only in that some such derivation must be fixed for a validator to compute the commitment. Where a ZIP argues the same property in its rationale only, that argument remains designed but unspecified; a proof in this volume does not change its class.

Every construction states its class where it is made, and every result the premises on which it is conditional. “Classification of the components” (§10) tabulates both.

Definition 1.4 (Carrier of a construction). The *carrier* of a construction is the set of transaction fields that encode it, together with the digests that commit to it: the transaction identifier, the authorising-data digest and the signature digest, the three digests of the *Ironwood Guide*, §“Transaction digests and signatures”. A construction can be specified while its carrier is not.

1.3 The OrchardZSA protocol

Definition 1.5 (Asset, Custom Asset, Native Asset). An *Asset* is a type of note that can be transferred on the Zcash block chain. Each Asset is identified by an Asset Identifier (ZIP 227, Terminology). ZEC is the default, and currently the only defined, Asset on Mainnet, and TAZ the default, and currently the only defined, Asset on Testnet; each is the *Native Asset* of its network. A *Custom Asset* is an Asset other than ZEC and TAZ (ZIP 227, Terminology; ZIP 226, “Burn Mechanism”). The Asset Identifier of a Custom Asset is constructed in “Asset descriptions and Asset Identifiers” (§2.2); no ZIP constructs one for the Native Asset, whose Asset Base is fixed directly (Definition 2.15). In this volume *issuance* means the issuance of a Custom Asset, in the sense of ZIP 227. It is distinct from the creation of ZEC by the block subsidy in a coinbase transaction (*Ironwood Guide*, Definition “Coinbase transaction”), which the OrchardZSA protocol does not modify.

Definition 1.6 (OrchardZSA protocol). The *OrchardZSA protocol* is the extension of the Orchard protocol that ZIP 226 and ZIP 227 define jointly: ZIP 227 specifies the issuance of Custom Assets, and ZIP 226 their transfer and burn. ZIP 227 is to be implemented only together with ZIP 226, because the notes that issuance creates can be transferred only by the OrchardZSA transfer protocol (ZIP 227, Abstract, where “must” is lower-case and is not a key word of BCP 14). Relative to the Orchard protocol it makes the following changes, each constructed in the section named with it.

- (1) Each note gains one field, a point of the Pallas group that names its Asset (“Notes and note commitments”, §3).
- (2) The note commitment becomes a case split that leaves the commitments of ZEC notes unchanged (same section).
- (3) The net value commitment takes that point as its value base, in place of the fixed base V^{Orchard} (same section).
- (4) The binding validating key further subtracts the burnt amounts (“Burn and the binding signature”, §5).
- (5) An Action of a Custom Asset that consumes no note pads its consumed side with a copy of a note already committed, of the same Asset and counted at value 0, instead of a dummy note (“OrchardZSA Actions and Action Groups”, §4).
- (6) The Action statement changes accordingly (same section).
- (7) ZIP 227 adds issuance and a global issuance state (“Issuance of Custom Assets”, §6; “The global issuance state”, §7).

The sources of these changes are ZIP 227, Abstract, and ZIP 226, Abstract and Specification, with “Split Notes”, “Value Balance Verification” and “Backward Compatibility”.

Remark 1.7 (The Orchard-pool premise). ZIP 226 places OrchardZSA notes in the Orchard pool, which the Orchard and the OrchardZSA protocols then share, and adds their note commitments to the same note commitment tree (ZIP 226, “Privacy Implications”, “Backward Compatibility” and “Rationale for Note Commitment”). It also assumes ZIP 2005 applied (ZIP 226, Abstract). ZIP 2005 applies its recoverable note plaintext format, of lead byte $0x03$, only to notes of the Ironwood pool (ZIP 258, “ZIP 2005 activation”), and note plaintexts of the Orchard pool take the lead byte $0x02$ only (protocol specification, §“Note Plaintexts and Memo Fields”). Since NU6.3 the Orchard pool is sealed (ZIP 258, “Consensus rules from NU6.3 activation”; *Ironwood Guide*, §“The Orchard protocol and its two pools”):

- (a) a coinbase transaction has no Orchard-pool Actions;
- (b) every Orchard-pool Action creates its note at the address of the note it consumes, cross-address transfers being disabled;
- (c) the balancing value of the Orchard pool is non-negative.

Read literally, these rules have consequences that no ZIP states. Rule (b) excludes, for a Custom Asset, every created note at an address other than that of the note which the same Action consumes or copies; it thus excludes a transfer of a Custom Asset to such an address, and every padding copy at an address other than that of its Action's created note. Rule (c) is stated for ZEC only, and leaves unaddressed the entry of issued value into a pool that admits no inflow. ZIP 226 and ZIP 227 have not been revised for the pool rules of NU6.3 (ZIP 226 has only given up lead byte $0x03$), ZIP 258 does not mention them, and no ZIP places OrchardZSA notes in the Ironwood pool: ZIP 2005 treats the extension as undeployed ("if that were deployed", Terminology) and requires it to take a new lead byte ("Usage with other proposals requiring note plaintext format changes").

Class: which pool carries OrchardZSA notes is an open problem (Definition 1.2). Live texts contradict one another about it: ZIP 258 states that no new value may enter the Orchard pool, while ZIP 227 adds the commitments of issued notes to the note commitment tree (ZIP 227, "Addition to the Note Commitment Tree") that ZIP 226 places in that pool, and no live text places the notes elsewhere. The later sections state each construction as ZIP 226 and ZIP 227 state it, and recall this remark where a consequence falls: at Split Inputs, at reference notes and at the order of the note commitment tree.

1.4 Requirements on shielded assets

Definition 1.8 (Requirements Z1 to Z5). A protocol for shielded assets meets the following requirements. Each refines one of the *Ironwood Guide*'s Definition "Requirements R1 to R5", or is new (ZIP 227, Requirements; ZIP 226, "Privacy Implications", "Rationale for Value Commitment" and "Backward Compatibility").

- Z1. *Per-Asset conservation*. Requirement R4 holds Asset by Asset: in every valid bundle, for each Asset, the value consumed equals the value created plus the value that leaves the pool, burnt value counted as leaving.
- Z2. *No counterfeiting across Assets*. No transaction creates value of one Asset from value of another Asset, or from nothing; value of a Custom Asset enters only by issuance.
- Z3. *Supply integrity*. The amount of a Custom Asset in circulation equals the amount issued minus the amount burnt, can be tracked publicly, and stops growing once its issuer finalises the Asset.
- Z4. *Issuance authority*. Only the holder of an issuer's key issues or finalises that issuer's Assets.
- Z5. *Hiding of the Asset type*. A shielded transfer reveals which Assets it moves no more than burn and issuance publish. This refines the hiding parts of R1 and R4.

Two design requirements are recorded with Z2. First, the identification of an Asset should be unique among all shielded pools, and different issuer keys should not produce the same Asset Identifier (ZIP 227, Requirements, where "should" is lower-case). Second, the identity of an Asset maps to a point of the Pallas group that serves as a private value base inside the Action proof (ZIP 226, Overview, "Rationale for Value Commitment" and "Value Commitment Correctness").

Table 2 names, for each requirement, the result of this volume that discharges it. The checks that serve each requirement are mapped in Table 6 of “Verification of an OrchardZSA transaction” (§7.4), where all of them are defined.

Table 2. Requirements and the results that discharge them. Each result states its assumptions and the premises on which it is conditional (Definition 1.3).

Requirement	Result
Z1, Z2	Theorem 9.5, “No counterfeiting across Assets”
Z3	Theorem 9.7, “Supply integrity”; Proposition 7.8, “Finalised supply never grows”
Z4	Theorem 9.8, “Issuance authority”
Z5	Theorem 9.11, “Privacy of the Asset type”

2 Asset identity

This section constructs the identity of an Asset in dependency order: the issuance key pair, the issuance authorisation signature scheme and the issuer identifier (§2.1); the asset description and the Asset Identifier (§2.2); the Asset Digest, the Asset Base and its byte encoding (§2.3); and the Asset Base of the Native Asset (§2.4). Every construction of the section is taken from ZIP 227 and ZIP 226, both Draft. The results of the section separate Assets computationally: distinct pairs of issuer and description yield distinct Asset Identifiers and distinct Asset Bases, and no Asset Base of a Custom Asset coincides with a fixed generator of the Orchard protocol, except with negligible probability.

Throughout, $\mathbb{B}^{\mathbb{Y}[k]}$ denotes the set of byte strings of length k and $\mathbb{B}^{\mathbb{Y}[N]}$ the set of byte strings of arbitrary length; $\parallel$ denotes concatenation. For a 16-byte personalisation P and a byte string x , $\text{BLAKE2b-256}(P, x)$ and $\text{BLAKE2b-512}(P, x)$ denote unkeyed BLAKE2b with personalisation P , input x and an output of 32 and 64 bytes respectively (protocol specification, §“BLAKE2 Hash Functions”). The former is the function h of the *Ironwood Guide*, §“Transaction digests and signatures”.

2.1 Issuance keys and the issuer identifier

Definition 2.1 (Issuance key pair). The *issuance authorising key* is a byte string $\text{isk} \in \mathbb{B}^{\mathbb{Y}[32]}$. The *issuance validating key* is $\text{ik} := \text{DerivePublic}(\text{isk})$, with DerivePublic the algorithm of Definition 2.2; it lies in $\mathbb{B}^{\mathbb{Y}[32]} \cup \{\perp\}$, and a key pair is usable only when $\text{ik} \neq \perp$. The key isk authorises the issuance of Asset Identifiers by one issuer and is used only by that issuer; the key ik validates those issuances. Neither key is used for spend authorisation, which remains with the Orchard spend authorising key (ZIP 227, “Issuance Keys” and “Rationale”). Class: specified.

Definition 2.2 (Issuance authorisation signature scheme). The *issuance authorisation signature scheme* IssueAuthSig is a digital signature scheme in the sense of the *Crypto Guide*’s Definition “Digital signature scheme” (protocol specification, §“Signature”), instantiated as BIP-

340 Schnorr signatures over secp256k1 with the standard secp256k1 parameters (*Crypto Guide*, §“The transparent layer’s schemes: ECDSA and BIP-340 over secp256k1”, Construction “BIP-340 Schnorr over secp256k1, schematically”; cited, not re-derived). Let n be the order of the secp256k1 group. The types are

$$\begin{aligned} \text{Message} &= \mathbb{B}^{\mathbb{Y}[N]}, & \text{Signature} &= \mathbb{B}^{\mathbb{Y}[64]} \cup \{\perp\}, \\ \text{Public} &= \mathbb{B}^{\mathbb{Y}[32]} \cup \{\perp\}, & \text{Private} &= \mathbb{B}^{\mathbb{Y}[32]}, \end{aligned}$$

a public key being the 32-byte big-endian x -coordinate of BIP 340. The algorithms are the following.

1. The algorithm $\text{DerivePublic}(\text{isk})$ returns the BIP-340 public key of isk , or $\perp$ exactly when BIP-340 key derivation fails, that is, when isk read as a big-endian integer is 0 or at least n .
2. The algorithm $\text{Sign}(\text{isk}, M)$ runs BIP-340 signing on M under isk with auxiliary data $a := [\text{0x00}]^{32}$ and returns its output $\sigma \in \mathbb{B}^{\mathbb{Y}[64]}$, or $\perp$ if it fails. With a fixed, signing is a deterministic function of (isk, M) .
3. The algorithm $\text{Validate}(\text{ik}, M, \sigma)$ returns 0 if $\sigma = \perp$; otherwise it returns 1 if BIP-340 verification of σ on M under ik succeeds, and 0 if not.

ZIP 227 writes the verification key in Validate as key , although the parameter is named ik ; the reading ik is the only one that type-checks. Batch verification MAY be used; precomputation MAY be used if and only if it produces equivalent results (ZIP 227, “Issuance Authorization Signature Scheme”, “Orchard ZSA Issuance Authorization Signature Scheme”, “Derivation of issuance validating key” and “Issuance Authorization Signing and Validation”). Class: specified.

Assumption 2.3 (Unforgeability of the issuance signature). The scheme IssueAuthSig , with key generation that samples isk uniformly from $\mathbb{B}^{\mathbb{Y}[32]}$ conditioned on $\text{DerivePublic}(\text{isk}) \neq \perp$, and with $H_{\text{BIP0340}/\text{challenge}}$ and $H_{\text{BIP0340}/\text{nonce}}$ modelled as independent random oracles, is existentially unforgeable under chosen-message attack (*Crypto Guide*, Definition “Existential unforgeability under chosen-message attack”).

The assumption is justified, not proved. The *Crypto Guide*’s Theorem “EUF-CMA security of Schnorr signatures in the ROM” (§“Security in the random oracle model and the forking lemma”) proves the property for Schnorr signatures under the discrete-logarithm assumption, and that volume’s remark following Construction “BIP-340 Schnorr over secp256k1, schematically” carries the proof to BIP-340 with the tagged challenge hash $H_{\text{BIP0340}/\text{challenge}}$ modelled as a random oracle. With a fixed, the nonce is $H_{\text{BIP0340}/\text{nonce}}(t \parallel \text{ik} \parallel M)$ for the secret $t := d \oplus H_{\text{BIP0340}/\text{aux}}(a)$, where d is the BIP-340 secret scalar of isk , fixed per key. With $H_{\text{BIP0340}/\text{nonce}}$ also modelled as a random oracle, independent of the challenge oracle, the nonce is a fresh uniform value per message to any party ignorant of t , and the proof carries over as for deterministic nonces (*Crypto Guide*, Construction “EdDSA, schematically”).

Definition 2.4 (Issuer identifier and signature encodings). For $ik \neq \perp$, the *validating-key encoding* is $ik_encoding := [0x00] \parallel ik$, of 33 bytes, and the *issuer identifier* is $issuer := ik_encoding$. For $\sigma \neq \perp$, the *signature encoding* is $issueAuthSig := [0x00] \parallel \sigma$, of 65 bytes. In both encodings the initial byte names the signature scheme and MUST be $0x00$, which denotes BIP 340 (ZIP 227, “Derivation of issuance validating key”, “Issuance Authorization Signing and Validation”). The map $ik \mapsto issuer$ is injective. Note owners and validators use the identifier to associate an Asset with its issuer; ZIP 227, “Issuer Identifier”, records that the equality $issuer = ik_encoding$ may cease to hold once key rotation is specified. Class: specified; its carrier is not.

The encoding $ik_encoding$ occurs only in the issuer field of the Issuance Bundle of “Issuance Actions and Issuance Bundles” (§6.2). No transaction format carrying that field is specified.

2.2 Asset descriptions and Asset Identifiers

Definition 2.5 (Asset description). An *asset description* $asset_desc$ is a byte string chosen by the issuer, carrying any information about the issuance. ZIP 227 requires it to be non-empty and states that it SHOULD be a well-formed UTF-8 code-unit sequence according to Unicode 15.0.0 or later; no maximum length is imposed (ZIP 227, “ZIP 227 Asset Identifiers”). Only its hash $assetDescHash$ (Definition 2.6) is visible to consensus, so neither non-emptiness nor the UTF-8 condition can be enforced by consensus: both are issuer-side conventions. Within the scope of one issuer, distinct Assets are obtained from distinct descriptions. Class: specified.

Definition 2.6 (Asset Identifier and its encoding). For an issuer identifier $issuer \in \mathbb{B}^{\mathbb{Y}[33]}$ and an asset description $asset_desc$, let

$$\begin{aligned} assetDescHash &:= \text{BLAKE2b-256}(\text{ZSA-AssetDescCRH}, asset_desc) \in \mathbb{B}^{\mathbb{Y}[32]}, \\ AssetId &:= (issuer, assetDescHash), \\ \text{EncodeAssetId}(AssetId) &:= [0x00] \parallel issuer \parallel assetDescHash \in \mathbb{B}^{\mathbb{Y}[66]}. \end{aligned}$$

The pair $AssetId$ is the *Asset Identifier* and $\text{EncodeAssetId}(AssetId)$ its canonical encoding (ZIP 227, “ZIP 227 Asset Identifiers”). The leading byte of the encoding is a version byte, reserving room for other issuance protocols; it is distinct in role from the scheme byte that begins issuer, so every current encoding begins with two $0x00$ bytes of different meaning. The encoding is injective, since its three components have the fixed lengths 1, 33 and 32 bytes. Class: specified.

Proposition 2.7 (Uniqueness of Asset Identifiers). Consider two distinct pairs of issuer identifier and asset description, $(issuer, asset_desc)$ and $(issuer', asset_desc')$, with Asset Identifiers $AssetId$ and $AssetId'$.

1. If $issuer \neq issuer'$, then $AssetId \neq AssetId'$.
2. If $issuer = issuer'$, then $asset_desc \neq asset_desc'$, and $AssetId = AssetId'$ holds exactly when the two descriptions collide under $\text{BLAKE2b-256}(\text{ZSA-AssetDescCRH}, \cdot)$.

Hence, under the Ironwood Guide’s Assumption “Collision resistance of BLAKE2b”, no efficient algorithm outputs two distinct pairs of issuer and description with equal Asset Identifiers, or with equal encodings `EncodeAssetId`, except with negligible probability. Status: proved here for the specified construction; conditional on no unspecified object.

Proof. The issuer identifier is a component of `AssetId`, so distinct issuers give distinct pairs, which is claim 1. For one issuer the descriptions differ, and equal identifiers mean equal values `assetDescHash` on distinct descriptions, a BLAKE2b-256 collision under one personalisation; this is claim 2. An algorithm that outputs such a collision outputs two distinct inputs $(P, x) \neq (P, x')$ of h with equal values, which the *Ironwood Guide*’s Assumption “Collision resistance of BLAKE2b” (§“Transaction digests and signatures”) excludes except with negligible probability. Equal encodings imply equal identifiers by the injectivity recorded in Definition 2.6. $\square$

The requirement of ZIP 226, “Asset Identifiers”, that every new Asset have a new and unique Asset Identifier is met by the choice of a new description. ZIP 227, “Rationale”, argues the separation of issuers informally; that argument is a rationale of the ZIP, whose class is designed but unspecified, and Proposition 2.7 supplies its proof. Since the proposition separates Assets only through the issuer and any issuer may reuse any description, ZIP 227, “ZIP 227 Asset Identifiers”, requires that wallets MUST NOT display `asset_desc` alone as the name of an Asset (see also ZIP 227, “Displaying Asset Identifier information to users”).

2.3 Asset Digests and Asset Bases

OrchardZSA adds three inputs of `GroupHash` to those of the Orchard protocol, which the *Ironwood Guide* lists in its table of §“GroupHash, domain separation, and nothing-up-my-sleeve generators”. Table 3 lists them as strings, each with the subsection that uses it (ZIP 227, “OrchardZSA Asset Bases”; ZIP 226, “Split Notes” and “Note Structure and Commitment”). The objects they define are constructed at their uses.

Table 3. Group-hash inputs of OrchardZSA, in addition to those of the Orchard protocol. The subsections are “Asset Digests and Asset Bases”, “Nullifiers of OrchardZSA notes” and “Note commitments”. The commitment to a Custom Asset note reuses the blinding input $(z.cash:Orchard-NoteCommit-r, \epsilon)$ and the `Sinsemilla` table entries of the Orchard protocol, so they add no row. The table lists input strings only.

Domain separator D	Message M	Use	Subsection
<code>z.cash:OrchardZSA</code>	64-byte Asset Digest	Asset Base of a Custom Asset	§2.3
<code>z.cash:Orchard</code>	<code>L</code>	additional term of a split nullifier	§4.2
<code>z.cash:SinsemillaQ</code>	<code>z.cash:ZSA-NoteCommit-M</code>	hash base, Custom Asset note commitment	§3.2

Lemma 2.8 (Distinctness of the OrchardZSA group-hash inputs). *The input pairs of Table 3 are pairwise distinct and distinct from every input pair of the Ironwood Guide’s table of §“GroupHash, domain separation, and nothing-up-my-sleeve generators”. Hence, under the Ironwood Guide’s Assumption “GroupHash as a random oracle”, the values of GroupHash at all these pairs are independent uniform points of $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$, and, under that assumption and the Ironwood Guide’s Assumption “Discrete logarithms on Pallas”, that volume’s Lemma*

“Relations among fixed generators” applies to any polynomial collection of them together with the Orchard generators. Status: proved here; conditional on no unspecified object.

Proof. The separator `z.cash:OrchardZSA` occurs in no row of the Orchard table. Under `z.cash:Orchard` the Orchard table has only the messages `G` and `K`, both distinct from `L`. Under `z.cash:SinsemillaQ` it has only the messages `z.cash:Orchard-NoteCommit-M`, `z.cash:Orchard-CommitIvk-M` and `z.cash:Orchard-MerkleCRH`, all distinct from `z.cash:ZSA-NoteCommit-M`. The three new pairs have pairwise distinct separators or messages. The map from (D, M) to the input hashed by `GroupHash` is injective, as the *Ironwood Guide*’s Remark “Domain separation” records, so distinct pairs are distinct oracle inputs; the random-oracle assumption then gives independent uniform values, and the cited lemma applies verbatim, its hypothesis being only that the inputs are distinct. $\square$

The *Ironwood Guide*’s Assumption “Collision resistance of BLAKE2b” concerns the 32-byte output only. The Asset Digest is a 64-byte output, hence a separate assumption.

Assumption 2.9 (Collision resistance of BLAKE2b-512). No efficient algorithm outputs, except with negligible probability, two inputs $x \neq x'$ with $\text{BLAKE2b-512}(P, x) = \text{BLAKE2b-512}(P, x')$ under one 16-byte personalisation P , in the sense of the *Crypto Guide*’s Definition “Collision resistance” (§“Security notions: preimage, second-preimage, and collision resistance”).

Construction 2.10 (Asset Digest and Asset Base). For the Asset Identifier `AssetId` of a Custom Asset, the *Asset Digest* is

$$\text{AssetDigest}_{\text{AssetId}} := \text{BLAKE2b-512}(\text{ZSA-Asset-Digest}, \text{EncodeAssetId}(\text{AssetId})) \in \mathbb{B}^{\mathbb{Y}[64]}.$$

For $D \in \mathbb{B}^{\mathbb{Y}[64]}$ let

$$\text{ZSAValueBase}(D) := \text{GroupHash}(\text{z.cash:OrchardZSA}, D) \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}),$$

with `GroupHash` the *Ironwood Guide*’s Construction “GroupHash on Pallas” (protocol specification, §“Group Hash into Pallas and Vesta”); no cofactor is cleared. The separator gives a string DST within the 255-byte bound of that construction. The *Asset Base* of the Asset is

$$\text{AssetBase}_{\text{AssetId}} := \text{ZSAValueBase}(\text{AssetDigest}_{\text{AssetId}})$$

(ZIP 227, “Asset Digests”, “Asset Bases” and “OrchardZSA Asset Bases”). The subscript is dropped when the identifier is clear. The whole chain, from issuer and description to Asset Base, is

$$\begin{aligned} \text{asset_desc} &\xrightarrow{\text{BLAKE2b-256}(\text{ZSA-AssetDescCRH}, \cdot)} \text{assetDescHash}, \\ (\text{issuer}, \text{assetDescHash}) = \text{AssetId} &\xrightarrow{\text{EncodeAssetId}} \text{EncodeAssetId}(\text{AssetId}), \\ \text{EncodeAssetId}(\text{AssetId}) &\xrightarrow{\text{BLAKE2b-512}(\text{ZSA-Asset-Digest}, \cdot)} \text{AssetDigest}, \\ \text{AssetDigest} &\xrightarrow{\text{ZSAValueBase}} \text{AssetBase}. \end{aligned}$$

Class: specified.

The Asset Identifier is global across the shielded protocols that support Zcash Shielded Assets; the Asset Base is per protocol, the map ZSAValueBase being instantiated per protocol, here for OrchardZSA (ZIP 227, “Asset Identifier”, “Asset Bases”).

Let $\mathcal{E}^* := \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) \setminus \{\mathcal{O}\}$. ZIP 226, “Note Structure and Commitment”, types the Asset Base of a note in $\mathcal{E}^*$, as “a valid group element that is not the identity”; ZIP 227 types the Asset Base of an Issue Note (“Issue Note”) and the domain of the issuance state (“Global Issuance State”) likewise. The codomain of GroupHash, by contrast, is the whole group.

Lemma 2.11 (Non-identity of derived Asset Bases). *Under the Ironwood Guide’s Assumption “GroupHash as a random oracle”, for every $D \in \mathbb{B}^{\mathbb{Y}[64]}$,*

$$\Pr[\text{ZSAValueBase}(D) = \mathcal{O}] = 1/p_{\text{Vesta}},$$

and an algorithm that makes q queries to the oracle finds an Asset Identifier whose Asset Base is $\mathcal{O}$ with probability at most q/p_{Vesta} , the inputs of its output being counted among the queries. Status: proved here; conditional on no unspecified object.

Proof. In the model each value is uniform on the Pallas group, of order p_{Vesta} (Ironwood Guide, §“Fields, groups, and encodings”), so it is $\mathcal{O}$ with probability $1/p_{\text{Vesta}}$. A union bound over the q queries gives the second claim. $\square$

Remark 2.12 (Identity output of the Asset Base derivation). The map ZSAValueBase has no case for the identity, and ZIP 227 states no rule for an issuance whose derived Asset Base is $\mathcal{O}$. The typing in $\mathcal{E}^*$ of the Asset Base of an Issue Note and of the domain of the issuance state makes such an issuance unconstructible, hence invalid. Class: specified; the rejection follows from the normative typing in $\mathcal{E}^*$, and ZIP 227 states no separate rule. By Lemma 2.11 the event has negligible probability.

Proposition 2.13 (Separation of Asset Bases). *Let the Ironwood Guide’s Assumption “GroupHash as a random oracle” hold, and consider an algorithm that makes q queries to GroupHash, the inputs of its output being counted among the queries.*

1. *The map $\text{AssetId} \mapsto \text{AssetBase}_{\text{AssetId}}$ is not injective; “unique” in ZIP 226, “Note Structure and Commitment”, means only that it is deterministic.*
2. *If the algorithm outputs $\text{AssetId} \neq \text{AssetId}'$ with equal Asset Bases, then either it has found a collision of*

$$\text{BLAKE2b-512}(\text{ZSA-Asset-Digest}, \cdot)$$

on distinct encodings, or it is in an event of probability at most $q^2/(2p_{\text{Vesta}})$.

3. *It outputs an Asset Identifier whose Asset Base equals one of t fixed generators, among them V^{Orchard} , R^{Orchard} and K^{Orchard} , each the value of GroupHash at an input of the Orchard table or of Table 3 other than the first row, with probability at most qt/p_{Vesta} .*

Hence, under Assumption 2.9, the Ironwood Guide’s Assumptions “Collision resistance of BLAKE2b” and “GroupHash as a random oracle”, and by Proposition 2.7, distinct pairs

(issuer, asset_desc) yield distinct Asset Bases, and no Asset Base of a Custom Asset is an Orchard generator, except with negligible probability. Status: proved here for the specified chain; conditional on no unspecified object.

Proof. Claim 1. The map is defined on all pairs in $\mathbb{B}^{\mathbb{Y}[33]} \times \mathbb{B}^{\mathbb{Y}[32]}$, of which there are more than 2^{256} , and takes values in a group of prime order $p_{\text{Vesta}} < 2^{255}$; it is therefore not injective. It is a composition of deterministic maps.

Claim 2. By the injectivity of EncodeAssetId (Definition 2.6), distinct identifiers have distinct encodings; equal digests of distinct encodings are a BLAKE2b-512 collision under ZSA-Asset-Digest. Otherwise the digests are distinct, hence distinct oracle inputs under `z.cash:OrchardZSA`, whose values are independent uniform points in the model. Among q queried inputs two share a value with probability at most $\binom{q}{2} / p_{\text{Vesta}} \leq q^2 / (2p_{\text{Vesta}})$.

Claim 3. By Lemma 2.8 every input under `z.cash:OrchardZSA` differs from the inputs of the t generators, so each queried Asset Base is uniform and independent of them, and equals a given one with probability $1/p_{\text{Vesta}}$. A union bound over the q queries and the t generators gives qt/p_{Vesta} .

The final sentence composes claims 2 and 3 with Proposition 2.7: distinct pairs of issuer and description yield distinct identifiers except with negligible probability, distinct identifiers yield distinct Asset Bases except with negligible probability by claim 2 and Assumption 2.9, and the bounds of claims 2 and 3 are negligible for polynomial q and t . $\square$

Definition 2.14 (Byte encoding of an Asset Base). The *byte encoding* of an Asset Base is the 32-byte string

$$\text{asset_base} := \text{LEBS2OSP}_{256}(\text{AssetBase}^*) \in \mathbb{B}^{\mathbb{Y}[32]},$$

with LEBS2OSP_{256} and the star encoding those of the *Ironwood Guide*, §“Fields, groups, and encodings”, Definitions “Integer and byte encodings” and “Star encoding” (ZIP 226, “Note Structure and Commitment”). It is typed as a byte string throughout, although ZIP 226 also types it as a bit string in its clause on the note plaintext. The encoding is canonical on the Pallas group, by the canonicity of the star encoding established after the same Definition; ZIP 226 notes that canonicity may fail for future shielded protocols. Class: specified; its carrier is not.

The placements of `asset_base` in a note plaintext and in the entries of a burn set are encodings of ZIP 230, which is Withdrawn; they are not specified.

Table 4. Sizes along the identity chain, in bytes. ZIP 227 publishes no test vectors, and no worked instance is given.

Object	Bytes	Defined in	Source
<code>ik</code>	32	Definition 2.2	ZIP 227, “Derivation of issuance validating key”
<code>issuer = ik_encoding</code>	33	Definition 2.4	ZIP 227, “Issuer Identifier”
<code>issueAuthSig</code>	65	Definition 2.4	ZIP 227, “Issuance Authorization Signing and Validation”
<code>assetDescHash</code>	32	Definition 2.6	ZIP 227, “ZIP 227 Asset Identifiers”
<code>EncodeAssetId(AssetId)</code>	66	Definition 2.6	ZIP 227, “ZIP 227 Asset Identifiers”
<code>AssetDigest</code>	64	Construction 2.10	ZIP 227, “Asset Digests”
<code>asset_base</code>	32	Definition 2.14	ZIP 226, “Note Structure and Commitment”

Every formula of the chain is specified, in Draft ZIP text: the issuance keys and `IssueAuthSig`, the identifier issuer, the hash `assetDescHash`, the identifier `AssetId` and its encoding, the Asset Digest, the Asset Base, its encoding `asset_base` and the Native Asset Base of §2.4. The identity case is specified, by typing (Remark 2.12). The encodings that would carry the chain on chain belong to the carrier.

2.4 The Native Asset Base

Definition 2.15 (Native Asset Base). The Native Asset, ZEC on Mainnet and TAZ on Testnet (ZIP 227, “Terminology”), has no Asset Identifier under ZIP 227 and does not pass through the chain of Construction 2.10. Its Asset Base is fixed as

$$\text{AssetBase} := V^{\text{Orchard}} = \text{GroupHash}(\text{z.cash:orchard-cv}, v),$$

the value base of the *Ironwood Guide*’s Definition “Net value commitment” (§“Value commitments”), where V^{Orchard} and its companion $R^{\text{Orchard}} = \text{GroupHash}(\text{z.cash:orchard-cv}, r)$ are constructed; they are cited, not re-derived. ZIP 226, “Asset Identifiers” and “Value Commitment”, writes the definition for ZEC; its burn rules (“Burn Mechanism”, “Additional Consensus Rules for the assetBurn set”) identify the Native Asset, ZEC or TAZ, with the base V^{Orchard} . Class: specified.

The value commitment and the note commitment of a Native Asset note are computed by the Orchard formulas. Two things differ: the derivation of `rcm` takes the OrchardZSA lead byte (Remark 3.6), and the note plaintext carries `asset_base` (Definition 3.7; ZIP 226, “Note Structure and Commitment”). The value commitment of ZIP 226, “Value Commitment”, is $[v^{\text{net}}] \text{AssetBase} + [\text{rcv}] R^{\text{Orchard}}$, which at the base V^{Orchard} is the Orchard net value commitment $[v^{\text{net}}] V^{\text{Orchard}} + [\text{rcv}] R^{\text{Orchard}}$ of the cited Definition. The note commitment of ZIP 226, “Note Structure and Commitment”, is by its case split the Orchard note commitment when the Asset Base is V^{Orchard} ; the case split is stated in “Note commitments” (§3.2). The base V^{Orchard} is a group-hash output under a separator distinct from `z.cash:orchardZSA`, so by Proposition 2.13 no Asset Base of a Custom Asset equals it, except with negligible probability.

3 Notes and note commitments

This section extends the Orchard note by one field, the Asset Base (§3.1); extends the note commitment by a case split whose native branch is the Orchard commitment (§3.2); states the derivations of the note’s randomness from its seed and what no live text fixes of them (§3.3); states the field set of the note plaintext (§3.4); and replaces the fixed value base of the net value commitment by the Asset Base (§3.5). It proves hiding and binding of both commitments within and across Assets. Every construction of the section is taken from ZIP 226, which is Draft; the Orchard objects it extends are those of the *Ironwood Guide* and are cited, not restated.

3.1 The OrchardZSA note

Definition 3.1 (OrchardZSA note). An *OrchardZSA note* is a tuple

$$n = (d, \text{pk}_d, v, \text{AssetBase}, \rho, \psi, \text{rcm})$$

of type

$$\begin{aligned} \text{Note}^{\text{OrchardZSA}} := & \{0, 1\}^{88} \times (\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) \setminus \{\mathcal{O}\}) \times \{0, \dots, 2^{64} - 1\} \\ & \times \mathcal{E}^* \times \mathbb{F}_{p_{\text{Pallas}}} \times \mathbb{F}_{p_{\text{Pallas}}} \times \mathbb{F}_{p_{\text{Vesta}}}, \end{aligned}$$

with $\mathcal{E}^* = \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) \setminus \{\mathcal{O}\}$ as in §2.3. It is the note of the *Ironwood Guide*’s Definition “Note” (§“Notes”) with exactly one added field, the *Asset Base* $\text{AssetBase} \in \mathcal{E}^*$ of the note’s Asset: for a Custom Asset the point of Construction 2.10, for the Native Asset the point V^{Orchard} of Definition 2.15. The components $(d, \text{pk}_d, v, \rho, \psi, \text{rcm})$ and their types, with rcm typed in $\mathbb{F}_{p_{\text{Vesta}}}$, are those of the cited definition. ZIP 226 types the added field in $\mathbb{P}^*$, a group element that is neither the identity nor $\perp$ (Table 1). The value v counts units of the note’s Asset; the *Ironwood Guide*’s Definition “Unit of value” (the zatoshi) applies only when $\text{AssetBase} = V^{\text{Orchard}}$ (ZIP 226, “Note Structure and Commitment”; protocol specification, §“Shielded Pools and Notes”). Class: specified.

3.2 Note commitments

Definition 3.2 (OrchardZSA note commitment). Let $n = (d, \text{pk}_d, v, \text{AssetBase}, \rho, \psi, \text{rcm})$ be an OrchardZSA note, $\text{g}_d := \text{DiversifyHash}(d)$, and

$$M(n) := \text{g}_d^* \parallel \text{pk}_d^* \parallel \text{LE}_{64}(v) \parallel \text{LE}_{255}(\rho) \parallel \text{LE}_{255}(\psi)$$

the 1086-bit Orchard message of the *Ironwood Guide*’s Definition “Note commitment”. Let

$$M^{\text{ZSA}}(n) := M(n) \parallel \text{AssetBase}^*,$$

where the 256-bit star encoding AssetBase^* has the bytes `asset_base` of Definition 2.14. The string $M^{\text{ZSA}}(n)$ has the fixed length 1342: 135 chunks of 10 bits with padding 8, against 109 chunks for $M(n)$, within the chunk bound $c = 253$ of the *Ironwood Guide*’s Construction “Sinsemilla hash on Pallas”. With the Orchard blinding base and the hash point

$$\begin{aligned} H_D &:= \text{GroupHash}(\text{z.cash:Orchard-NoteCommit-r}, \varepsilon), \\ \hat{M}^{\text{ZSA}} &:= \text{SinsemillaHashToPoint}(\text{z.cash:ZSA-NoteCommit-M}, M^{\text{ZSA}}(n)), \end{aligned}$$

the *OrchardZSA note commitment* $\text{cm} := \text{NoteCommit}_{\text{rcm}}^{\text{OrchardZSA}}(n)$ of n is defined in two branches. In the *native branch*, $\text{AssetBase} = V^{\text{Orchard}}$, it is

$$\text{cm} := \text{NoteCommit}_{\text{rcm}}(M(n)),$$

with NoteCommit the Orchard commitment of the cited definition. In the *custom branch*, $\text{AssetBase} \neq V^{\text{Orchard}}$, it is

$$\text{cm} := \begin{cases} \hat{M}^{\text{ZSA}} + [\text{rcm}] H_D & \text{if } \hat{M}^{\text{ZSA}} \neq \perp, \\ \perp & \text{otherwise.} \end{cases}$$

In both branches $\text{cm} \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) \cup \{\perp\}$, and the *extracted note commitment* is $\text{cmx} := \text{Extract}_{\mathbb{P}}^{\perp}(\text{cm}) \in \mathbb{F}_{p_{\text{Pallas}}} \cup \{\perp\}$, as for Orchard. The custom branch is not the routed SinsemillaCommit of the *Ironwood Guide*’s Construction “Sinsemilla commitment and short commitment” for any domain: its hash domain is new, while its blinding base is Orchard’s. The hash base $Q(\text{z.cash:ZSA-NoteCommit-M})$ is a row of Table 3. The new domain hashes the single length 1342, as the *Ironwood Guide*’s Remark “Fixed length” requires of every domain (ZIP 226, “Note Structure and Commitment”; protocol specification, §“Sinsemilla commitments”). Class: specified.

In the native branch the Asset Base is not an input of the commitment. Hence the OrchardZSA commitment of a note with $\text{AssetBase} = V^{\text{Orchard}}$ equals, bit for bit, the Orchard commitment of its Orchard components; an Orchard note and a native OrchardZSA note with the same Orchard components have the same cmx; and every leaf of the Orchard-pool note commitment tree created before OrchardZSA remains a valid opening target. Both branches output in $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) \cup \{\perp\}$, so custom-branch commitments enter the same tree. Each statement is immediate from Definition 3.2 (ZIP 226, “Note Structure and Commitment” and “Rationale for Note Commitment”).

Proposition 3.3 (Hiding and binding of OrchardZSA note commitments).

- (a) *Hiding with one output distribution.* Let rcm be uniform on $\mathbb{F}_{p_{\text{Vesta}}}$ and independent of the content. For every content $(d, \text{pk}_d, v, \text{AssetBase}, \rho, \psi)$ whose hash point in its branch is not $\perp$, the commitment cm is uniform on $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$. Hence the distributions of cm and of cmx depend neither on the content nor on the branch, and commitments to ZEC and to Custom Assets are identically distributed.
- (b) *Binding.* Under the Ironwood Guide’s Assumptions “Discrete logarithms on Pallas” and “GroupHash as a random oracle”, no efficient algorithm outputs, except with negligible probability, OrchardZSA notes $n \neq n'$ with $\text{cmx}(n) = \text{cmx}(n') \neq \perp$; in particular none with $\text{AssetBase} = V^{\text{Orchard}} \neq \text{AssetBase}'$, and none whose contents are equal except for the Asset Base.
- (c) *Under the same assumptions, no efficient algorithm outputs, except with negligible probability, a custom-branch content with $\hat{M}^{\text{ZSA}} = \perp$.*

Status (Definition 1.3): proved here for the specified construction; conditional on no unspecified object. Part (a) is stated for uniform rcm and does not cover a derived trapdoor, whose derivation is classified in Remark 3.6.

Proof. Write

$$Q_O := Q(\text{z.cash:Orchard-NoteCommit-M}), \quad Q_Z := Q(\text{z.cash:ZSA-NoteCommit-M}).$$

The generators $Q_O, Q_Z, S(0), \dots, S(1023)$ and H_D are values of GroupHash at pairwise distinct inputs, by Lemma 2.8 and the *Ironwood Guide*'s table of §“GroupHash, domain separation, and nothing-up-my-sleeve generators”.

(a) In both branches $\text{cm} = \hat{M} + [\text{rcm}] H_D$, with the same blinding base $H_D \neq \mathcal{O}$ and a point $\hat{M}$ fixed by the content. The proof of the *Ironwood Guide*'s Lemma “Hiding and binding of SinsemillaCommit” (i) uses only these two facts, the first established for H_D before that volume's Proposition “Hiding and binding of note commitments”; so cm is uniform, whatever the content and the branch. The value cmx is a fixed function of cm .

(b) Step 1. The map from a content to its branch and message is injective except on a diversifier collision: the branch is fixed by whether $\text{AssetBase} = V^{\text{Orchard}}$; in the native branch the Asset Base is determined, and the message $M(n)$ determines the Orchard components; in the custom branch the message $M^{\text{ZSA}}(n)$ determines them and, by the injectivity of the star encoding, the Asset Base. A diversifier collision is excluded as in Step 1 of the proof of the *Ironwood Guide*'s Proposition “Hiding and binding of note commitments” (b). Outside that event, $n \neq n'$ gives distinct branches, or one branch with distinct messages, or one branch and one message with $\text{rcm} \neq \text{rcm}'$.

Step 2. Let $\text{cmx}(n) = \text{cmx}(n') \neq \perp$. Both commitments are points, and by the *Ironwood Guide*'s Lemma “Coordinate extraction identifies opposite points”, $\text{cm}' = \pm \text{cm}$; both hash points are not $\perp$. Three cases remain.

Both native. The Orchard components differ, since the Asset Bases agree, and the *Ironwood Guide*'s Proposition “Hiding and binding of note commitments” (b) applies.

Both custom. The three steps of the *Ironwood Guide*'s Proposition “Collision resistance of the Sinsemilla instances” hold for the message length $l = 1342$ and $n = 135$ chunks. Since $2^{135} \leq 2^e \leq (p_{\text{Vesta}} - 1)/2$, every difference $\chi_j(m) - \chi_j(m')$ of the unfolding in its proof has absolute value below p_{Vesta} , so χ is injective, and the coefficient 2^{136} on Q_Z in the opposite case is non-zero modulo p_{Vesta} . Equal points on distinct messages, or opposite points, therefore give a non-trivial relation among Q_Z , the $S(j)$ and H_D . Equal messages with $\text{rcm} \neq \text{rcm}'$ and equal points give $[\text{rcm} - \text{rcm}'] H_D = \mathcal{O}$, impossible for $H_D \neq \mathcal{O}$ in a group of prime order p_{Vesta} ; opposite points on equal messages are the opposite case above.

Across branches. Let n be native and n' custom, with pieces m and m' of $M(n)$ and $M^{\text{ZSA}}(n')$. Unfolding both hash points gives

$$[2^{109}] Q_O \mp [2^{135}] Q_Z + \sum_j [\chi_j(m) \mp \chi_j(m')] S(j) + [\text{rcm} \mp \text{rcm}'] H_D = \mathcal{O}.$$

The coefficient 2^{109} on Q_O is non-zero modulo p_{Vesta} .

In each case the relation is non-trivial among values of GroupHash at distinct inputs, which the *Ironwood Guide*'s Lemma “Relations among fixed generators” excludes except with negligible probability. The first particular case of (b) is the case across branches; the second is either that case or the case of two custom notes with distinct messages.

(c) Step (2) of the proof of the cited proposition applies to the domain of Q_Z : an operand $\mathcal{O}$ arises only if Q_Z or some $S(j)$ is $\mathcal{O}$, itself a non-trivial relation, and an exceptional case at step i

yields a relation with coefficient $\alpha 2^{i-1}$ on $\mathcal{Q}_Z$, where $\alpha \in \{-1, 1, 2\}$ and $0 < |\alpha 2^{i-1}| \leq 2^{135} < p_{\text{Vesta}}$. The same lemma excludes both. $\square$

ZIP 226 argues the property in rationale only, attributing it to the Sinsemilla hash function (“by definition of the Sinsemilla hash function”, “Rationale for Note Commitment”; see also “Privacy Implications”). The unblinded hash is deterministic and hides nothing; the hiding comes from the term $[\text{rcm}] H_D$. The plaintext equivalence of Orchard and native OrchardZSA notes, which is withdrawn ZIP 230 text, is not used.

Remark 3.4 (Contextual distinguishability of commitments). Commitments are hidden only as group elements. ZIP 226, “Privacy Implications”, states that Orchard note commitments are distinguishable from OrchardZSA note commitments by context, since the two protocols are carried by different transaction versions (in its text, version 5 for Orchard and version 6 for OrchardZSA), whether or not both are active. No current format assigns OrchardZSA a version: version 6 is the format of ZIP 229, “Transaction Format”, which carries Orchard-pool and Ironwood-pool Actions and no ZSA. ZIP 248, “Potential Future Bundle Types”, lists OrchardZSA provisionally as variant 2 of bundle type 3, the Orchard bundle type, whose public variant identifier would likewise separate the two. Within one OrchardZSA bundle, commitments to ZEC and to Custom Assets are identically distributed under a uniform trapdoor (Proposition 3.3(a)); the derived trapdoor, undefined in live text, is not covered (Remark 3.6). The carrier is treated in “The transaction carrier” (§8).

3.3 The note seed and the lead byte

ZIP 226, “Abstract”, is defined relative to the protocol with ZIP 2005 applied, and reads H^{rcm} , H^ψ and the recoverable plaintexts as ZIP 2005 defines them; it calls ZIP 2005 “Orchard Quantum Recoverability”. ZIP 2005 is now “Ironwood Quantum Recoverability” (Proposed). It is active with NU6.3 and merged into the protocol specification (ZIP 258, “ZIP 2005 activation”; ZIP 2005, “Specification Updates”), and it applies its recoverable plaintext format, lead byte $0x03$, only to Ironwood-pool notes (ZIP 229, “Transaction Format”), while ZIP 226 places OrchardZSA notes in the Orchard pool (Remark 1.7). The two premises no longer coincide. Two constructions depend on them: the lead byte, here, and the seed of split nullifiers, at “Nullifiers of OrchardZSA notes” (§4.2). The derivations of ZIP 2005 are constructed in the *Ironwood Guide*, §“The note seed: Ironwood derivations (lead byte $0x03$)”, and are cited, not restated.

Definition 3.5 (Seed derivations of an OrchardZSA note). Let an OrchardZSA note be created with recipient (d, pk_d) , value v , element $\rho \in \mathbb{F}_{p_{\text{Pallas}}}$, note seed rseed uniform on the 32-byte strings, and lead byte leadByte . Let $\underline{\rho} := \text{I2LEOSP}_{256}(\rho)$ and $\text{g}_d := \text{DiversifyHash}(d)$. Then

$$\begin{aligned}\psi &:= \text{ToBase}(\text{PRFexpand}_{\text{rseed}}([\text{0x09}] \parallel \underline{\rho})), \\ \text{rcm} &:= \text{Derive_rcm}_{\text{rseed}}(\text{leadByte}, \text{g}_d^*, \text{pk}_d^*, v, \underline{\rho}, \psi),\end{aligned}$$

where ψ does not depend on the lead byte, and Derive_rcm is defined only for two lead bytes:

$$\text{Derive_rcm}_{\text{rseed}}(\text{leadByte}, \dots) := \begin{cases} \text{ToScalar}(\text{PRFexpand}_{\text{rseed}}([\text{0x05}] \parallel \underline{\rho})) & \text{if } \text{leadByte} = \text{0x02}, \\ \text{the pre_rcm derivation} & \text{if } \text{leadByte} = \text{0x03}, \end{cases}$$

the latter being that of the *Ironwood Guide*’s Construction “Derivations under lead byte 0x03”. Neither input list contains AssetBase. ZIP 226, “Note Structure and Commitment”, requires leadByte to be the placeholder $\{\{ZSALEADBYTE\}\}$ whenever §“Sending Notes (Orchard)” or §“Dummy Notes (Orchard)” of the protocol specification is invoked, directly or indirectly, for an OrchardZSA note. That rule speaks of “the computation of ρ and ψ ”; under the amended specification the lead byte enters only Derive_rcm. The specification admits lead byte 0x02 in the Orchard pool and 0x03 in the Ironwood pool only (protocol specification, §“Note Plaintexts and Memo Fields”), and ZIP 2005, “Usage with other proposals requiring note plaintext format changes”, requires such proposals to use a new lead byte value or values. Classes: the derivation of ψ is specified; the derivation of rcm and the value of the lead byte are classified in Remark 3.6.

Remark 3.6 (Unassigned lead byte). Each part carries one class of Definition 1.2.

- (i) The derivation of ψ is specified.
- (ii) The derivation of rcm is designed but unspecified. A complete design existed: ZIP 226 fixed 0x03 with H^{rcm} before 0x03 passed to ZIP 2005, and the design needs only a lead byte.
- (iii) The value of the lead byte is an open problem. The removed design’s 0x03 is the Ironwood lead byte in live text, a design that conflicts with live text; the placeholders $\{\{ZSALEADBYTE\}\}$ (ZIP 226), $\{\{LEADBYTE\}\}$ (withdrawn ZIP 230) and $\{\{MBLEADBYTE\}\}$ (ZIP 231, “Note plaintext lead byte assignment”) are unassigned.

Until the byte is assigned, the rcm of an OrchardZSA note is undefined in live text, and the *Ironwood Guide*’s Proposition “Hiding under the derived trapdoor” does not transfer. The OrchardZSA Action statement, like the Orchard one, does not check the derivation (*Ironwood Guide*, Remark “What the statement does not check” (ii)), so for the outputs of a transfer the gap binds only sender and recipient. Its consequence for validators is stated at “Issue Notes” (§6.1).

3.4 The note plaintext

Definition 3.7 (OrchardZSA note plaintext). The *OrchardZSA note plaintext* of a created OrchardZSA note carries the fields (leadByte, d , v , rseed, memo) of the *Ironwood Guide*’s Definition “Note plaintext” and, in addition, the 32-byte string asset_base of Definition 2.14 (ZIP 226, “Note Structure and Commitment”). The recipient needs asset_base to recompute cmx by Definition 3.2 and to spend the note. Given an encoding of fixed length and an assigned lead byte, the *Ironwood Guide*’s Constructions “Encryption to the recipient” and “Trial decryption” apply unchanged. Its acceptance checks (§“Trial decryption and note acceptance”) apply with the lead-byte check and the derivation of rcm replaced by those of that lead byte (Remark 3.6), asset_base required to decode to a point of $\mathcal{E}^*$, and the recomputed commitment that of Definition 3.2. Classes: the field set is specified; the encoding is an open problem.

ZIP 226 delegates the encoding to ZIP 230, which is Withdrawn. Its layout

$$\text{leadByte} \parallel d \parallel v \parallel \text{rseed} \parallel \text{asset_base} \parallel K^{\text{memo}}$$

presupposes the memo key K^{memo} of ZIP 231, which is now defined only for version-7 transactions (ZIP 231, “Specification”), and a lead byte that live text assigns otherwise (Remark 3.6). No current ZIP defines an OrchardZSA plaintext encoding. The statement of withdrawn ZIP 230 that Orchard and native OrchardZSA notes need not be distinguished in plaintexts carries no weight; the commitment half of that statement follows from Definition 3.2.

3.5 The per-Asset value commitment

Definition 3.8 (Per-Asset value commitment). Let an Action (*Ironwood Guide*, Definition “Action”) consume an OrchardZSA note of value v^{old} and create an OrchardZSA note of value v^{new} , both with Asset Base $\text{AssetBase} \in \mathcal{E}^*$. Its *net value* is the integer

$$v^{\text{net}} := v^{\text{old}} - v^{\text{new}} \in \{-2^{64} + 1, \dots, 2^{64} - 1\}.$$

For rcv drawn by the *Ironwood Guide*’s Construction “Value-commitment trapdoor”, its *per-Asset value commitment* is

$$\text{cv}^{\text{net}} := \text{ValueCommit}_{\text{rcv}}^{\text{OrchardZSA}}(\text{AssetBase}, v^{\text{net}}) := [v^{\text{net}} \bmod p_{\text{Vesta}}] \text{AssetBase} + [\text{rcv}] R^{\text{Orchard}},$$

with R^{Orchard} and the reading of v^{net} modulo p_{Vesta} as in the *Ironwood Guide*’s Definition “Net value commitment”. For $\text{AssetBase} = V^{\text{Orchard}}$ it equals $\text{ValueCommit}_{\text{rcv}}^{\text{Orchard}}(v^{\text{net}})$. The only change against Orchard is that the fixed-base multiplication by V^{Orchard} becomes a variable-base multiplication by the Action’s Asset Base; the randomness term is unchanged. The condition that controls the base is stated in “The OrchardZSA Action statement” (§4.3) (ZIP 226, “Value Commitment”; protocol specification, §“Homomorphic Pedersen commitments (Sapling and Orchard)”). Class: specified.

Lemma 3.9 (Hiding and binding of per-Asset value commitments).

- (i) *Perfect hiding.* For every $B \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$ and every integer v , with rcv uniform on $\mathbb{F}_{p_{\text{Vesta}}}$ and independent of (B, v) , the point $[v \bmod p_{\text{Vesta}}] B + [\text{rcv}] R^{\text{Orchard}}$ is uniform on $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$.
- (ii) *Binding across bases.* Call a base admissible if it is given as the value of `GroupHash` at the input $(z.\text{cash}:\text{Orchard-cv}, v)$ or at $(z.\text{cash}:\text{OrchardZSA}, D)$ for a byte string D . Under the *Ironwood Guide*’s Assumptions “Discrete logarithms on Pallas” and “GroupHash as a random oracle”, no efficient algorithm outputs, except with negligible probability, two families $((B_i, v_i))_i$ and $((B'_j, v'_j))_j$ of polynomial size, of admissible bases with integers, and trapdoors rcv, rcv' , with

$$\sum_i [v_i] B_i + [\text{rcv}] R^{\text{Orchard}} = \sum_j [v'_j] B'_j + [\text{rcv}'] R^{\text{Orchard}}$$

while the base-value vectors differ: for some group-hash input x , the sum of the v_i at bases with input x and the sum of the v'_j at bases with input x differ modulo p_{Vesta} . For one pair each, (B, v) and (B', v') with inputs x and x' , equal commitments force $x = x'$, hence $B = B'$, and $v \equiv v'$, or else $v \equiv v' \equiv 0 \pmod{p_{\text{Vesta}}}$.

- (iii) *Binding fails for arbitrary bases. For $B \neq \mathcal{O}$, $v \neq 0$ and $v' \neq 0$, $v \pmod{p_{\text{Vesta}}}$, the point $B' := [v v'^{-1}] B$ satisfies $B' \notin \{\mathcal{O}, B\}$ and $[v] B = [v'] B'$, so the two openings (B, v) and (B', v') share every trapdoor and every commitment. Hence the Action statement must control the base.*

Status (Definition 1.3): proved here; conditional on no unspecified object.

Proof. (i) The proof of the *Ironwood Guide*'s Lemma "Hiding and binding of net value commitments" (i) uses only $R^{\text{Orchard}} \neq \mathcal{O}$ and the prime order p_{Vesta} of the group, not the value base: $[\text{rcv}] R^{\text{Orchard}}$ is uniform, and so is its translate by the fixed point $[v \pmod{p_{\text{Vesta}}}] B$.

(ii) Let P_x denote the value of GroupHash at input x , and c_x the difference, modulo p_{Vesta} , of the two sums of values at input x . Grouping both sides by input, a double opening gives

$$\sum_x [c_x] P_x + [\text{rcv} - \text{rcv}'] R^{\text{Orchard}} = \mathcal{O}$$

with some $c_x \neq 0$, the relation of the proof of the *Crypto Guide*'s Theorem "Properties of the vector commitment" (§"Pedersen vector commitments"). The inputs x are pairwise distinct by construction, and distinct from the input $(z.\text{cash}:\text{Orchard-cv}, r)$ of R^{Orchard} : the input of V^{Orchard} differs from it in the message, and every input under $z.\text{cash}:\text{OrchardZSA}$ differs from both by Lemma 2.8. The relation is therefore non-trivial among values of GroupHash at distinct inputs, and the *Ironwood Guide*'s Lemma "Relations among fixed generators" excludes it except with negligible probability. For one pair each, $x \neq x'$ gives $c_x = v$ and $c_{x'} = -v'$, non-trivial unless $v \equiv v' \equiv 0$; and $x = x'$ gives $c_x = v - v'$, non-trivial unless $v \equiv v'$.

(iii) Direct computation: $[v'] B' = [v' v v'^{-1}] B = [v] B$; the scalar $v v'^{-1}$ is neither 0 nor 1 modulo p_{Vesta} , and B generates the group of prime order p_{Vesta} , so $B' \neq \mathcal{O}$ and $B' \neq B$. $\square$

ZIP 226, "Rationale for Value Commitment", states the change of value base without a binding argument; Lemma 3.9 supplies it for bases of the admissible form, and part (iii) shows that the form cannot be dropped.

4 OrchardZSA Actions and Action Groups

This section shows that the membership exemption of Orchard dummy notes is unsound once the value base of an Action is a witness, and defines the Split Inputs that replace dummy consumed notes for Custom Assets (§4.1); constructs the split nullifier and proves its independence from the nullifier of the copied note (§4.2); states the OrchardZSA Action statement as a delta on the Orchard one, with named conditions (§4.3); states the assumptions on its proof and proves that a Split Input requires the spend authority of the copied note (§4.4); and defines the Action Group (§4.5). The objects of the section are taken from ZIP 226 and ZIP 228, both Draft, with the rules of ZIP 227 that bound the number of Action Groups. The Orchard objects they modify are those of the *Ironwood Guide* and are cited, not restated.

4.1 Dummy notes and Split Inputs

By the *Ironwood Guide*'s Definition "Dummy notes" (§"Actions, bundles, and dummy notes"), an Action whose consumed side is unused consumes a dummy note of value 0: under a fresh

key or, when `enableCrossAddress = 0` and the created note is real, at the expanded receiver of the created note and under the key from which that receiver derives. Condition A3 of that volume’s Definition “Orchard Action statement”, namely $v^{\text{old}} \cdot (\text{root} - \text{rt}) = 0$ or some evaluation of `SinsemillaHash` in the fold returns $\perp$, exempts every consumed note of value 0 from membership (protocol specification, §“Dummy Notes (Orchard)” and §“Action Statement (Orchard)”). In the Orchard protocol the exemption is sound for value: condition A4 multiplies the value by the fixed generator V^{Orchard} (*Ironwood Guide*, Definition “Net value commitment”), so a consumed note of value 0 contributes $[0] V^{\text{Orchard}} = \mathcal{O}$, whatever path it fails to prove. The per-Asset value commitment of Definition 3.8 multiplies by a witnessed Asset Base instead. The next proposition shows that the exemption, kept unchanged, then admits a balance violation.

Proposition 4.1 (Unsoundness of the dummy exemption under a witnessed base). *Let $\mathcal{R}^-$ be the relation obtained from the Ironwood Guide’s Definition “Orchard Action statement” by adding an auxiliary input $A \in \mathcal{E}^*$ and making three replacements: in A1 and A2, the commitment `NoteCommit` is replaced by `NoteCommit`^{OrchardZSA} of Definition 3.2 with Asset Base A for both notes; in A4, the net value commitment becomes*

$$\text{cv}^{\text{net}} = [v^{\text{old}} - v^{\text{new}}] A + [\text{rcv}] R^{\text{Orchard}},$$

as in Definition 3.8; condition A3 and conditions A5 to A10 are unchanged, so a consumed note of value 0 is exempt from membership whatever A is. Then for every $v \in \{1, \dots, 2^{64} - 1\}$ there is a bundle of two Actions whose witnesses satisfy $\mathcal{R}^-$, with balancing value 0 and no burn, whose binding signature verifies, and which creates a ZEC note, of Asset Base V^{Orchard} , of value v while every consumed note has value 0:

Action 1 witnesses $A := [-1] V^{\text{Orchard}}$, $v^{\text{old}} = 0$ and $v^{\text{new}} = v$;

Action 2 witnesses $A := V^{\text{Orchard}}$, a dummy consumed note and $v^{\text{new}} = v$.

More generally, a witnessed base $A = \sum_i [a_i] B_i$ with known coefficients a_i over Asset Bases B_i contributes $\sum_i [a_i v] B_i$ for a committed value v , so a value committed on A cancels values committed on the B_i . This is the attack of ZIP 226, “Rationale for Split Notes”: without the enforcement “the prover could input a multiple (or linear combination) of an existing Asset Base, and thereby attack the network by overflowing the ZEC value balance and hence counterfeiting ZEC funds”. Status (Definition 1.3): proved here; ZIP 226 states the attack in words only.

Proof. The point $[-1] V^{\text{Orchard}}$ is not $\mathcal{O}$, since V^{Orchard} has prime order p_{Vesta} , so Action 1 is well typed. Both consumed notes have value 0, so A3 holds for both Actions without a path, and A8 holds for either value of `enableSpends`; A9 holds with `enableOutputs = 1`. The remaining conditions hold when the commitments, the nullifiers and `rk` are computed from the witness by their definitions, under a key of the prover; each consumed note is placed at the expanded receiver of the note its Action creates, so that A10 holds for either value of `disableCrossAddress`, as in the

Ironwood Guide’s Remark “Meaning of A10”. The net value commitments are

$$\begin{aligned} \text{cv}_1^{\text{net}} &= [0 - v] \left([-1] V^{\text{Orchard}} \right) + [\text{rcv}_1] R^{\text{Orchard}} = [v] V^{\text{Orchard}} + [\text{rcv}_1] R^{\text{Orchard}}, \\ \text{cv}_2^{\text{net}} &= [0 - v] V^{\text{Orchard}} + [\text{rcv}_2] R^{\text{Orchard}}. \end{aligned}$$

With balancing value 0 and no burn, the *Ironwood Guide*’s Definition “Binding validating key” gives

$$\text{bvk} = \text{cv}_1^{\text{net}} + \text{cv}_2^{\text{net}} = [\text{rcv}_1 + \text{rcv}_2] R^{\text{Orchard}},$$

whose discrete logarithm $\text{bsk} = \text{rcv}_1 + \text{rcv}_2$ to the base R^{Orchard} the prover knows, so that volume’s Construction “Binding signature” produces a valid signature. Action 2 creates a ZEC note of value v , and no consumed note carries value. For the general case, $[w] \left(\sum_i [a_i] B_i \right) = \sum_i [a_i w] B_i$ for every integer w , by bilinearity of scalar multiplication, and the same computation applies. $\square$

Definition 4.2 (Split Input and Split Action). Let $B \in \mathcal{E}^*$ with $B \neq V^{\text{Orchard}}$. A *Split Input* of Asset Base B is an OrchardZSA note $n = (d, \text{pk}_d, v, B, \rho, \psi, \text{rcm})$ (Definition 3.1), whose extracted commitment

$$\text{Extract}_p \left(\text{NoteCommit}_{\text{rcm}}^{\text{OrchardZSA}}(n) \right)$$

(Definition 3.2) is a leaf of the note commitment tree under the anchor of the consuming Action, consumed by an Action that sets the witness bit $\text{split_flag} := 1$ of the statement defined in “The OrchardZSA Action statement” (§4.3). A Split Input differs from a spend of n in two respects only: its value is replaced by 0 in the net value commitment, while the commitment still opens with the true value v ; and the Action publishes the split nullifier of “Nullifiers of OrchardZSA notes” (§4.2) in place of the nullifier of n . A *Split Action* is an Action whose consumed note is a Split Input. ZIP 226 calls n “a previously issued input note (that is, a note that has previously been included in the Merkle tree)”; any leaf of Asset Base B qualifies, whether its note was created by an Action or by issuance (ZIP 226, “Terminology” and “Split Notes”). Class: specified.

Padding rule. In a bundle (*Ironwood Guide*, Definition “Bundle”) of OrchardZSA Actions, for each Custom Asset whose Asset Base B has fewer consumed notes than created notes, the sender gives each input-less Action of B a Split Input of B (ZIP 226, “Split Notes”). Actions of ZEC keep the dummy consumed notes of the *Ironwood Guide*’s Definition “Dummy notes”; Actions of Custom Assets take no dummy consumed note (ZIP 226, “Backward Compatibility”). Both regimes occur within one bundle of OrchardZSA Actions. That bundle is not the Orchard-pool bundle of a version-6 transaction: version 6 is the format of ZIP 229, which carries no ZSA fields (ZIP 229, “Non-requirements”). ZIP 226 enforces in the statement that every consumed note of a Custom Asset, split or not and of any value, is a leaf: “for Custom Assets we enforce that *every* input note to an OrchardZSA Action must be proven to exist in the set of note commitments” (“Rationale for Split Notes”). The condition is constructed in “The OrchardZSA Action statement” (§4.3), and Corollary 9.4, in “Security”, states what it yields for the Asset Base of the created note.

Remark 4.3 (Choice of the copied note). ZIP 226, “Split Notes”, lets the sender copy one of three notes of Asset Base B :

- (1) another note of B consumed by a real spend in the same bundle, but not by this Split Input;
- (2) a different unspent note of B , which is not thereby spent;
- (3) an already spent note of B , the zeroed value preventing a second spend.

It states that “we do not care about whether the previously issued note copied to create a Split Input is owned by the sender, or whether it was nullified before”. Proposition 4.12, in “The OrchardZSA Action proof” (§4.4), states which of these choices are available to a sender who does not hold the spend authority of the copied note. Under Remark 1.7, if OrchardZSA notes are in the Orchard pool, which ZIP 258, “Consensus rules from NU6.3 activation”, restricts to Actions with `enableCrossAddress = 0`, then a Split Action whose created note is at an expanded receiver other than that of the copied note violates the cross-address rule. ZIP 226 does not address this case.

4.2 Nullifiers of OrchardZSA notes

An OrchardZSA note consumed by a real spend, with `split_flag = 0`, has the Orchard nullifier of the *Ironwood Guide*’s Definition “Nullifier”:

$$\text{nf} = \text{DeriveNullifier}_{\text{nk}}(\rho, \psi, \text{cm}) = \text{Extract}_{\text{p}}\left(\left[(\text{PRF}_{\text{nk}}^{\text{nfOrchard}}(\rho) + \psi) \bmod p_{\text{Pallas}}\right] K^{\text{Orchard}} + \text{cm}\right),$$

with $\text{cm} = \text{NoteCommit}_{\text{rcm}}^{\text{OrchardZSA}}(n)$ of Definition 3.2 (ZIP 226, “Note Structure and Commitment”: “generated in the same manner as in the Orchard protocol”; protocol specification, §“Computing ρ values and Nullifiers”). It depends on the Asset Base only through cm ; for a note with $\text{AssetBase} = V^{\text{Orchard}}$ it is the Orchard nullifier bit for bit. The extension of the *Ironwood Guide*’s Lemma “One nullifier per note” to OrchardZSA notes is Lemma 9.6, in “Security”.

A Split Action publishes one nullifier, which enters the nullifier set of its pool, and a nullifier must not repeat, either within a transaction or across the block chain (*Ironwood Guide*, §“Nullifier sets”; protocol specification, §“Nullifier Sets”). The copied note may be unspent, choice (2) of Remark 4.3, or spent in the same bundle, choice (1). Publishing its nullifier would, in the first case, make it unspendable and, in the second, repeat a nullifier within the transaction, which invalidates the transaction. The nullifier of a Split Input must therefore differ from that of the copied note. ZIP 226 states the purpose of the change as privacy: it is made “to prevent the identification of notes” (“Circuit Statement”).

Construction 4.4 (Split nullifier). Let

$$L^{\text{Orchard}} := \text{GroupHash}(\text{z.cash:Orchard}, L),$$

the input of the second row of Table 3. For a Split Input with nullifier deriving key nk , element ρ^{old} and commitment cm^{old} , the sender draws rseed_nf uniformly from $\mathbb{B}^{\text{Y}[32]}$ and sets

$$\psi^{\text{nf}} := \text{ToBase}\left(\text{PRFexpand}_{\text{rseed_nf}}([\text{0x0A}] \parallel \text{I2LEOSP}_{256}(\rho^{\text{old}}))\right) \in \mathbb{F}_{p_{\text{Pallas}}}.$$

The *split nullifier* is

$$\text{nf}^{\text{old}} := \text{Extract}_{\text{p}}\left(\left[(\text{PRF}_{\text{nk}}^{\text{nfOrchard}}(\rho^{\text{old}}) + \psi^{\text{nf}}) \bmod p_{\text{Pallas}}\right] K^{\text{Orchard}} + \text{cm}^{\text{old}} + L^{\text{Orchard}}\right).$$

Against the *Ironwood Guide*’s Definition “Nullifier” there are two changes: ψ^{old} is replaced by the fresh ψ^{nf} , and the fixed point L^{Orchard} is added before extraction. The derivation adds one row to that volume’s Construction “Domain bytes of PRF expansion”:

b	Key κ	Remainder t'	f	Output	Section
0x0A	rseed_nf	l2LEOSP ₂₅₆ (ρ^{old})	ToBase	ψ^{nf}	§4.2

The key rseed_nf is secret, uniform and used once. The byte 0x0A is reserved for this use by ZIP 2005’s amendment of §“Sending Notes (Orchard)” (ZIP 2005, “Changes to the Protocol Specification”; ZIP 226, “Split Notes”); the specification’s own list of expansion uses (§“Pseudo Random Functions”) does not record it, and ZIP 2005’s amendment of that list adds 0x0A and 0x0B. It differs from the bytes 0x04 of esk, 0x05 and 0x0B of rcm and 0x09 of ψ of the Orchard sending derivations (protocol specification, §“Sending Notes (Orchard)” Definition 3.5). Class: specified, as a sender rule; Remark 4.8 states what the statement enforces.

Proposition 4.5 (Independence of split nullifiers). *Let n be an OrchardZSA note with commitment $\text{cm} \neq \perp$, elements ρ, ψ , and nullifier nf under nk . For $\psi' \in \mathbb{F}_{p_{\text{Pallas}}}$ let $\text{nf}^{\text{split}}(\psi')$ be the split nullifier of a copy of n with $\psi^{\text{nf}} = \psi'$.*

- (i) *Under the Ironwood Guide’s Assumptions “GroupHash as a random oracle” and “Discrete logarithms on Pallas”, no efficient algorithm outputs nk , a note n with the opening of its commitment, and $\psi' \in \mathbb{F}_{p_{\text{Pallas}}}$ with $\text{nf}^{\text{split}}(\psi') = \text{nf}$, except with negligible probability. This holds for every choice of ψ' , honest or not.*
- (ii) *Let rseed_nf be uniform and used only for this derivation, and let $K^{\text{Orchard}} \neq \mathcal{O}$. To an adversary that knows n , nk and the full viewing key, the split nullifier is indistinguishable from $\text{Extract}_{\text{P}}(U)$ for U uniform on $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$, with advantage at most*

$$\epsilon_{\text{expand}} + \epsilon_{\text{ToBase}} + \delta,$$

where ϵ_{expand} bounds the Ironwood Guide’s Assumption “Pseudorandomness of PRF expansion” for one query, $\epsilon_{\text{ToBase}} \leq p_{\text{Pallas}}/2^{512} < 2^{-257}$ is the statistical distance of ToBase on a uniform 64-byte input from the uniform distribution on $\mathbb{F}_{p_{\text{Pallas}}}$, and $\delta := (p_{\text{Vesta}} - p_{\text{Pallas}})/p_{\text{Vesta}}$ as in that volume’s Proposition “Nullifier unlinkability”.

Status (Definition 1.3): proved here for the specified construction. Part (ii) concerns honest senders, since the statement does not enforce the derivation (Remark 4.8).

Proof. (i) Write

$$s := (\text{PRF}_{\text{nk}}^{\text{nfOrchard}}(\rho) + \psi) \bmod p_{\text{Pallas}}, \quad s' := (\text{PRF}_{\text{nk}}^{\text{nfOrchard}}(\rho) + \psi') \bmod p_{\text{Pallas}},$$

and $K := K^{\text{Orchard}}$, $L := L^{\text{Orchard}}$. By the *Ironwood Guide*’s Lemma “Coordinate extraction identifies opposite points”, $\text{nf}^{\text{split}}(\psi') = \text{nf}$ means $[s'] K + \text{cm} + L = \pm([s] K + \text{cm})$. With the sign +,

$$L + [s' - s] K = \mathcal{O};$$

with the sign −,

$$L + [s + s'] K + [2] \text{cm} = \mathcal{O}.$$

Given its opening, $\text{cm} \neq \perp$ is a known integer combination of the Sinsemilla generators of its domain and of its blinding base, by the unfolding of the *Ironwood Guide*'s Construction "Sinsemilla hash on Pallas" used in the proof of its Proposition "Collision resistance of the Sinsemilla instances"; in either branch of Definition 3.2 these generators are values of GroupHash at inputs other than $(z.\text{cash}:\text{Orchard}, L)$ and $(z.\text{cash}:\text{Orchard}, K)$. In both cases L has coefficient 1, and its input differs from every other input involved (Lemma 2.8), so the algorithm has output a non-trivial relation among values of GroupHash at distinct inputs, which the *Ironwood Guide*'s Lemma "Relations among fixed generators" excludes except with negligible probability. This is the role of the offset L , which ZIP 226 does not state: without it, $\psi' = \psi$ reproduces nf .

(ii) The hybrids follow the proof of the *Ironwood Guide*'s Proposition "Nullifier unlinkability". First, $\text{PRFexpand}_{\text{rseed_nf}}$, evaluated at the single input $[\text{0x0A}] \parallel \text{I2LEOSP}_{256}(\rho)$, is replaced by a uniform 64-byte string, at cost ϵ_{expand} : the key is used nowhere else, and the rest of the game is an efficient function of the answer and of (n, nk) . Second, ToBase of that uniform string is replaced by a uniform $\psi^{\text{nf}} \in \mathbb{F}_{p_{\text{Pallas}}}$, at cost ϵ_{ToBase} (*Math Guide*, §"Uniform sampling and the bias of modular reduction"; *Ironwood Guide*, §"Pseudorandom functions and field reductions"). Translation by $\text{PRF}_{\text{nk}}^{\text{nfOrchard}}(\rho)$ is a bijection of $\mathbb{F}_{p_{\text{Pallas}}}$, so s' is then uniform on $\{0, \dots, p_{\text{Pallas}} - 1\}$ and independent of nk, ρ and all else. Third, s' , read in $\mathbb{F}_{p_{\text{Vesta}}}$, is replaced by a uniform scalar $u \in \mathbb{F}_{p_{\text{Vesta}}}$, at cost δ , the statistical distance computed in the cited proposition. Since $K \neq \mathcal{O}$ generates the group of prime order p_{Vesta} , the point $[u]K + \text{cm} + L$ is then uniform on $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$ whatever cm and L are, and the split nullifier is Extract_p of a uniform point. $\square$

In the model of the *Ironwood Guide*'s Assumption "GroupHash as a random oracle", the event $K^{\text{Orchard}} = \mathcal{O}$ excluded in part (ii) has probability $1/p_{\text{Vesta}}$.

4.3 The OrchardZSA Action statement

The statement is defined as a delta on the relation $\mathcal{R}_{\text{Action}}$ of the *Ironwood Guide*'s Definition "Orchard Action statement", itself a relation in the sense of the *Crypto Guide*'s Definition "Language, relation, witness" (§"Languages, relations, and witnesses"). Notation, types and conditions not named below are those of that definition; conditions A6 to A9 are cited by name and not restated.

Definition 4.6 (OrchardZSA Action statement). *Base.* The seven components of the primary input of the *Ironwood Guide*'s Definition "Orchard Action statement" other than $\text{disableCrossAddress}$, and its conditions A1 to A9. The input $\text{disableCrossAddress}$ and condition A10 are omitted (Remark 4.7).

Primary input.

$$\text{pub} = (rt, \text{cv}^{\text{net}}, \text{nf}^{\text{old}}, \text{rk}, \text{cmx}, \text{enableSpends}, \text{enableOutputs}, \text{enableZSA}),$$

with the first seven components typed as in the cited definition and $\text{enableZSA} \in \{0, 1\}$.

Auxiliary input. The auxiliary input aux of the cited definition, together with

$$\text{AssetBase} \in \mathcal{E}^*, \quad \text{is_native_asset} \in \{0, 1\}, \quad \text{split_flag} \in \{0, 1\}, \quad \psi^{\text{nf}} \in \mathbb{F}_{p_{\text{Pallas}}}.$$

Notation. Let $v' := 0$ if $\text{split_flag} = 1$ and $v' := v^{\text{old}}$ otherwise. With M^{old} and M^{new} the 1086-bit messages of the cited definition, built from the witnessed diversified bases, write

$\text{NoteCommit}_{\text{rcm}}^{\text{OrchardZSA}}(g_d^*, pk_d^*, v, \rho, \psi, \text{AssetBase})$ for the commitment of Definition 3.2 with $M(n)$ replaced by the message built from these arguments: $\text{NoteCommit}_{\text{rcm}}(M)$ in the native branch, and $\text{SinsemillaHashToPoint}(z.\text{cash}: \text{ZSA-NoteCommit-M}, M \parallel \text{AssetBase}^*) + [\text{rcm}] H_D$, or $\perp$, in the custom branch. Let L^{Orchard} be as in Construction 4.4.

Relation. The relation $\mathcal{R}_{\text{ActionZSA}}$ is the set of pairs (pub, aux) of the types above that satisfy the following conditions.

A1'. Old note commitment integrity:

$$\begin{aligned} & \text{NoteCommit}_{\text{rcm}^{\text{old}}}^{\text{OrchardZSA}}((g_d^{\text{old}})^*, (pk_d^{\text{old}})^*, v^{\text{old}}, \rho^{\text{old}}, \psi^{\text{old}}, \text{AssetBase}) \\ & \in \{\text{cm}^{\text{old}}, \perp\}. \end{aligned}$$

A2'. New note commitment integrity:

$$\begin{aligned} & \text{Extract}_{\text{p}}^{\perp}(\text{NoteCommit}_{\text{rcm}^{\text{new}}}^{\text{OrchardZSA}}((g_d^{\text{new}})^*, (pk_d^{\text{new}})^*, v^{\text{new}}, \rho^{\text{new}}, \psi^{\text{new}}, \text{AssetBase})) \\ & \in \{\text{cmx}, \perp\}, \end{aligned}$$

with $\rho^{\text{new}} := \text{nf}^{\text{old}}$ and the same AssetBase as in A1'.

C1. Native indicator: $\text{is_native_asset} = 1$ if and only if $\text{AssetBase} = V^{\text{Orchard}}$.

C2. Pause: if $\text{enableZSA} = 0$, then $\text{is_native_asset} = 1$.

A3'. Merkle path validity: ($v^{\text{old}} = 0$ and $\text{is_native_asset} = 1$), or $\text{root} = \text{rt}$, or some evaluation of SinsemillaHash in the fold returns $\perp$; the value root , the fold and the encoding bits c are those of A3.

A4'. Value commitment integrity:

$$\begin{aligned} \text{cv}^{\text{net}} &= \text{ValueCommit}_{\text{rcv}}^{\text{OrchardZSA}}(\text{AssetBase}, v' - v^{\text{new}}) \\ &= [v' - v^{\text{new}}] \text{AssetBase} + [\text{rcv}] R^{\text{Orchard}}, \end{aligned}$$

a variable-base multiplication by the witnessed AssetBase (Definition 3.8).

C3. Split only for Custom Assets: if $\text{split_flag} = 1$, then $\text{is_native_asset} = 0$.

A5'. Nullifier integrity: if $\text{split_flag} = 0$, then $\psi^{\text{nf}} = \psi^{\text{old}}$ and

$$\text{nf}^{\text{old}} = \text{DeriveNullifier}_{\text{nk}}(\rho^{\text{old}}, \psi^{\text{old}}, \text{cm}^{\text{old}});$$

if $\text{split_flag} = 1$, then

$$\begin{aligned} \text{nf}^{\text{old}} &= \text{Extract}_{\text{p}}([(\text{PRF}_{\text{nk}}^{\text{nfOrchard}}(\rho^{\text{old}}) + \psi^{\text{nf}}) \bmod p_{\text{Pallas}}] K^{\text{Orchard}} \\ &\quad + \text{cm}^{\text{old}} + L^{\text{Orchard}}), \end{aligned}$$

the split nullifier of Construction 4.4.

A6, A7. Spend authority and diversified address integrity: unchanged and unconditional; no factor of `split_flag` relaxes them.

A8, A9. Enable spend flag and enable output flag: unchanged; A8 reads v^{old} , not v' , and A9 reads v^{new} .

(ZIP 226, “Circuit Statement”, paragraphs “Asset Base Equality”, “The enableZSA Flag”, “Value Commitment Correctness” and “Asset Identifier Consistency for Split Actions”; ZIP 226, “Split Notes” and “Note Structure and Commitment”; protocol specification, §“Action Statement (Orchard)”). Class: specified at statement level; its status is stated in Remark 4.7.

The following consequences are read off Definition 4.6, Definition 3.2 and Definition 2.15.

One witness. Conditions $A1'$, $A2'$ and $A4'$ use the single witnessed AssetBase, so the consumed and the created note of an Action have one Asset Base, and its net value commitment is on that base.

Native branch. By C1 and the case split of Definition 3.2, an Action with $\text{AssetBase} = V^{\text{Orchard}}$ and $\text{split_flag} = 0$ satisfies $A1'$, $A2'$, $A4'$ and $A5'$ exactly when $\psi^{\text{nf}} = \psi^{\text{old}}$ and it satisfies the Orchard $A1$, $A2$, $A4$ and $A5$.

Pause. With $\text{enableZSA} = 0$, C2 and C1 force $\text{AssetBase} = V^{\text{Orchard}}$, and C3 then forces $\text{split_flag} = 0$, so the relation admits only Orchard Actions.

Membership. Since $0 \leq v^{\text{old}} < 2^{64} < p_{\text{Pallas}}$, the first disjunct of $A3'$ holds only for a consumed ZEC note of value 0. Every consumed note of a Custom Asset, split or not and of any value, satisfies $\text{root} = \text{rt}$ or the $\perp$ clause. The $\perp$ clause gives no escape: for witnesses extracted under Assumption 4.9, the argument of the *Ironwood Guide*’s Remark “ $\perp$ -weakened conditions”, by that volume’s Proposition “Collision resistance of the Sinsemilla instances” (iii) under its Assumptions “Discrete logarithms on Pallas” and “GroupHash as a random oracle”, excludes it except with negligible probability; the same holds for the $\perp$ cases of $A1'$ and $A2'$ in both branches of $\text{NoteCommit}^{\text{OrchardZSA}}$, by Proposition 3.3(c) for the custom branch.

Split Actions. By $A4'$ the copied value never enters cv^{net} . By A8, which reads v^{old} , a Split Action that copies a note of non-zero value requires $\text{enableSpend} = 1$, while one that copies a note of value 0 does not.

These consequences are direct from the cited definitions and from the *Ironwood Guide*’s Remark “ $\perp$ -weakened conditions”, with Assumption 4.9 in place of that volume’s knowledge-soundness assumption (ZIP 226, “Circuit Statement” and “Overview”). Table 5 pairs each condition with the Orchard condition it replaces and its ZIP 226 paragraph; Figure 1 draws the new witnesses and the conditions that read them.

Remark 4.7 (Status of the OrchardZSA Action statement). The delta is specified at statement level (ZIP 226, Draft, “Circuit Statement”). ZIP 226 states it against the specification’s statement with seven primary inputs and conditions A1 to A9. The current Orchard statement has the eighth primary input `disableCrossAddress` and condition A10 (*Ironwood Guide*, Definition “Orchard Action statement” and Remark “Status”; protocol specification, §“Action Descriptions”; ZIP 258, “Consensus rules from NU6.3 activation”), which ZIP 226 never mentions. The composition of the delta with A10 is therefore an open problem (Definition 1.2). The statement

Table 5. Conditions of the OrchardZSA Action statement of Definition 4.6: each condition, the condition of the *Ironwood Guide*’s Definition “Orchard Action statement” that it replaces, and the paragraph of ZIP 226, “Circuit Statement”, that states it.

Condition	Replaces	ZIP 226, “Circuit Statement”, paragraph
A1’	A1	“Asset Base Equality”
A2’	A2	“Asset Base Equality”
C1	none	“Asset Base Equality” (the witness is <code>_native_asset</code>)
C2	none	“The enableZSA Flag”
A3’	A3	“Asset Identifier Consistency for Split Actions” (Merkle Path Validity)
A4’	A4	“Value Commitment Correctness”; “Asset Identifier Consistency for Split Actions” (v)
C3	none	“Asset Identifier Consistency for Split Actions”
A5’	A5	“Asset Identifier Consistency for Split Actions” (Nullifier Integrity), with “Split Notes” for the formula and “Note Structure and Commitment” for <code>split_flag = 0</code>
A6 to A9	themselves	none (unchanged)
A10	omitted	none (Remark 4.7)

uses enableZSA, which ZIP 226 never lists as a primary input. The encoded instance of the current Orchard statement has ten elements, the nine that encode its first seven primary inputs and disableCrossAddress (*Ironwood Guide*, §“The Action statement”); no live text places enableZSA in it, so its position is an open problem. ZIP 226 defers “All modifications in the Circuit” to an unversioned document outside the ZIPs, “Modifications to the Orchard circuit for the OrchardZSA Protocol”; no ZIP fixes a complete OrchardZSA circuit, which is designed but unspecified. The reading of A8 on v^{old} is specified by the delta, which leaves A8 unchanged; only its confirmation in that circuit document is designed but unspecified, and Table 7 records it.

Remark 4.8 (What the OrchardZSA statement does not check). Items (i) to (iii) of the *Ironwood Guide*’s Remark “What the statement does not check” carry over. In addition:

- (iv) The witness ψ^{nf} is constrained only through A5’ to the published nullifier. For `split_flag = 1` only the equality $\psi^{\text{nf}} = \psi^{\text{old}}$ is switched off, and no condition derives ψ^{nf} from a seed. The derivation of Construction 4.4 is therefore a sender rule whose formula is specified and which nothing enforces: `rseed_nf` is transmitted nowhere, and no text proposes an enforcement. Proposition 4.5(i) holds for every ψ^{nf} ; part (ii) holds for senders that follow the rule.
- (v) The derivations of `rcm` and ψ from `rseed` are unchecked, as for Orchard.
- (vi) The witnessed AssetBase is checked only against the consumed leaf, through A1’ and A3’, and never against a record of issued Assets. Lemma 9.3, in “Security”, concerns the leaves.
- (vii) The statement does not check that the prover owns the copied note of a Split Input beyond A6 and A7; this is the subject of Proposition 4.12.

(ZIP 226, “Split Notes” and “Circuit Statement”).

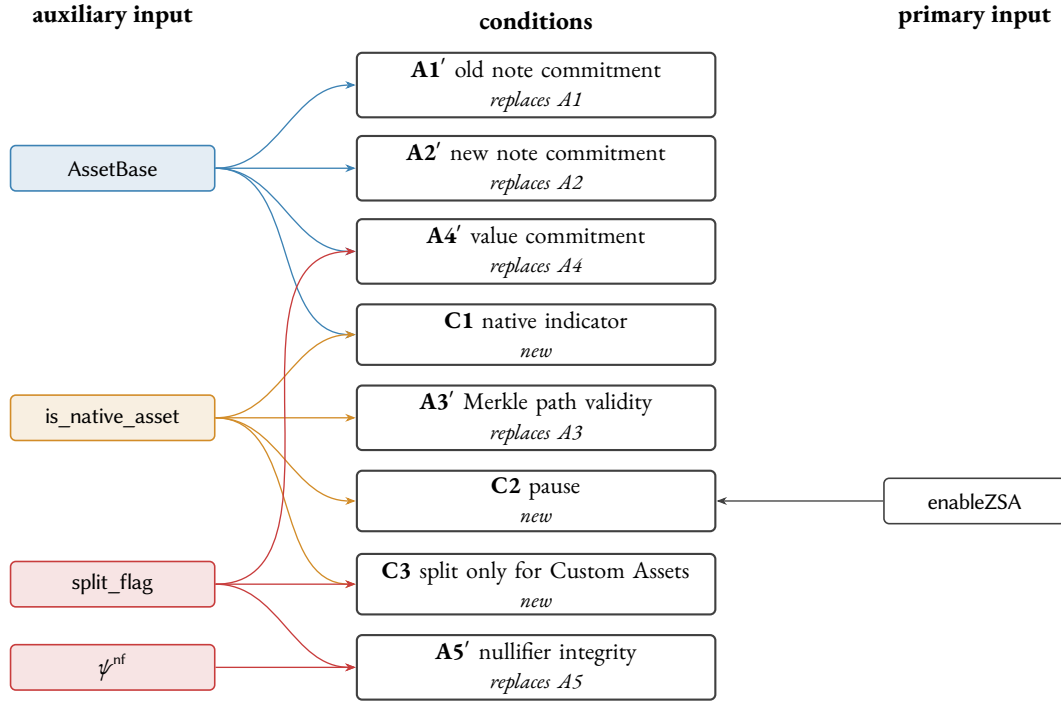


Figure 1. The witnessed Asset Base in the OrchardZSA Action statement of Definition 4.6. The four new witnesses are on the left, the new primary input enableZSA on the right, and the conditions that read them in the centre, each tagged with the Orchard condition it replaces or as new. One witness AssetBase enters A1', A2' and A4', which forces equal Asset Bases of the consumed and the created note and a value commitment on that base; through C1 and A3', only a consumed ZEC note of value 0 escapes membership.

4.4 The OrchardZSA Action proof

The *Ironwood Guide*'s Assumptions “Knowledge soundness of the Action proof” and “Zero knowledge of the Action proof” name the relation $\mathcal{R}_{\text{Action}}$ and its verifying key. The corresponding assumptions for $\mathcal{R}_{\text{ActionZSA}}$ are stated here in the same form; the *OrchardZSA verifying key* denotes the verifying key of a circuit for $\mathcal{R}_{\text{ActionZSA}}$, which no ZIP names.

Assumption 4.9 (Knowledge soundness of the OrchardZSA Action proof). Let the hash of the Fiat–Shamir transform be modelled as a random oracle. For every efficient adversary that outputs primary inputs $\text{pub}_1, \dots, \text{pub}_n$ and an aggregate proof π , there is an efficient extractor, with access to the adversary and to its random-oracle queries, that outputs auxiliary inputs $\text{aux}_1, \dots, \text{aux}_n$ such that the probability of the event

$$\begin{aligned} \pi \text{ is accepted for } (\text{pub}_1, \dots, \text{pub}_n) \text{ under the OrchardZSA verifying key} \\ \text{and } (\text{pub}_i, \text{aux}_i) \notin \mathcal{R}_{\text{ActionZSA}} \text{ for some } i \end{aligned}$$

is negligible. This is the knowledge-soundness clause of the *Crypto Guide*'s Definition “SNARK” (§“SNARKs: succinct non-interactive arguments of knowledge”) for the relation of Definition 4.6, for a Halo 2 proof (*Halo 2 Guide*, §“The Halo 2 proof system” and §“A rigorous treatment of knowledge soundness”), the primary input bound to the proof as in

the *Halo 2 Guide*, §“Instance columns and binding the public statement”. It includes that every satisfying assignment of the circuit yields a witness of $\mathcal{R}_{\text{ActionZSA}}$; in particular, the multiplication of $A4'$ is by the witnessed AssetBase and nothing else. No ZIP names the circuit or the verifying key: the assumption concerns a designed but unspecified object.

Assumption 4.10 (Zero knowledge of the OrchardZSA Action proof). Let the hash of the Fiat–Shamir transform be modelled as a programmable random oracle. There is an efficient simulator that, given only $\text{pub}_1, \dots, \text{pub}_n$ and programming the random oracle, outputs an aggregate proof statistically indistinguishable from an honest proof under the OrchardZSA verifying key, for any $\text{aux}_1, \dots, \text{aux}_n$ with $(\text{pub}_i, \text{aux}_i) \in \mathcal{R}_{\text{ActionZSA}}$ for every i , against every distinguisher that makes polynomially many random-oracle queries. This is zero knowledge of statistical grade in the non-interactive, random-oracle form of the *Crypto Guide*’s Definition “Zero knowledge” (§“The simulation paradigm and zero knowledge”), for a Halo 2 proof (*Halo 2 Guide*, §“Zero knowledge: hiding the witness in Halo 2”). It concerns the same designed but unspecified circuit and key.

Remark 4.11 (Circuit, verifying key and proof length). The statement is specified at delta level (ZIP 226). Its arithmetisation exists only in the document outside the ZIPs named in Remark 4.7, so the circuit and its verifying key are designed but unspecified, and Assumptions 4.9 and 4.10 are the only hypotheses through which later results depend on them. Condition $A4'$ is sound only if the circuit multiplies by the witnessed AssetBase and nothing else. The Orchard variable-base scalar multiplication was corrected at NU6.2 because it left the base under-constrained (ZIP 257, “NU6.2 deployment”). ZIP 226, which turns the value base into a witness (“Value Commitment Correctness”), does not mention the correction, which any OrchardZSA circuit must carry. ZIP 257 made the Orchard proof length $2720 + 2272n$ bytes, for n Actions, a consensus rule (“Canonical Orchard Action proof length”). No ZIP fixes an OrchardZSA proof length, which the circuit determines and which no live text contradicts: it is designed but unspecified, as the circuit and the verifying key are.

Proposition 4.12 (Ownership of Split Inputs). *Consider the Ironwood Guide’s Definition “Spend-authority experiment” with $\mathcal{R}_{\text{ActionZSA}}$ in place of $\mathcal{R}_{\text{Action}}$: a key generated with $\text{use_qsk} = \text{false}$, the adversary holding its full viewing key but not ask. Under Assumption 4.9 and the Ironwood Guide’s Assumptions “Discrete logarithms on Pallas”, “GroupHash as a random oracle”, “The RedPallas hash as a random oracle” and “Pseudorandomness of PRF expansion”, no efficient adversary outputs, except with negligible probability, a block chain with an accepted Split Action whose witness copies a leaf that is a note of the key, and whose spend-authorisation signature is valid under its rk on a signature digest for which it made no signing query with that rk . Consequently the sentence of ZIP 226, “Split Notes”, “we do not care about whether the previously issued note copied to create a Split Input is owned by the sender, or whether it was nullified before” holds for spentness and not for ownership: a sender can use choices (2) and (3) of Remark 4.3 only with the spend authority of the copied note, except for a note whose key is public, such as a reference note (Proposition 6.11). Status (Definition 1.3): proved here from specified conditions; conditional on Assumption 4.9, which concerns the designed but unspecified circuit.*

Proof. The proof is a reduction to the proof of the *Ironwood Guide*’s Theorem “Spend authority”. By Assumption 4.9 the extractor yields, for the accepted Split Action, a witness of $\mathcal{R}_{\text{ActionZSA}}$ with `split_flag` = 1. By C3, `is_native_asset` = 0, so $A3'$ gives $root = rt$ outside its $\perp$ case, which is excluded as in the consequences of Definition 4.6; with the anchor rule, the *Ironwood Guide*’s Proposition “Membership soundness” makes $\text{Extract}_p(\text{cm}^{\text{old}})$ a leaf. Condition $A1'$ with Proposition 3.3(b) makes the witnessed consumed note the note of that leaf, so its witnessed expanded receiver $(g_d^{\text{old}}, pk_d^{\text{old}})$ is the receiver of the leaf’s note, a note of the key; the witness therefore consumes a note of the key in the sense of the experiment. Conditions A6 and A7, unchanged and unconditional, then bind rk to a randomisation of $\pm ak^P$ of that key, by the *Ironwood Guide*’s Proposition “Binding of ivk to (ak, nk) ” and Lemma “Coordinate extraction identifies opposite points”, and a valid signature under rk without a matching signing query contradicts that volume’s Proposition “Unforgeability under re-randomisation”, exactly as in steps (1) to (5) of the proof of Theorem “Spend authority”. Neither `split_flag` nor v' enters A6 or A7, the only conditions that proof uses; $A1'$ and membership serve here only to identify the witnessed receiver with that of the copied leaf’s note, so the steps apply unchanged.

For spentness: by $A4'$ the copied value is not committed, and by Proposition 4.5(i) the copied note’s nullifier is not published, except with negligible probability. Choice (1) of Remark 4.3 copies a note that the same bundle spends, so its sender holds that note’s spend authority in any case. $\square$

4.5 Action Groups

Definition 4.13 (Action Group). An *Action Group* is a non-empty finite sequence of OrchardZSA Actions together with one anchor rt , the flags `enableSpends`, `enableOutputs` and `enableZSA` in $\{0, 1\}$, and one aggregate proof for the relation of Definition 4.6 at the primary inputs

$$(rt, cv_i^{\text{net}}, nf_i, rk_i, cmx_i, \text{enableSpends}, \text{enableOutputs}, \text{enableZSA})$$

of its Actions i . Each Action carries the public fields $(cv^{\text{net}}, nf, rk, cmx, epk, C_{\text{enc}}, C_{\text{out}})$ of an Orchard Action description (*Ironwood Guide*, Definition “Action description”) and its own spend-authorisation signature under rk . The term is that of ZIP 228, “Terminology”: Actions that share the tuple

$$(rt, \text{enableSpends}, \text{enableOutputs}, \text{enableZSA}, nAG\text{ExpiryHeight}).$$

ZIP 227 requires $nAG\text{ExpiryHeight} = 0$ and at most one Action Group per transaction (“Specification: Consensus Rule Changes”), so the fifth component is constant and is omitted. The nullifier $nf_{0,0}$ of the first Action of the Action Group is the value that issuance uses (ZIP 227, “Computation of ρ ”). The Action Group has no burn set; the remaining components of a transaction’s bundle are defined in “Burn sets and OrchardZSA bundles” (§5.1). Class: specified.

The structure is specified by ZIPs 226 to 228, all Draft. Its encoding exists only in ZIP 230, which is Withdrawn; the layout that ZIP 230 calls version 6 is not the version 6 of ZIP 229, which has no Action Groups (ZIP 229, “Transaction Format”). The byte lengths of the note ciphertext and of

an Action depend on a plaintext encoding that no current ZIP defines (Definition 3.7). The flag `enableZSA` has no flag bit in any current format. ZIP 230 assigns it bit 2 of the Orchard flags byte, which ZIP 229, “Consensus Rules”, reserves as 0 in version 6 and which is `enableCrossAddress` in the Ironwood flags byte (*Ironwood Guide*, §“Bundle flags”; ZIP 258, “Consensus rules from NU6.3 activation”). The design conflicts with live text, so the flag bit is an open problem (Definition 1.2).

5 Burn and the binding signature

This section defines the burn set and the OrchardZSA bundle that it completes (§5.1); constructs the binding validating key, which subtracts the burnt amounts Asset by Asset (§5.2); and proves that a verifying binding signature forces per-Asset balance whenever every witnessed Asset Base and every Asset Base of a burn pair is a group-hash output at a suitably distinct input (§5.3). The premise on the bases concerns a single bundle; it is discharged for valid chains in “Security” (§9). The constructions are taken from ZIP 226 and ZIP 227, both Draft. The Orchard binding signature that they extend is that of the *Ironwood Guide*, §“The binding signature”, and is cited, not restated.

5.1 Burn sets and OrchardZSA bundles

Definition 5.1 (Burn set). *Burn* destroys a stated amount of a Custom Asset (Definition 1.5): it sends nothing to any address, lowers the amount of the Asset in circulation, and publishes the Asset Base and the amount, so that the destruction is publicly verifiable. A *burn set* `assetBurn` is a finite set of pairs $(\text{AssetBase}, v)$ with $\text{AssetBase} \in \mathcal{E}^*$ and $v \in \mathbb{Z}$. The author of a transaction adds one pair for each Custom Asset that the transaction burns, v being the amount burnt. The constant

$$\text{MAX_BURN_VALUE} := 2^{63} - 1 = 9\,223\,372\,036\,854\,775\,807$$

bounds the amount of one Custom Asset burnt in one transaction, so that every burnt amount is representable in a signed 64-bit balance field. A burn set is *valid* when it satisfies the following rules.

- (B1) For every pair, $\text{AssetBase} \neq V^{\text{Orchard}}$: the Native Asset Base (Definition 2.15), of ZEC and of TAZ alike, is never burnt.
- (B2) For every pair, $0 < v \leq \text{MAX_BURN_VALUE}$.
- (B3) Every Asset Base occurs in at most one pair. The *burnt amount* v_B of a point B is therefore well defined: the v of the pair with $\text{AssetBase} = B$, or 0 if there is none.

Rules (B1) to (B3) constrain the pairs only. Rule (B1) excludes the one point V^{Orchard} and no other, and no rule requires AssetBase to be the Asset Base of an issued Asset. The state rule that a burn cannot exceed the amount of the Asset in circulation is ZIP 227’s; it is rule (T4) of Construction 7.3, in “State transition of a transaction” (§7.2) (ZIP 226, “Burn Mechanism”, “Additional Consensus Rules for the `assetBurn` set” and “Rationale for `MAX_BURN_VALUE`”;

ZIP 227, “Specification: Consensus Rule Changes”). Class: specified; the carrier of the burn set is stated in “Transaction formats” (§8.1).

Definition 5.2 (OrchardZSA bundle). The *OrchardZSA bundle* of a transaction consists of at most one Action Group (Definition 4.13), one burn set assetBurn satisfying (B1) to (B3), one balancing value v^{balance} and one binding signature. It extends the *Ironwood Guide*’s Definition “Bundle” by the burn set; its anchor, flags and proof are those of the Action Group. ZIP 226 places one burn set in the transaction, and ZIP 227 limits the transaction to at most one Action Group, so the two readings coincide (ZIP 226, “Burn Mechanism”; ZIP 227, “Specification: Consensus Rule Changes”). Class: the structure is specified; its encoding, one burn set per Action Group in withdrawn ZIP 230, belongs to the carrier and is classified in Remark 8.1.

Remark 5.3 (No transparent pool for Custom Assets). ZIPs 226 and 227 leave the transparent protocol unchanged and single-asset. Transparent inputs and outputs, and the *Ironwood Guide*’s transparent transaction value pool (Definition “Transparent transaction value pool”), carry the Native Asset only; Custom Assets have no transparent value pool and cannot be unshielded by a transfer. ZIP 226 conditions this on future consensus changes, which no ZIP specifies (ZIP 226, “Burn Mechanism”, note; “Rationale for Value Balance Verification”: “All Custom Assets are contained within the shielded pool, and cannot be unshielded via a regular transfer”). Structurally, the binding validating key of the next subsection (Definition 5.4) has, besides the Actions’ net value commitments, only the balancing value at V^{Orchard} and one burn term per pair; a non-zero net amount of a Custom Asset in a bundle can therefore be absorbed only by a burn pair of that Asset. The computational statement is Proposition 5.6. Class: specified.

5.2 The binding validating key

Definition 5.4 (Binding validating key of an OrchardZSA bundle). Let an OrchardZSA bundle have n Actions with net value commitments $\text{cv}_1^{\text{net}}, \dots, \text{cv}_n^{\text{net}}$ (Definition 3.8), trapdoors $\text{rcv}_1, \dots, \text{rcv}_n$, balancing value v^{balance} and burn set assetBurn . The *binding signing key* is unchanged:

$$\text{bsk} := \text{rcv}_1 + \dots + \text{rcv}_n \in \mathbb{F}_{p_{\text{Vesta}}}$$

(*Ironwood Guide*, Definition “Binding signing key”). The verifier computes, from public fields only, the *binding validating key*

$$\text{bvk} := \left(\sum_{i=1}^n \text{cv}_i^{\text{net}} \right) - [v^{\text{balance}}] V^{\text{Orchard}} - \sum_{(\text{AssetBase}, v) \in \text{assetBurn}} [v] \text{AssetBase},$$

where the subtracted points are the zero-trapdoor commitments

$$\text{ValueCommit}_0^{\text{OrchardZSA}}(V^{\text{Orchard}}, v^{\text{balance}}) \quad \text{and} \quad \text{ValueCommit}_0^{\text{OrchardZSA}}(\text{AssetBase}, v),$$

each signed integer entering as its residue modulo p_{Vesta} . Against the *Ironwood Guide*’s Definition “Binding validating key” two things are new: the bases of the Actions’ commitments vary, being the witnessed Asset Bases, and one burn term is subtracted per pair.

The balancing value keeps the meaning of the *Ironwood Guide*'s Definitions “Transparent transaction value pool” and “Balancing value”: the net flow of ZEC between the pool and the transparent transaction value pool, a signed integer in $\{-2^{63}, \dots, 2^{63} - 1\}$. It enters bvk at V^{Orchard} only. Its meaning rests on the key of ZIP 226; its encoding, the field `valueBalance0rchard` of withdrawn ZIP 230, belongs to the carrier.

The binding signature is that of the *Ironwood Guide*'s Construction “Binding signature”, the binding-signature instance of RedPallas on the base R^{Orchard} (*Ironwood Guide*, §“The Red-Pallas signature scheme”): the author signs `SigHash` with `bsk`, and the verifier MUST validate the signature on `SigHash` under `bvk`. Here `SigHash` is the signature digest of the transaction (*Ironwood Guide*, Definition “Signature digest”) as modified for OrchardZSA; the modifications and their class are stated in “Transaction digests and the signature message” (§8.2, Remark 8.3) (ZIP 226, “Value Balance Verification” and “Value Commitment”; protocol specification, §“Balance and Binding Signature (Orchard)”). Class: specified; the signed message is not.

For an honestly constructed bundle each cv_i^{net} equals

$$[v'_i - v_i^{\text{new}}] \text{AssetBase}_i + [\text{rcv}_i] R^{\text{Orchard}},$$

by condition A4' of Definition 4.6. By the additive homomorphism (*Crypto Guide*, §“The Pedersen commitment”, Theorem “Additive homomorphism”), the trapdoor terms sum to $[\text{bsk}] R^{\text{Orchard}}$, and the value terms group by Asset Base. When the net values of each Custom Asset sum to its burnt amount and those of ZEC to v^{balance} , every value term cancels, so $\text{bvk} = [\text{bsk}] R^{\text{Orchard}}$, the pair (bsk, bvk) is a key pair of the instance, and the honest signature is accepted, as in the paragraph after the *Ironwood Guide*'s Construction “Binding signature”. In the notation of ZIP 226, with S_{CA} the indices of the Actions of Custom Assets, a correctly constructed bundle satisfies

$$\sum_{j \in S_{\text{CA}}} \text{cv}_j^{\text{net}} - \sum_{(\text{AssetBase}, v) \in \text{assetBurn}} [v] \text{AssetBase} = \left[\sum_{j \in S_{\text{CA}}} \text{rcv}_j \right] R^{\text{Orchard}}$$

(ZIP 226, “Rationale for Value Balance Verification”). For a Split Action $v' = 0$, so the value of the copied note never enters `bvk`: the commitment $[0 - v^{\text{new}}] \text{AssetBase} + [\text{rcv}] R^{\text{Orchard}}$ cancels against the value of the notes of the same Asset consumed by real spends in other Actions (ZIP 226, “Split Notes”).

The key `bvk` is computed from the Actions' commitments, the balancing value and the burn set, and of these only the burn set names an Asset, by publishing each burnt Asset Base with its amount. The hiding of the Actions' Asset Bases is Theorem 9.11, in “Security”.

5.3 Per-Asset balance

Lemma 5.5 (Modular per-Asset balance is integer balance). *Let $n \leq 62\,500$. For $1 \leq i \leq n$ let v'_i and v_i^{new} be integers in $\{0, \dots, 2^{64} - 1\}$ and AssetBase_i a point. Let `assetBurn` satisfy (B2) and (B3), with burnt amounts v_B , and let v^{balance} be an integer in $\{-2^{63}, \dots, 2^{63} - 1\}$. For a point B*

put

$$c_B := \sum_{i: \text{AssetBase}_i = B} (v'_i - v_i^{\text{new}}) - v_B \quad (B \neq V^{\text{Orchard}}),$$

$$c_{V^{\text{Orchard}}} := \sum_{i: \text{AssetBase}_i = V^{\text{Orchard}}} (v'_i - v_i^{\text{new}}) - v^{\text{balance}},$$

rule (B1) giving $v_{V^{\text{Orchard}}} = 0$ for a valid burn set. If $c_B \equiv 0 \pmod{p_{\text{Vesta}}}$, then $c_B = 0$ as an integer.

The hypotheses hold for every OrchardZSA bundle of a valid block. Every Action carries its own 32-byte nullifier (Ironwood Guide, Definition “Action description”; protocol specification, §“Action Description Encoding and Consensus”), and a block, hence a transaction, holds at most 2 000 000 bytes (protocol specification, §“Block Header Encoding and Consensus”), so

$$n \leq 2\,000\,000/32 = 62\,500 < 2^{16}$$

whatever the carrier. The values are 64-bit by Definition 3.1, with $v' \in \{0, v^{\text{old}}\}$ by the notation of Definition 4.6; burnt amounts are bounded by (B2), and the balancing value by Definition 5.4. No carrier-specific bound on the number of Actions is needed. Status (Definition 1.3): proved here; conditional on no unspecified object.

Proof. The argument is that of the Ironwood Guide’s Lemma “Modular balance is integer balance”. Each difference $v'_i - v_i^{\text{new}}$ has absolute value at most $2^{64} - 1$, and $n \leq 2^{16} - 1$. For $B \neq V^{\text{Orchard}}$, rule (B2) gives $0 \leq v_B \leq 2^{63} - 1$, so

$$|c_B| \leq (2^{16} - 1)(2^{64} - 1) + (2^{63} - 1) = 1\,208\,916\,596\,242\,592\,319\,864\,832;$$

for V^{Orchard} , with $|v^{\text{balance}}| \leq 2^{63}$,

$$|c_{V^{\text{Orchard}}}| \leq (2^{16} - 1)(2^{64} - 1) + 2^{63} = 1\,208\,916\,596\,242\,592\,319\,864\,833,$$

the upper end of the range in the protocol specification, §“Balance and Binding Signature (Orchard)”. Both bounds are below $2^{80} < 2^{253} \leq (p_{\text{Vesta}} - 1)/2$ (Ironwood Guide, §“Fields, groups, and encodings”; protocol specification, §“Pallas and Vesta”). The hypothesis makes c_B a multiple of p_{Vesta} , the order of the Pallas group, and the only such multiple of absolute value below p_{Vesta} is 0. $\square$

The protocol specification’s argument in §“Balance and Binding Signature (Orchard)” bounds the number of Actions by $2^{16} - 1$ through a separate consensus rule, which it notes is redundant given the 2 MB transaction size limit; the lemma uses the block size itself.

Fix an OrchardZSA bundle as in Definition 5.4, and let $\mathfrak{B}$ be the set of points consisting of V^{Orchard} , the Asset Bases of the Actions and the Asset Bases of the burn pairs. An *opening* of cv_i^{net} is a triple (B_i, u_i, r_i) of a point $B_i \in \mathfrak{B}$, an integer u_i and a scalar $r_i \in \mathbb{F}_{p_{\text{Vesta}}}$ with

$$\text{cv}_i^{\text{net}} = [u_i \bmod p_{\text{Vesta}}] B_i + [r_i] R^{\text{Orchard}}.$$

For openings, c_B is defined for $B \in \mathfrak{B}$ as in Lemma 5.5, with B_i in place of AssetBase_i and u_i in place of $v'_i - v_i^{\text{new}}$. The set $\mathfrak{B}$ meets the *premise on the bases* when every $B \in \mathfrak{B}$ other than V^{Orchard} is

given with an input x_B of GroupHash such that $B = \text{GroupHash}(x_B)$ and x_B differs from the inputs $(z.\text{cash}:\text{Orchard-cv}, v)$ of V^{Orchard} and $(z.\text{cash}:\text{Orchard-cv}, r)$ of R^{Orchard} . As in the *Ironwood Guide*, a *binding signature* under a point X is a signature valid under X in the binding-signature instance of RedPallas.

Proposition 5.6 (Per-Asset balance and key knowledge). *For the bundle fixed above:*

- (i) *Identity. If $\text{cv}_i^{\text{net}} = [v'_i - v_i^{\text{new}}] \text{AssetBase}_i + [\text{rcv}_i] R^{\text{Orchard}}$ for every i , and $\text{bsk} := \sum_i \text{rcv}_i$, then*

$$\text{bvk} - [\text{bsk}] R^{\text{Orchard}} = \sum_{B \in \mathfrak{B}} [c_B] B;$$

hence $c_B \equiv 0 \pmod{p_{\text{Vesta}}}$ for every $B \in \mathfrak{B}$ implies $\text{bvk} = [\text{bsk}] R^{\text{Orchard}}$.

- (ii) *Knowledge. Given openings of every cv_i^{net} and $b \in \mathbb{F}_{p_{\text{Vesta}}}$ with $\text{bvk} = [b] R^{\text{Orchard}}$,*

$$\sum_{B \in \mathfrak{B}} [c_B] B + \left[\sum_{i=1}^n r_i - b \right] R^{\text{Orchard}} = \mathcal{O}.$$

If $\mathfrak{B}$ meets the premise on the bases and some c_B is non-zero modulo p_{Vesta} , this is a non-trivial relation among values of GroupHash at distinct inputs.

Consequently, under the Ironwood Guide's Assumptions "Discrete logarithms on Pallas", "GroupHash as a random oracle" and "The RedPallas hash as a random oracle", an efficient party that outputs a bundle whose set $\mathfrak{B}$ meets the premise on the bases, openings of every net value commitment and a binding signature under the resulting bvk on a message of its choice satisfies $c_B \equiv 0 \pmod{p_{\text{Vesta}}}$ for every $B \in \mathfrak{B}$, and knows $b = \sum_i r_i$, except with negligible probability. The statement holds for every message and does not depend on the signed message.

Under Assumption 4.9 in addition, let an efficient adversary output OrchardZSA bundles of valid blocks, hence with at most 62 500 Actions each, whose Action proofs and binding signatures verify and whose witnessed and burn-pair Asset Bases meet the premise on the bases. Then, except with negligible probability, as integers:

- (a) *for every Asset Base $B \neq V^{\text{Orchard}}$, $\sum_{i: \text{AssetBase}_i = B} (v'_i - v_i^{\text{new}}) = v_B$, which is 0 when B is not burnt;*
- (b) *$\sum_{i: \text{AssetBase}_i = V^{\text{Orchard}}} (v'_i - v_i^{\text{new}}) = v^{\text{balance}}$;*
- (c) *no burn pair has an Asset Base that no Action witnesses;*
- (d) *the signer knows the trapdoor sum $\sum_i \text{rcv}_i$.*

Here AssetBase_i , v'_i , v_i^{new} and rcv_i are read from the extracted witnesses. Status (Definition 1.3): proved here for the specified bvk ; conditional on Assumption 4.9, which concerns the designed but unspecified circuit, and not on Assumption 6.13. The premise on the bases is discharged for every accepted bundle of a valid chain only in "Security", in Theorem 9.5: for witnessed bases by Corollary 9.4, for burn-pair bases by rule (T4) of Construction 7.3 and Lemma 7.7. ZIP 226 argues per-Asset balance in rationale only ("Rationale for Value Balance Verification"), and that argument stays designed but unspecified.

Proof. (i) Expand each cv_i^{net} and apply the additive homomorphism (*Crypto Guide*, §“The Pedersen commitment”, Theorem “Additive homomorphism”):

$$\sum_{i=1}^n \text{cv}_i^{\text{net}} = \sum_{B \in \mathfrak{B}} \left[\sum_{i: \text{AssetBase}_i = B} (v'_i - v_i^{\text{new}}) \right] B + [\text{bsk}] R^{\text{Orchard}}.$$

By (B3) each Asset Base of a burn pair occurs in exactly one subtracted term $[v_B] B$, and by (B1) none of them is V^{Orchard} , whose only subtracted term is $[v^{\text{balance}}] V^{\text{Orchard}}$. Subtracting these terms as in Definition 5.4 and grouping by base gives the identity. If every c_B vanishes modulo p_{Vesta} , the right-hand side is $\mathcal{O}$.

(ii) With the openings in place of the honest values, the same grouping gives $\text{bvk} = \sum_B [c_B] B + [\sum_i r_i] R^{\text{Orchard}}$; equating with $[b] R^{\text{Orchard}}$ gives the relation. Its coefficients are scalars modulo p_{Vesta} . Under the premise, the base V^{Orchard} is the value at its input, and each other $B \in \mathfrak{B}$ at x_B . The points of $\mathfrak{B}$ are pairwise distinct, so the chosen inputs are pairwise distinct, one per point; two distinct inputs with one value would themselves be a relation of the same kind. The inputs differ from that of R^{Orchard} , by the premise and since the inputs of V^{Orchard} and R^{Orchard} differ in the message. A non-zero c_B makes the relation non-trivial.

Consequence. Let an efficient party $\mathcal{A}$ output, with probability ϵ , a bundle meeting the premise, openings, a message and a binding signature on it under the resulting bvk , with some c_B non-zero modulo p_{Vesta} ; the event is efficiently decidable from the output. The value b is extracted exactly as in the proof of the *Ironwood Guide*’s Proposition “Balance and key knowledge”: if $\text{bvk} = \mathcal{O}$, then $b := 0$; otherwise special soundness with forking in the random-oracle model for the RedPallas hash yields b with $\text{bvk} = [b] R^{\text{Orchard}}$ (*Crypto Guide*, §“Binding signatures as a proof of knowledge of a discrete logarithm”, Theorem “The binding signature is a proof of knowledge of $\log_p B$ ”). Key prefixing places bvk in the critical query, so the openings and the bases of the first run satisfy the hypothesis of part (ii) for it. Part (ii) then outputs a non-trivial relation among values of GroupHash at distinct inputs, which the *Ironwood Guide*’s Lemma “Relations among fixed generators” excludes except with negligible probability; so ϵ is negligible. In the model of that volume’s Assumption “GroupHash as a random oracle” the inputs x_B may equally be read from the queries of $\mathcal{A}$: a point that equals the value at an input $\mathcal{A}$ never queried occurs with probability at most $1/p_{\text{Vesta}}$ for each point. For Asset Bases of Custom Assets, Lemma 2.8 places every input under z.cash:OrchardZSA apart from those of V^{Orchard} and R^{Orchard} . When every c_B vanishes, the relation reduces to $[\sum_i r_i - b] R^{\text{Orchard}} = \mathcal{O}$, and R^{Orchard} has prime order p_{Vesta} , so $b = \sum_i r_i$.

Bundles. The adversary outputs polynomially many bundles, and the union bound reduces the claim to one of them, as in the proof of the *Ironwood Guide*’s Theorem “Balance”. The algorithm that runs the adversary with the extractor of Assumption 4.9 outputs, for that bundle, the triples $(\text{AssetBase}_i, v'_i - v_i^{\text{new}}, \text{rcv}_i)$ read from the witnesses; by A4’ of Definition 4.6 each is an opening of cv_i^{net} , except with the negligible probability that a witness fails the relation. The bundle’s binding signature verifies on SigHash under bvk , so the algorithm is a party of the consequence, and every c_B vanishes modulo p_{Vesta} and $b = \sum_i \text{rcv}_i$, which is (d). The hypotheses of Lemma 5.5 hold for the witnesses, the bundle being in a valid block, and the lemma lifts each congruence to an equality of integers, which is (a) and (b). A burn pair of base B that no Action witnesses has $c_B = -v_B$, with $0 < v_B < p_{\text{Vesta}}$ by (B2), so $c_B \neq 0$; this is (c). $\square$

The premise on the bases cannot be dropped, for witnessed bases or for the bases of burn pairs. For witnessed bases this is Proposition 4.1. For burn pairs, let B be the Asset Base of an issued Asset and $0 < v \leq \text{MAX_BURN_VALUE}$. Split Actions of base B contribute $v' = 0$, so Actions of base B whose net values sum to $-2v$ create $2v$ units of B from nothing. The pair $([-2] B, v)$ satisfies (B2) and (B3), and (B1) unless $[-2] B = V^{\text{Orchard}}$, a relation that Lemma “Relations among fixed generators” of the *Ironwood Guide* excludes except with negligible probability. With balancing value 0 and $t := \sum_i \text{rcv}_i$, Definitions 3.8 and 5.4 give

$$\text{bvk} = [-2v] B + [t] R^{\text{Orchard}} - [v] ([-2] B) = [t] R^{\text{Orchard}},$$

whose discrete logarithm the author knows, so the binding signature verifies. Likewise, with balancing value 0, ZEC Actions whose net values sum to $-v$ and the pair $([-1] V^{\text{Orchard}}, v)$ give

$$\text{bvk} = [-v] V^{\text{Orchard}} + [t] R^{\text{Orchard}} - [v] ([-1] V^{\text{Orchard}}) = [t] R^{\text{Orchard}} :$$

an amount of v zatoshi of ZEC appears from nothing, and (B1) does not exclude the pair, since $[-1] V^{\text{Orchard}} \neq V^{\text{Orchard}}$ in a group of odd order. Rule (B1) excludes only V^{Orchard} ; the premise on burn-pair bases is discharged by the issuance state, through rule (T4) of Construction 7.3 and Lemma 7.7, not by the burn rules (ZIP 226, “Rationale for Split Notes”, on inputs that are a multiple or linear combination of an existing Asset Base; ZIP 226, “Additional Consensus Rules for the assetBurn set”).

Remark 5.7 (Homomorphism across Asset Bases). ZIP 226, “Rationale for Value Balance Verification”, states that the relation between bvk and bsk holds “if and only if”, per Custom Asset, the net values sum to the burnt amount (0 if absent) and, for ZEC, to the balancing value, and that the commitments “add up homomorphically only with respect to the same value base point”. Neither is a literal statement about the Pallas group. The group is cyclic of prime order p_{Vesta} (protocol specification, §“Pallas and Vesta”), so any two non-identity bases satisfy $B' = [\lambda] B$ for some λ ; commitments to distinct bases add homomorphically as group elements; and $\sum_B [c_B] B = \mathcal{O}$ has non-zero solutions (c_B). The “if” direction is part (i) of Proposition 5.6. The “only if” direction holds computationally, for group-hash bases and an efficient signer, as part (ii) and its consequence prove, and then as integers by Lemma 5.5. The argument of the ZIP is rationale and keeps its class (Definition 1.3).

6 Issuance of Custom Assets

This section constructs the objects of ZIP 227 that create value of a Custom Asset: the Issue Note and the rule by which validators compute its commitment (§6.1); the Issuance Action and the Issuance Bundle that carry Issue Notes (§6.2); the element ρ of an Issue Note, derived from the first nullifier of the Action Group (§6.3); the reference note (§6.4); and the rules and the digest hypothesis that bind an Issuance Bundle to its issuer and to one transaction (§6.5). Issuance is public. The one privacy property that it keeps, the unlinkability of a later spend of an Issue Note, is proved in §6.1. The rules that update the global issuance state are those of “The global issuance state” (§7). Every construction of the section is taken from ZIP 227, which is Draft, together with the objects of ZIP 226 that it uses.

6.1 Issue Notes

Definition 6.1 (Issue Note). An *Issue Note* represents the issuance of a value v of one Custom Asset to one recipient. It is a tuple

$$(d, \text{pk}_d, v, \text{AssetBase}, \rho, \text{rseed})$$

of type

$$\begin{aligned} \text{Note}^{\text{Issue}} := & \{0, 1\}^{88} \times (\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) \setminus \{\mathcal{O}\}) \times \{0, \dots, 2^{64} - 1\} \\ & \times \mathcal{E}^* \times \mathbb{F}_{p_{\text{Pallas}}} \times \mathbb{B}^{\mathbb{Y}[32]}, \end{aligned}$$

with the following components.

1. The pair (d, pk_d) is the recipient's address, of the type of the *Ironwood Guide*'s Construction "Diversified address": the diversifier d and the diversified transmission key pk_d , a point other than the identity (ZIP 227 types it in $\text{KA}^{\text{Orchard}}.\text{Public}$, protocol specification, §"Orchard Key Components").
2. The value v counts units of the Asset. The lengths 88 and 64 are the constants ℓ_d and ℓ_{value} of the protocol specification, §"Constants".
3. The point $\text{AssetBase} \in \mathcal{E}^*$ is the Asset Base of the Custom Asset issued (Construction 2.10).
4. The element ρ is fixed by Construction 6.7, in "Computation of ρ for Issue Notes" (§6.3).
5. The note seed rseed is a 32-byte string that the issuer MUST sample uniformly at random.

An Issue Note carries no ψ and no rcm ; both are to be derived from rseed (Remark 6.2). ZIP 227 writes the fourth and fifth factors of the type as P^* and $\mathbb{F}_{q_p}$ (Table 1; ZIP 227, "Issue Note"). Class: specified.

Commitments of Issue Notes. No note commitment of an Issue Note is transmitted. Every validator MUST compute the commitment cm of each Issue Note by $\text{NoteCommit}^{\text{OrchardZSA}}$ (Definition 3.2), exactly as for an OrchardZSA note with the same fields, and, when the transaction is included in the block chain, append its extracted form cmx to the note commitment tree after the commitments of the Action Group's Actions. The order is Definition 7.4, in "State transition of a transaction" (§7.2), and the tree is the one that Remark 1.7 leaves open (ZIP 227, "Issue Note", "Specification: Consensus Rule Changes" and "Addition to the Note Commitment Tree"). Class: specified; its evaluability is qualified by Remark 6.2.

Remark 6.2 (Commitment inputs of an Issue Note). The OrchardZSA note of an Issue Note $(d, \text{pk}_d, v, \text{AssetBase}, \rho, \text{rseed})$ is $(d, \text{pk}_d, v, \text{AssetBase}, \rho, \psi, \text{rcm})$, with ψ and rcm derived from

rseed as in Definition 3.5. With $\underline{\rho} := \text{I2LEOSP}_{256}(\rho)$ and $\mathbf{g}_d := \text{DiversifyHash}(d)$,

$$\begin{aligned}\psi &:= \text{ToBase}(\text{PRFexpand}_{\text{rseed}}([\mathbf{0x09}] \parallel \underline{\rho})), \\ \text{rcm} &:= \text{Derive_rcm}_{\text{rseed}}(\text{leadByte}, \mathbf{g}_d^*, \mathbf{pk}_d^*, v, \underline{\rho}, \psi).\end{aligned}$$

The derivation of ψ does not depend on the lead byte and is specified. The derivation of rcm is designed but unspecified, and the value of its lead byte is an open problem (Remark 3.6). An Issue Note has no plaintext that could carry a lead byte. Three consequences follow.

- (a) Until the value is assigned, the commitment that every validator **MUST** compute is not computable from live text. The gap is at consensus level. For the created note of a transfer the same gap binds only sender and recipient, since consensus sees only cmx.
- (b) The recipient needs the same rcm to open the commitment when it spends the note, in condition A1' of Definition 4.6.
- (c) The *Ironwood Guide*'s Proposition "Hiding under the derived trapdoor" does not transfer to Issue Notes. Hiding plays no role for them: every field, rseed included, is public (Remark 6.6).

(ZIP 227, "Issue Note" and "Specification: Consensus Rule Changes"; ZIP 226, "Note Structure and Commitment"; ZIP 2005, "Changes to the Protocol Specification", amending the protocol specification's §"Sending Notes (Orchard)").

Remark 6.3 (Commitment $\perp$ for an Issue Note). The commitment $\text{NoteCommit}^{\text{OrchardZSA}}$ may return $\perp$. For a note created by an Action, the sender repeats the derivation with a new rseed when the extracted commitment is $\perp$ (protocol specification, §"Sending Notes (Orchard)"). ZIP 227 states no rule for an Issue Note whose commitment is $\perp$, and no text proposes one: the consequence for the transaction is an open problem. An Issue Note is in the custom branch of Definition 3.2, its Asset Base being a derived one (Proposition 2.13), where the commitment is $\perp$ only if the hash point is; for uniform rseed the event therefore has negligible probability, by Proposition 3.3(c) (ZIP 227, "Issue Note" and "Specification: Consensus Rule Changes").

Proposition 6.4 (Unlinkability of spends of Issue Notes). *Let a key be generated with $\text{use_qsk} = \text{false}$ from a uniform spending key, with nullifier deriving key nk . Consider the game of the Ironwood Guide's Proposition "Nullifier unlinkability", modified as follows. The adversary, which receives (dk, ivk) and ak^{P} but not nk , submits adaptively Issue Notes n_i of its choice, in particular to addresses of the key, each published in full as issuance publishes it, together with elements $\psi_i \in \mathbb{F}_{\text{Pallas}}$ and $\text{rcm}_i \in \mathbb{F}_{\text{P_Vesta}}$ of its choice, such that the elements ρ_i are pairwise distinct and the OrchardZSA commitments cm_i of the notes $(d_i, \text{pk}_{d_i}, v_i, \text{AssetBase}_i, \rho_i, \psi_i, \text{rcm}_i)$ are not $\perp$. In the real game it receives the nullifier $\text{DeriveNullifier}_{\text{nk}}(\rho_i, \psi_i, \text{cm}_i)$ that a later spend of the note publishes by condition A5' with $\text{split_flag} = 0$; in the ideal game it receives $\text{Extract}_{\text{P}}([u_i] K^{\text{Orchard}} + \text{cm}_i)$ for independent uniform $u_i \in \mathbb{F}_{\text{P_Vesta}}$. Then its distinguishing advantage is at most the bound of the cited proposition, which is negligible under the assumptions that proposition names, the Ironwood Guide's Assumption "Poseidon is a PRF" among them.*

The ideal answers are independent of the published notes when $K^{\text{Orchard}} \neq \mathcal{O}$, so a spend's nullifier does not link it to its issuance for any party without nk, the issuer included. The statement holds for every ψ_i and every rcm_i , in particular for the derived ψ_i and for any value that a future lead byte gives rcm_i ; it is therefore independent of the open lead byte. It concerns the nullifier only; the other published components of the spending Action are treated in “Hiding of the Asset type” (§9.5). It fails for the reference note (Definition 6.10), whose recipient key is the all-zero spending key: its nk is public, and its nullifier is computable by anyone. Status (Definition 1.3): proved here for the specified construction; conditional on the assumptions of the cited proposition. ZIP 227, “Issue Note”, states the property as rationale.

Proof. The proof is that of the *Ironwood Guide*'s Proposition “Nullifier unlinkability”. Its game already lets the adversary choose every field of every submitted note, so publishing an Issue Note in full, rseed included, gives the adversary nothing that it does not hold. Its hybrids use a submitted note only through the triple $(\rho_i, \psi_i, \text{cm}_i)$, with cm_i a point other than $\perp$ and the ρ_i pairwise distinct: the hops H_1 to H_3 change the key material only, the hop H_4 uses the distinctness of the ρ_i for the lazily sampled random function, and the hop H_5 and the final statement use cm_i only as a point to which $[s_i] K^{\text{Orchard}}$ or $[u_i] K^{\text{Orchard}}$ is added. The nullifier of an OrchardZSA note is *DeriveNullifier* of the *Ironwood Guide*'s Definition “Nullifier” with the OrchardZSA commitment in place of the Orchard one (ZIP 226, “Note Structure and Commitment”; §4.2). Every hybrid and every bound therefore carries over verbatim. The hypothesis that the ρ_i are pairwise distinct is supplied, for the Issue Notes of a valid chain, by Proposition 6.9, except with negligible probability.

For the reference note, $\text{nk} := \text{ToBase}(\text{PRFexpand}_{\text{sk}}([\text{0x07}]))$ with sk the all-zero string (*Ironwood Guide*, §“The spending key and the spend-side secrets”) is a public constant, so the real answer is computable from public data and differs from the ideal one. $\square$

6.2 Issuance Actions and Issuance Bundles

ZIP 227 specifies issuance through three components: the Issuance Bundle of a transaction, in which a publicly named issuer creates Issue Notes; the flag *finalize* of each Issuance Action, which, once set, ends issuance of its Asset permanently; and the global issuance state kept by every node, constructed in “The global issuance state” (§7).

Definition 6.5 (Issuance Action and Issuance Bundle). Let

$$\text{IssueNoteFields} := \{0, 1\}^{88} \times (\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) \setminus \{\mathcal{O}\}) \times \{0, \dots, 2^{64} - 1\} \times \mathbb{F}_{p_{\text{Pallas}}} \times \mathbb{B}^{\mathbb{Y}[32]},$$

the fields $(d, \text{pk}_d, v, \rho, \text{rseed})$ of an Issue Note without its Asset Base.

1. An *Issuance Action* is a triple

$$(\text{assetDescHash}, v\text{Notes}, \text{flagsIssuance}),$$

where the hash $\text{assetDescHash} \in \mathbb{B}^{\mathbb{Y}[32]}$ is the description hash of Definition 2.6; $v\text{Notes}$ is a finite sequence of elements of IssueNoteFields , the unencrypted Issue Notes of the Asset; and flagsIssuance is a byte that carries the Boolean *finalize*, which the issuer sets to signal that no further issuance of the Asset will occur.

2. An *Issuance Bundle* is a triple

$$(\text{issuer}, \text{vIssueActions}, \text{issueAuthSig}),$$

where $\text{issuer} = \text{ik_encoding} \in \mathbb{B}^{Y[33]}$ is the issuer identifier of Definition 2.4; vIssueActions is a finite sequence of Issuance Actions; and $\text{issueAuthSig} = [0x00] \parallel \sigma$ encodes a signature σ by isk on the transaction's SigHash (Definition 6.12).

The pair $(\text{issuer}, \text{assetDescHash})$ is an Asset Identifier and determines AssetBase by Construction 2.10. With it, each element $(d, \text{pk}_d, v, \rho, \text{rseed})$ of vNotes determines the Issue Note $(d, \text{pk}_d, v, \text{AssetBase}, \rho, \text{rseed})$ of Definition 6.1 completely. The commitment is omitted, because every validator recomputes it (ZIP 227, “Issuance Action”, “Issuance Bundle”, “Issue Note” and “Issuance Protocol”). Class: specified.

Encoding status. The byte encoding of the Issuance Bundle and of the Issuance Action exists only in withdrawn ZIP 230 and is designed but unspecified; it is not the version-6 format of ZIP 229 (Remark 8.1; ZIP 227, “Issuance Action” and “Issuance Bundle”).

Issuance Actions without notes. The rules of ZIP 227 place no requirement on the number of Issue Notes of an Issuance Action. Withdrawn ZIP 230 admits an Issuance Action with none, so that an issuer can finalise an issued Asset without issuing more of it. The validity of such an action for an Asset without an entry in the issuance state is the open case of Remark 7.5, in “State transition of a transaction” (§7.2) (ZIP 227, “Specification: Consensus Rule Changes”; ZIP 230, “Issuance Action Description”).

Remark 6.6 (Public issuance). ZIP 227 enables only public issuance, so that the supply of each Asset can be tracked (ZIP 227, “Motivation”; “Requirements”: the issuance mechanism should enable public tracking of the supply). The Issue Notes are unencrypted. The issuer, the Asset (through assetDescHash , from which anyone recomputes AssetBase), every value, every recipient address, every ρ and rseed , and every flag finalize are public, so issuance publicly links each recipient address to an Asset and an amount. Since rseed is public, so is ψ , and so, once the lead byte is assigned, are the commitment and the leaf of each Issue Note. With the public burn set (Definition 5.1), the amount of each Asset in circulation, issued less burnt, is a public function of the chain, which ZIP 227 records as the balance of the global issuance state (“Global Issuance State”). The later use of an Issue Note remains private, to the extent of Proposition 6.4. In this volume issuance is called public and Issue Notes unencrypted (ZIP 227, “Issuance Action”).

6.3 Computation of ρ for Issue Notes

An Issue Note is created outside any Action, so the *Ironwood Guide*’s Construction “Chaining rule” (§“Nullifier chaining”) gives it no ρ .

Construction 6.7 (Derivation of issued rho values). Define

$$\text{DeriveIssuedRho}: \mathbb{F}_{p_{\text{Pallas}}} \times \{0, \dots, 2^{32} - 1\} \times \{0, \dots, 2^{32} - 1\} \rightarrow \mathbb{F}_{p_{\text{Pallas}}}$$

by

$$\begin{aligned} & \text{DerivelIssuedRho}(\text{nf}, i_A, i_N) \\ & := \text{ToBase}(\text{PRFexpand}_{\text{I2LEOSP}_{256}(\text{nf})}([\text{0x84}] \parallel \text{I2LEOSP}_{32}(i_A) \parallel \text{I2LEOSP}_{32}(i_N))), \end{aligned}$$

with PRFexpand the *Ironwood Guide*'s Construction "Expansion function" and ToBase its Definition "Field reductions". The expansion input after the key has $1 + 4 + 4 = 9$ bytes. The element ρ of the Issue Note at zero-based position i_N of the Issuance Action at zero-based position i_A of the Issuance Bundle is

$$\rho := \text{DerivelIssuedRho}(\text{nf}_{0,0}, i_A, i_N),$$

where $\text{nf}_{0,0}$ is the nullifier published by the first Action of the transaction's Action Group (Definition 4.13), whatever the kind of its consumed note. The definition presupposes an Action Group, which rule (I1) of Definition 6.12 enforces. The derivation adds one row to the *Ironwood Guide*'s Construction "Domain bytes of PRF expansion":

b	Key κ	Remainder t'	f	Output	Section
0x84	$\text{I2LEOSP}_{256}(\text{nf}_{0,0})$	$\text{I2LEOSP}_{32}(i_A) \parallel \text{I2LEOSP}_{32}(i_N)$	ToBase	ρ	§6.3

The key is a public nullifier. No other ZIP and no section of the protocol specification uses the byte 0x84, and no registry of a lower volume lists it. Because the key is public, neither the *Ironwood Guide*'s Assumption "Pseudorandomness of PRF expansion" nor its Lemma "Independence of domain-separated expansions" applies (ZIP 227, "Computation of ρ "; protocol specification, §"Pseudo Random Functions" and §"Orchard Key Components"). Class: specified.

Assumption 6.8 (BLAKE2b-512 as a random oracle). For each fixed 16-byte personalisation P , the function $x \mapsto \text{BLAKE2b-512}(P, x)$ is a random oracle with 512-bit output in the sense of the *Crypto Guide*'s Definition "Random oracle" (§"The random oracle model"), independent of the oracles of the other personalisations. It is used for the personalisation Zcash_ExpandSeed of PRFexpand, where the key of Construction 6.7 is public. For a secret uniform key and an adversary that makes polynomially many oracle queries, it implies the *Ironwood Guide*'s Assumption "Pseudorandomness of PRF expansion". Its uses in this volume are Proposition 6.9 and Lemma 9.6, whose proof programs one query of the oracle.

Proposition 6.9 (Uniqueness of issued rho values).

- (a) The expansion inputs, key and remainder, of the Issue Notes of a valid chain are pairwise distinct, and each differs from the input of every other row of PRFexpand.
- (b) Under Assumption 6.8, consider an execution in which at most q queries in total are made to the oracle of personalisation Zcash_ExpandSeed . The probability that two Issue Notes of the chain have equal ρ , or that the ρ of an Issue Note equals one of M elements ρ of notes created by Actions and fixed before the query that yields it, is at most

$$\left(\frac{q(q-1)}{2} + qM \right) \left(\frac{1}{p_{\text{Pallas}}} + 2^{-512} \right) \leq \frac{(q+M)^2}{p_{\text{Pallas}}}.$$

With no query beyond the N issued values, $q = N$.

Literal uniqueness is not claimed, nor distinctness from the ρ of notes created by later Actions, which the derivation does not control. Distinct OrchardZSA notes have distinct commitments by Proposition 3.3(b). The proof of the Ironwood Guide’s Proposition “Nullifier uniqueness” then applies with the hash base and message of each note’s branch of Definition 3.2, which unfold as in the proof of Proposition 4.5(i). So distinct notes have distinct nullifiers whatever their ρ , and a collision can affect only notes equal in every field. The proposition extends to Issue Notes part (a) of the Ironwood Guide’s Corollary “Faerie Gold resistance”, which covers only notes created by Actions. Status (Definition 1.3): proved here for the specified derivation; conditional on Assumption 6.8.

Proof. (a) Within a transaction the key is one and the index pairs (i_A, i_N) differ. Across transactions of a valid chain the keys $\text{I2LEOSP}_{256}(\text{nf}_{0,0})$ differ: $\text{nf}_{0,0}$ is a published nullifier, and the nullifier-set rule forbids a nullifier to repeat within a pool (*Ironwood Guide*, §“Nullifier sets”; the pool is that of Remark 1.7), and I2LEOSP_{256} is injective on $\mathbb{F}_{p_{\text{Pallas}}}$. The oracle input is the 32-byte key followed by the remainder. Every key of PRFexpand is a 32-byte string (*Ironwood Guide*, Construction “Expansion function”), so the byte after the key is the lead byte, and $0x84$ differs from the lead byte of every other row.

(b) By (a) each issued ρ is ToBase of the oracle value at an input of its own. In the lazy form of the *Crypto Guide*’s Definition “Random oracle”, the first query at an input returns a fresh uniform 512-bit string, independent of all earlier values. The map ToBase takes each value of $\mathbb{F}_{p_{\text{Pallas}}}$ on at most $\lceil 2^{512}/p_{\text{Pallas}} \rceil$ strings, so the fresh output equals a value fixed before the query with probability at most $\lceil 2^{512}/p_{\text{Pallas}} \rceil / 2^{512} < 1/p_{\text{Pallas}} + 2^{-512}$. Every issued ρ is the reduced output of one of the q queries. A union bound over the at most $q(q-1)/2$ pairs of queries and the qM pairs of a query and a fixed value (*Math Guide*, §“The union bound and a birthday calculation”, Theorem “Union bound / Boole’s inequality” and Proposition “Birthday bound”) gives the first bound. The second follows from $2(q(q-1)/2 + qM) \leq (q+M)^2$ and $1 + p_{\text{Pallas}} 2^{-512} < 2$. $\square$

6.4 Reference notes

Definition 6.10 (Reference note). The first Issue Note of the Issuance Action in which a Custom Asset is issued for the first time MUST be a *reference note*: an Issue Note with $v = 0$, the Asset Base of that Asset, and recipient (d, pk_d) the default diversified address, of diversifier index 0, of the all-zero Orchard spending key $\text{sk} = [0x00]^{32}$. The address is derived by the *Ironwood Guide*’s constructions of §“The spending key and the spend-side secrets”, §“Viewing keys” and Construction “Diversified address”. The all-zero key passes every acceptance condition of key generation: $\text{ask} \neq 0$; $\text{ivk} \notin \{0, \perp\}$; and the incoming viewing key of its internal key (ZIP 32, “Orchard internal key derivation”) is not in $\{0, \perp\}$, the condition of the protocol specification, §“Orchard Key Components”. At index 0 the diversified base g_d is not the identity. The raw encoding of the address, $\text{LEBS2OSP}_{88}(d) \parallel \text{LEBS2OSP}_{256}(\text{pk}_d^*)$ (protocol specification, §“Orchard

Raw Payment Addresses”), is the 43-byte string, given in ZIP 227, “Reference Notes”, as

(204, 54, 96, 25, 89, 33, 59, 107, 12, 219, 150, 167, 92, 23, 195, 166,
104, 169, 127, 13, 106, 140, 92, 225, 100, 165, 24, 234, 155, 169, 165,
14, 167, 81, 145, 253, 134, 27, 15, 241, 14, 98, 176).

Recomputing the address and every acceptance check from the cited constructions reproduces this array. Validators enforce the rule and store the reference note as n_{ref} in the global issuance state, by rule (I5) of Construction 7.3, in “State transition of a transaction” (§7.2) (ZIP 227, “Reference Notes” and “Specification: Consensus Rule Changes”). Class: specified; the reference note is mandated and stored in consensus state, and no section of ZIP 227 states its purpose.

Proposition 6.11 (Reference notes as universal Split Inputs). *Let n_{ref} be the reference note of a Custom Asset with Asset Base B , its extracted commitment a leaf of the note commitment tree.*

- (a) *Any party can construct a Split Action (Definition 4.2) whose Split Input copies n_{ref} , with a witness of Definition 4.6 and a spend-authorisation signature that verifies under its rk , for a created note of Asset Base B and value 0 to any receiver of its choice. Proposition 4.12 therefore imposes nothing on n_{ref} , whose recipient key is public by construction.*
- (b) *Let an efficient adversary output OrchardZSA bundles of valid blocks whose Action proofs and binding signatures verify and whose witnessed and burn-pair Asset Bases meet the premise on the bases of §5.3. Except with negligible probability, in every such bundle in which no extracted witness of Asset Base B has $\text{split_flag} = 0$, every created note of Asset Base B has value 0, and the burn set has no pair of Asset Base B .*

Hence n_{ref} serves exactly the created notes of B that a party holding no note of B can produce. Under Remark 1.7, a copy to a receiver other than that of the copied note would violate the cross-address rule if the notes were in the Orchard pool. Status (Definition 1.3): proved here for the specified constructions; part (a) is conditional on the open lead byte (Remark 6.2); part (b) is conditional on Assumption 4.9 through Proposition 5.6, and its premise on the bases is discharged for valid chains in “Security” (§9). The reading of the purpose is the volume’s; ZIP 227 states none.

Proof. (a) The construction is explicit. The all-zero spending key is accepted (Definition 6.10), so its ask , nk , r_{ivk} and ak^{p} are public constants. The note n_{ref} is published unencrypted with ρ and rseed (Remark 6.6), so ψ is public and, once the lead byte is assigned (Remark 6.2), so are rcm and the commitment. The position of its leaf in the tree is public, by Definition 7.4. The party takes a valid anchor rt whose tree contains the leaf and computes its authentication path, which satisfies $\text{A3}'$; it opens $\text{A1}'$ with the fields of n_{ref} and $v^{\text{old}} = 0$; it sets $\text{split_flag} = 1$, $\text{is_native_asset} = 0$ and $\text{enableZSA} = 1$, which satisfy C1 to C3 since $B \neq V^{\text{Orchard}}$; it draws ψ^{nf} and computes the split nullifier of $\text{A5}'$ with the public nk ; it creates a note of base B , value 0 and any receiver, whose commitment satisfies $\text{A2}'$, and takes any rcv for $\text{A4}'$, which reads $v' = 0$ and $v^{\text{new}} = 0$. Conditions A6 and A7 hold with the public $(\text{ak}, \text{nk}, \text{r}_{\text{ivk}})$, from which the reference address is derived; A8 holds since $v^{\text{old}} = 0$, and A9 since $v^{\text{new}} = 0$. The party signs under $\text{rsk} = \text{ask} + \alpha$ for its randomiser α , as

any owner does.

(b) By Proposition 5.6(a), except with negligible probability, the net values of the Actions of base B , read from the extracted witnesses, sum as integers to the burnt amount v_B , which is 0 if B is not burnt. Every such Action has `split_flag` = 1, so its value v' is 0 by the notation of Definition 4.6, and the sum is $-\sum_i v_i^{\text{new}} \leq 0$. A burn pair has $v_B > 0$ by rule (B2) of Definition 5.1, so $v_B \geq 0$. Both sides therefore vanish: every v_i^{new} of base B is 0, and $v_B = 0$ excludes a pair of base B . $\square$

6.5 Issuance authorisation

Definition 6.12 (Issuance authorisation rules). For a transaction that contains an Issuance Bundle (`issuer`, `vIssueActions`, `issueAuthSig`):

- (I1) It MUST contain an OrchardZSA Action Group; by Definition 4.13 it contains exactly one.
- (I2) The encoding `ik_encoding` of the issuance validating key `ik` MUST be taken from the field `issuer` and MUST start with the byte `0x00`, which indicates a BIP-340 public key; `ik` is the remaining 32 bytes (Definition 2.4).
- (I3) The first byte of `issueAuthSig` MUST be `0x00`, and

$$\text{IssueAuthSig.Validate}(\text{ik}, \text{SigHash}, \sigma) = 1$$

MUST hold for the 64-byte σ that follows it (Definition 2.2). ZIP 227 writes the rule with `issueAuthSig` in place of σ ; the scheme byte is stripped because `Validate` is typed over 32-byte keys and 64-byte signatures.

For every Issuance Action the Issue Notes are then constructed from the elements of its `vNotes`, with `AssetBase` derived from its `assetDescHash` and from `issuer` (Remark 6.14). The rules on the issuance state, (T1) to (T4) and (I4) to (I7), are Construction 7.3. The message `SigHash` is the transaction’s signature digest (*Ironwood Guide*, Definition “Signature digest”), which ZIP 227 defines by the protocol specification’s “SIGHASH Transaction Hashing” “with the modifications described in ZIP 226”, under `SIGHASH_ALL`. The cited section of ZIP 226 does not exist; ZIP 226, “Sighash modifications relative to ZIP 244”, defers to ZIP 246, which is Withdrawn; and the version-6 digest of ZIP 229 has no issuance child (ZIP 227, “Specification: Consensus Rule Changes” and “Issuance Authorization Signing and Validation”). Class: rules (I1) to (I3) specified; the signed message designed but unspecified (Remark 8.3).

Assumption 6.13 (Coverage of the signature digest). The signature digest `SigHash` of an OrchardZSA transaction is computed without any signature of the transaction: neither `issueAuthSig`, nor a spend-authorisation signature, nor the binding signature is an input. No efficient algorithm outputs, except with negligible probability, two transactions with equal `SigHash` that differ in

- (i) the effecting data of the Issuance Bundle: `issuer`, the number and order of the Issuance Actions, and, per Issuance Action, `assetDescHash`, `finalize` and every field

- (i) $(d, pk_d, v, \rho, rseed)$ of every Issue Note;
- (ii) the burn set; or
- (iii) the published data of the Actions of the Action Group, in particular $nf_{0,0}$.

This hypothesis (H-SIG) concerns a designed but unspecified object. “Transaction digests and the signature message” (§8.2) shows that the digest of withdrawn ZIP 246 meets it under the *Ironwood Guide*’s Assumption “Collision resistance of BLAKE2b”, and states it as the obligation on any future digest (Remark 8.2). The version-6 digest of ZIP 229 does not meet (i).

Remark 6.14 (Recomputation of issued Asset Bases). Of the identity chain of Construction 2.10, consensus sees exactly two links on the issuance side: issuer in the Issuance Bundle and `assetDescHash` in each Issuance Action. The Asset Identifier, the Asset Digest and the Asset Base of issued notes are never transmitted. Every validator derives

$$\text{AssetBase} := \text{ZSAValueBase}(\text{BLAKE2b-512}(\text{ZSA-Asset-Digest}, \text{EncodeAssetId}(\text{issuer}, \text{assetDescHash})))$$

by that construction. The same bytes issuer name the Asset and yield, by (I2), the key under which (I3) verifies the signature; under Assumption 6.13(i), both issuer and `assetDescHash` are in the signed message. The consequence for authority over issuance is Theorem 9.8, in “Authority over issuance” (§9.4) (ZIP 227, “Specification: Consensus Rule Changes” and “Issuance Bundle”).

Remark 6.15 (Signing order of an issuance bundle). Every ρ of an Issue Note is a function of $nf_{0,0}$ (Construction 6.7), fixed once the first Action of the Action Group is fixed. Under Assumption 6.13 the signed digest covers every ρ and every other effecting field of the transaction. The issuer can therefore sign only after the nullifier of the first Action and all effecting data are fixed. Its message is the SIGHASH_ALL digest that every spend-authorisation signature and the binding signature sign (*Ironwood Guide*, Definition “Signature digest”; ZIP 227, “Computation of ρ ”, “Issuance Protocol” and “Specification: Consensus Rule Changes”).

Proposition 6.16 (Replay protection of issuance). *Let ik be the issuance validating key of an honest issuer, which signs only digests SigHash of transactions it builds. Except with negligible probability, in a valid chain produced by an efficient adversary that obtains the honest issuer’s signatures:*

- (a) every Issuance Bundle whose issuer is `ik_encoding` of ik belongs to a transaction whose SigHash the issuer signed;
- (b) no transaction of the chain that contains an Issuance Bundle has the SigHash of another transaction of the chain.

Hence each digest that the issuer signs authorises the issuance of at most one transaction of the chain. Rule (I1) is what makes (b) hold: without an Action Group, the digest of a transaction that carries only an Issuance Bundle depends on no value that the chain forbids to repeat, and a duplicate of the bundle in a transaction with the same remaining digest inputs has the same

SigHash, so its signature verifies again (ZIP 227, “Rationale”). *Status (Definition 1.3): proved here for the specified rules (I1) to (I3); conditional on Assumption 6.13, a hypothesis on the designed but unspecified signed message, and on Assumption 2.3. ZIP 227, “Rationale”, states the replay argument informally.*

Proof. (a) An accepted bundle satisfies (I2) and (I3), so its signature verifies on the transaction’s SigHash under the key whose encoding is issuer, which is ik by the injectivity of $ik \mapsto \text{issuer}$ (Definition 2.4). A verifying signature on a digest that the issuer never signed is an existential forgery against Assumption 2.3: the reduction answers the adversary’s requests for the issuer’s signatures with its signing oracle and outputs the digest and the signature of the chain.

(b) By (I1) a transaction with an Issuance Bundle has an Action Group, and so a nullifier $nf_{0,0}$. Another transaction of the chain with equal SigHash has, by Assumption 6.13(iii), the same published Action data, except with negligible probability, since the chain is the output of an efficient algorithm; in particular it publishes $nf_{0,0}$ as well. The nullifier-set rule forbids a nullifier to repeat across the transactions of a valid chain (*Ironwood Guide*, §“Nullifier sets”), so no such transaction exists. By (a) and (b) the map from the transactions of the chain that carry a bundle under ik to the digests that the issuer signed is injective, which is the final claim. $\square$

7 The global issuance state

This section constructs the record that every node keeps of the issued Custom Assets: the global issuance state of ZIP 227 and its chaining through blocks and transactions (§7.1); the transition of that state under one transaction, in which burn precedes issuance, together with the order in which the note commitments of a transaction enter the note commitment tree (§7.2); the properties of finalisation and of the recorded balance (§7.3); and the complete verification of an OrchardZSA transaction, with each check paired with the requirement that it serves (§7.4). The rules are those of ZIP 227, which is Draft; one case that they leave undetermined is an open problem (Remark 7.5).

7.1 The issuance state and its chaining

Definition 7.1 (Global issuance state). Let

$$\text{MAX_ISSUE} := 2^{64} - 1 = 18\,446\,744\,073\,709\,551\,615,$$

the largest value of one Issue Note (Definition 6.1). The *global issuance state* is a finite partial map

$$\begin{aligned} \text{issued_assets} &: \mathcal{E}^* \rightarrow \{0, \dots, \text{MAX_ISSUE}\} \times \{0, 1\} \times (\text{Note}^{\text{Issue}} \cup \{\perp\}), \\ \text{AssetBase} &\mapsto (\text{balance}, \text{final}, \text{note}_{\text{ref}}), \end{aligned}$$

from Asset Bases to triples, with $\mathcal{E}^*$ as in §2.3. The components are written

$$\begin{aligned} \text{issued_assets}(\text{AssetBase}).\text{balance}, \quad & \text{issued_assets}(\text{AssetBase}).\text{final}, \\ & \text{issued_assets}(\text{AssetBase}).\text{note}_{\text{ref}}. \end{aligned}$$

An Asset Base outside the domain of the map, written

$$\text{issued_assets}(\text{AssetBase}) = \perp,$$

reads as $(0, 0, \perp)$, and a write to a component of such an Asset Base creates its entry from that default. The components mean the following.

1. The *balance* is the amount of the Asset in circulation: the amount issued less the amount burnt.
2. The *finalisation bit* final records whether $\text{finalize} = 1$ occurred in an earlier issuance of the Asset; it never changes from 1 to 0.
3. The *reference note* note_{ref} is the Asset's reference note (Definition 6.10), or $\perp$.

ZIP 227 types the third component in $\text{Note}^{\text{issue}}$ while its default reads $\perp$; it is typed here in $\text{Note}^{\text{issue}} \cup \{\perp\}$. The bound MAX_ISSUE applies to the balance, the amount in circulation, and not to cumulative issuance: after burns an unfinalised Asset can be issued again, so the sum of all values issued of one Asset may exceed MAX_ISSUE . The phrases “maximum total supply” and “limit the total issuance” of ZIP 227 are looser than its rule (I6) of Construction 7.3, which the volume follows.

Chaining. Every node MUST update the map while processing any transaction that contains a burn set or an Issuance Bundle. A transaction has an *input state* $\text{issued_assets}_{\text{IN}}$ and an *output state* $\text{issued_assets}_{\text{OUT}}$:

1. the input state of the first transaction of the OrchardZSA activation block is the empty map;
2. the input state of the first transaction of any later block is the final state of the preceding block;
3. the input state of every later transaction of a block is the output state of the preceding transaction;
4. the *final state* of a block is the output state of its last transaction.

The chaining is that of treestates in the *Ironwood Guide*'s Definition “Treestate and anchor” (§“Anchors”) (ZIP 227, “Global Issuance State” and “Management of the Global Issuance State”). Class: specified. The OrchardZSA activation block is not assigned, since ZIP 227, “Deployment”, reads “TBD”; this is an open problem, listed in “Open problems” (§10.2).

ZIP 227, “Rationale for Global Issuance State”, gives three reasons, which are rationale of the ZIP. The balance of an issued Custom Asset must never become negative, along the lines of ZIP 209, and no single transaction field carries both the issued and the burnt amounts, so nodes keep a running record of the amount in circulation. The bound MAX_ISSUE is a practical limit that lets an issuer issue the complete supply of an Asset in one transaction, indeed in one Issue Note. The finalisation bit lets nodes reject further issuance of a finalised Asset. The properties themselves are

proved in “Finalisation” (§7.3).

Remark 7.2 (Issuance state under reorganisation). By the chaining of Definition 7.1, the final state of a block is a function of the branch from the OrchardZSA activation block to that block. A reorganisation (*Ironwood Guide*, Definition “Reorganisation”) therefore replaces the state by the one obtained by chaining along the new branch from the last common block, as it replaces the treestates. ZIP 227 states forward chaining only and no rule for reorganisations: the statement here is a consequence of the definition, not a rule of the ZIP. Every result of this section, and Theorem 9.7, holds per branch.

7.2 State transition of a transaction

Construction 7.3 (Transition of the issuance state). *Input.* A transaction T that may contain an OrchardZSA bundle (Definition 5.2), with burn set assetBurn , empty when T has none, and an Issuance Bundle (issuer, vIssueActions , issueAuthSig) (Definition 6.5); and its input state $\text{issued_assets}_{\text{IN}}$.

Output. The output state $\text{issued_assets}_{\text{OUT}}$, defined only when every requirement below holds; otherwise T is invalid and contributes no state. Every write below is to $\text{issued_assets}_{\text{OUT}}$, and every read is of its current value. The labels follow the order of ZIP 227, which numbers none.

For every transaction:

- (T1) The transaction carries at most one Action Group, and that Action Group has no expiry height of its own.
- (T2) $\text{issued_assets}_{\text{OUT}} := \text{issued_assets}_{\text{IN}}$.
- (T3) The burn set assetBurn satisfies (B1) to (B3) of Definition 5.1.
- (T4) For every $(\text{AssetBase}, v) \in \text{assetBurn}$, it MUST hold that

$$\text{issued_assets}_{\text{OUT}}(\text{AssetBase}).\text{balance} \geq v;$$

the node then sets, for every pair,

$$\text{issued_assets}_{\text{OUT}}(\text{AssetBase}).\text{balance} := \text{issued_assets}_{\text{OUT}}(\text{AssetBase}).\text{balance} - v,$$

before any issuance is processed. By (B3) the pairs have distinct Asset Bases, so checking every pair and then subtracting equals processing the pairs one by one.

If T contains an Issuance Bundle: rules (I1) to (I3) of Definition 6.12 hold, and for each Issuance Action (assetDescHash , vNotes , flagsIssuance) of vIssueActions , in order, with zero-based position i_A in the bundle, let AssetBase be derived from issuer and assetDescHash by Construction 2.10 (Remark 6.14). Then:

- (I4) Every Issue Note description of the action MUST be a valid encoding, and each yields the Issue Note of Definition 6.5 with that AssetBase . It MUST hold that $\text{issued_assets}_{\text{OUT}}(\text{AssetBase}).\text{final} \neq 1$, evaluated against the state as updated by the earlier actions of the same bundle.

(I5) If $\text{issued_assets}_{\text{OUT}}(\text{AssetBase}).\text{note}_{\text{ref}} = \perp$, the first Issue Note of the action MUST be a reference note (Definition 6.10): its value MUST be 0, and its recipient (d, pk_d) MUST be the default diversified address of the all-zero Orchard spending key. The node then sets $\text{issued_assets}_{\text{OUT}}(\text{AssetBase}).\text{note}_{\text{ref}}$ to that note.

(I6) For each Issue Note of the action, at zero-based position i_N and of value v : its element ρ MUST equal $\text{DerivelIssuedRho}(\text{nf}_{0,0}, i_A, i_N)$ (Construction 6.7); it MUST hold that

$$\text{issued_assets}_{\text{OUT}}(\text{AssetBase}).\text{balance} + v \leq \text{MAX_ISSUE},$$

and the node then adds v to that balance; and the node MUST compute the note commitment of the Issue Note by Definition 3.2, whose inputs are qualified in Remark 6.2.

(I7) If $\text{finalize} = 1$ in $\text{flags}_{\text{Issuance}}$, the node sets

$$\text{issued_assets}_{\text{OUT}}(\text{AssetBase}).\text{final} := 1.$$

(ZIP 227, “Specification: Consensus Rule Changes”; ZIP 226, “Additional Consensus Rules for the assetBurn set”.) Class: specified. The encodings that (T1) and (I4) name belong to the carrier: ZIP 227 states (T1) “for every transaction” as $\text{nActionGroupsOrchard} \in \{0, 1\}$ and $\text{nAGExpiryHeight} = 0$, fields that exist only in withdrawn ZIP 230, and (I4) by reference to the field encodings of ZIP 230; that carrier is open (Remark 8.1). ZIP 227 writes the balance of (I6) as $\text{issued_assets}_{\text{OUT}}.\text{balance}$, without the argument AssetBase , which is restored here.

Four consequences are read off Construction 7.3. Burns are checked against the balance before issuance, so a transaction cannot burn units that it issues. A burn of an Asset Base without an entry fails (T4), since the default balance is 0 and (B2) gives $v > 0$. A finalised Asset remains burnable, since no rule of the burn half reads final . Since (I4) precedes (I7) within an action, one action may issue notes and finalise its Asset.

Definition 7.4 (Insertion order of note commitments). When a transaction T that passes Construction 7.3 is added to the block chain, the leaves that it appends to the note commitment tree of its treestate are, in this order:

1. for each Action of the Action Group of T , in order, the extracted commitment cmx of the note that the Action creates, whatever its value and whether or not the Action is a Split Action;
2. then, for each Issuance Action of the Issuance Bundle of T , in order, and each of its Issue Notes, in order, the extracted commitment of the note commitment computed by (I6).

The transactions of a block are taken in block order. The definition extends the *Ironwood Guide*’s Construction “State update” (§“Verification of a transaction”), which appends the commitments of Actions only. The position of every leaf, and hence every authentication path, is a function of the chain under this order, so nodes and wallets agree on positions only under it. Which pool’s tree the order extends is the open problem of Remark 1.7 (ZIP 227, “Addition

to the Note Commitment Tree”; protocol specification, §“Note Commitment Trees”). Class: specified.

Remark 7.5 (Empty Issuance Actions). The rules of ZIP 227 place no requirement on the number of Issue Notes of an Issuance Action, and withdrawn ZIP 230, “Issuance Action Description (IssueAction)”, admits an action without notes so that an issuer can finalise an Asset without issuing more of it. Consider such an action on the Asset Base `AssetBase`.

- (a) If $\text{note}_{\text{ref}} \neq \perp$ when the action is processed, an action with `finalize = 0` changes nothing, and one with `finalize = 1` sets `final` by (I7).
- (b) If $\text{note}_{\text{ref}} = \perp$, rule (I5) refers to a first Issue Note that does not exist, and the rules do not determine whether the action, finalising or not, is valid. If a finalising one were accepted, it would create the entry $(0, 1, \perp)$: an Asset finalised with no reference note and no issuance.

Class: case (b) is an open problem, listed in “Open problems” (§10.2); a rule that fixes the validity of such an action closes it.

An issuance state is *reachable* if it is the input or the output state of a transaction in a sequence of transactions, chained from the empty map as in Definition 7.1, each of which has an output state under Construction 7.3. A *valid chain* in the results of this section is such a sequence; an invalid transaction contributes no state. Every chain accepted by Construction 7.10 is a valid chain in this sense, since that procedure includes the transition.

Lemma 7.6 (First-issuance detection). *Assume that no valid chain contains an Issuance Action without Issue Notes on an Asset Base whose note_{ref} is $\perp$ when the action is processed, which is case (b) of Remark 7.5. Then in every reachable issuance state S , and for every `AssetBase`, the Asset Base is in the domain of S if and only if $S(\text{AssetBase}).\text{note}_{\text{ref}} \neq \perp$. Hence conditioning (I5) on the absence of an entry is equivalent to conditioning it on $\text{note}_{\text{ref}} = \perp$. Status (Definition 1.3): proved here for the specified rules; conditional on the hypothesis, which concerns the open case of Remark 7.5.*

Proof. Induction over the transactions of a valid chain, the invariant being checked after every write of Construction 7.3. It holds for the empty map. Rule (T2) copies a state. Rule (T4) writes only Asset Bases with `balance` $\geq v > 0$, by (B2), hence Asset Bases already in the domain, and leaves note_{ref} unchanged. Rule (I4) only reads. For an action on an Asset Base outside the domain, $\text{note}_{\text{ref}} = \perp$ and the action has at least one Issue Note by the hypothesis, so (I5) sets note_{ref} to that note before (I6) and (I7) write: the entry is created together with its reference note. For an action on an Asset Base in the domain, $\text{note}_{\text{ref}} \neq \perp$ by the invariant, and (I5) does not apply. No rule writes $\perp$ to note_{ref} or removes an entry. Both the burn step and the hypothesis are needed: without the hypothesis, a finalising action of case (b), if accepted, creates the entry $(0, 1, \perp)$ by (I7). $\square$

Lemma 7.7 (Keys of the issuance state). *In every reachable issuance state, every Asset Base in the domain equals `ZSAValueBase(AssetDigest)` for the Asset Identifier (issuer, assetDescHash) of some accepted Issuance Action (Construction 2.10), and is therefore the value of `GroupHash` at an input under `z.cash:OrchardZSA` (Table 3). For every transaction of a valid chain, the Asset Base*

of every pair of its burn set is in the domain of its input state, and is therefore of the same form. Status (Definition 1.3): proved here for the specified rules; unconditional, case (b) of Remark 7.5 included. The lemma discharges, in Theorem 9.5, the premise of Proposition 5.6 on the bases of burn pairs, which (B1) alone does not discharge.

Proof. Induction over the transactions of a valid chain. The initial map is empty. Rules (I4) to (I7) write only the entry at the Asset Base that Construction 7.3 derives from issuer and assetDescHash by Construction 2.10, whether or not the action has Issue Notes, and so does (I7) in case (b) of Remark 7.5 if such an action is accepted. Rule (T4) writes only an Asset Base in the domain: an Asset Base outside it reads balance = 0, and (B2) requires $v > 0$, so the check of (T4) fails. The same argument gives the second claim, since (T4) reads the input state, copied by (T2), before any issuance of the transaction. $\square$

7.3 Finalisation

Every Issuance Action carries the Boolean finalize in flagsIssuance (Definition 6.5). An Asset is *finalised* in a reachable issuance state S when $S(\text{AssetBase}).\text{final} = 1$ for its Asset Base. Setting finalize finalises the Asset by (I7), and every later Issuance Action for its Asset Base, with or without Issue Notes, fails (I4). Only (I7) writes final, and it writes 1, so final never returns from 1 to 0, as Definition 7.1 states. Finalisation is the only mechanism of the issuance life cycle that ZIP 227 specifies (ZIP 227, “Requirements”, “Issuance Action” and “Specification: Consensus Rule Changes”). Class: specified.

Proposition 7.8 (Finalised supply never grows). *In a reachable issuance state S of a valid chain, if $S(\text{AssetBase}).\text{final} = 1$, then in every later reachable state S' of the same chain $S'(\text{AssetBase}).\text{final} = 1$ and $S'(\text{AssetBase}).\text{balance} \leq S(\text{AssetBase}).\text{balance}$, and the balance is non-increasing from S on. The Asset can still be burnt. This is a property of the map; its relation to the notes in the tree is Theorem 9.7. Status (Definition 1.3): proved here for the specified rules; unconditional.*

Proof. Induction over the later transactions. Only (I7) writes final, and it writes 1. Rule (I4) rejects every later Issuance Action for AssetBase, with or without Issue Notes, since it reads final = 1, so (I6) never runs for that Asset Base again. The only other rule that writes balance is (T4), which subtracts. No rule of the burn half reads final, so a burn of the Asset is accepted whenever (T3) and (T4) hold. $\square$

Proposition 7.9 (The recorded balance). *In every reachable issuance state S of a valid chain and for every Asset Base B ,*

$$S(B).\text{balance} = \sum v_{\text{iss}} - \sum v_{\text{burn}} \in \{0, \dots, \text{MAX_ISSUE}\},$$

the first sum over the values of the Issue Notes of Asset Base B of the accepted Issuance Actions up to S , the second over the values of the pairs (B, v_{burn}) of the burn sets of the accepted transactions up to S . The balance increases only under (I6) and decreases only under (T4), so burn is the only mechanism that lowers it, and a burn of an Asset Base that has never been issued is rejected. The

bound is on circulation, not on cumulative issuance (Definition 7.1). This is a property of the map; its equality with the value of the notes created and not consumed is Theorem 9.7. Status (Definition 1.3): proved here for the specified rules; unconditional.

Proof. Induction over the transactions, the claim being checked after every write. For the empty map both sides are 0. Rule (T2) copies a state. Rule (T4) subtracts v after checking balance $\geq v$, which keeps the identity and balance ≥ 0 ; an Asset Base without an entry reads 0, and (B2) gives $v > 0$, so a burn of a never-issued Asset Base fails. Rule (I6) adds v after checking that the sum is at most MAX_ISSUE, which keeps the identity and the upper bound; the reference note of (I5) has value 0 and enters both sides as 0. No other rule writes balance, and a rejected transaction contributes no state. $\square$

Figure 2 draws the transitions of one entry of the map.

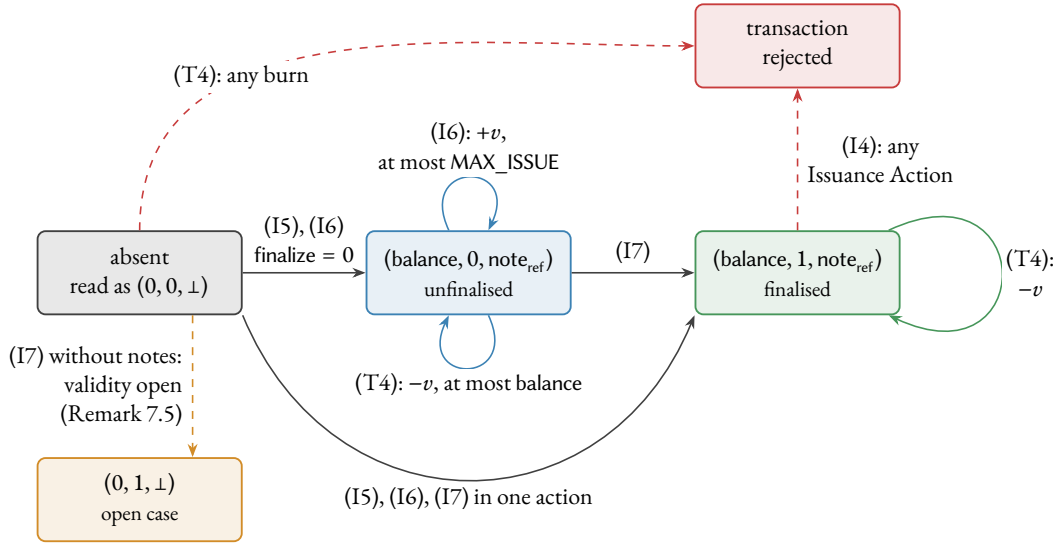


Figure 2. Transitions of one entry of the global issuance state of Definition 7.1. Solid edges are accepted transactions, labelled with the rules of Construction 7.3 that take them. The first issuance, by (I5) and (I6), creates the entry together with its reference note, as in Lemma 7.6; one action may also issue and finalise at once. The unfinalised entry grows under (I6), up to MAX_ISSUE, and shrinks under (T4), down to 0; the finalised entry only shrinks under (T4), again down to 0, and every Issuance Action on it is rejected by (I4), as in Proposition 7.8. A burn of an absent entry is rejected by (T4). The dashed edge to $(0, 1, \perp)$ is case (b) of Remark 7.5, an open problem.

7.4 Verification of an OrchardZSA transaction

The following procedure has the form of the *Ironwood Guide*’s Construction “Verification” (§“Verification of a transaction”). Each check is tagged with the requirements of Definition 1.8 that it serves and with the conditions of Definition 4.6 that it completes, and with its class in the sense of Definition 1.2: “specified” for the rule, followed by “carrier designed but unspecified” when its encoding exists only in withdrawn text, or “carrier open” when no design without conflict exists (Remark 8.1).

Construction 7.10 (Verification procedure for OrchardZSA). Given the note commitment tree and the nullifier set of the pool that carries OrchardZSA notes (Remark 1.7), and the issuance state $\text{issued_assets}_{\text{IN}}$, a transaction T is accepted, as far as its OrchardZSA bundle and its Issuance Bundle are concerned, if and only if the following checks pass.

Per Action.

- (1) *Proof.* The aggregate proof of the Action Group verifies for the relation $\mathcal{R}_{\text{ActionZSA}}$ of Definition 4.6, under the OrchardZSA verifying key of Assumption 4.9, at the primary input

$$(rt, cv^{\text{net}}, nf, rk, cmx, \text{enableSpend}, \text{enableOutput}, \text{enableZSA})$$

of each Action. [Z1, Z2, Z3 and Z5, through A1' to A5' and C1 to C3; statement specified, circuit and verifying key designed but unspecified.]

- (2) *Spend authorisation.* Each Action's spend-authorisation signature is valid under its rk on SigHash (*Ironwood Guide*, §“Randomised validating keys”; unchanged by ZIP 226). [Requirement R5 of the *Ironwood Guide*'s Definition “Requirements R1 to R5”, with A6 and A7; specified, signed message designed but unspecified.]

Per Action Group and OrchardZSA bundle.

- (3) *Flags.* The values enableSpend , enableOutput and enableZSA are read from the Action Group into every primary input of step (1) (Definition 4.13). [Z2, through C2; flags specified, the bit of enableZSA open.]
- (4) *Anchor.* The anchor rt of the Action Group is the root of the carrying pool's note commitment tree in the final treestate of an earlier block of the chain (*Ironwood Guide*, §“Anchors”). [Z2, through A3'; specified, the pool open.]
- (5) *Nullifiers.* The nullifiers of T , real and split, are pairwise distinct and absent from the carrying pool's nullifier set (*Ironwood Guide*, §“Nullifier sets”). [Z3, through A5'; specified.]
- (6) *Burn set.* The burn set satisfies (B1) to (B3) of Definition 5.1; this is (T3). [Z1 and Z3; specified, carrier designed but unspecified.]
- (7) *Balance.* The binding signature is valid on SigHash under the key bvk of Definition 5.4, computed from the Actions' net value commitments, the balancing value and the burn set; the balancing value enters the transparent transaction value pool as in the *Ironwood Guide*'s Definition “Balancing value”. [Z1 and Z2, through A4'; specified, carrier designed but unspecified; the fee terms are classified in Remark 8.4.]

Per transaction.

- (8) *Issuance authorisation.* If T contains an Issuance Bundle, rules (I1) to (I3) of Definition 6.12 hold. [Z4; specified, signed message designed but unspecified.]

- (9) *Issuance state.* The output state $\text{issued_assets}_{\text{OUT}}$ of Construction 7.3 is defined: rules (T1), (T2) and (T4), and (I4) to (I7) for every Issuance Action. [(T1): carrier open; (T4): Z2 and Z3; (I4): Z2, Z3 and Z4; (I5) to (I7): Z3; specified.]

State update, on accepting the block that contains T.

- (10) The leaves of Definition 7.4 are appended to the carrying pool’s note commitment tree. [Z2 and Z3; specified, the pool open.]
- (11) Every nullifier of T , real and split, enters the carrying pool’s nullifier set. [Z3; specified.]
- (12) The issuance state becomes $\text{issued_assets}_{\text{OUT}}$, chained as in Definition 7.1; the ZEC chain value pool balance of the carrying pool decreases by the balancing value, as in the *Ironwood Guide*’s Construction “State update”. [Z3; specified.]

The remaining checks of the *Ironwood Guide*’s Construction “Verification”, on the transaction’s other components and on its expiry, apply unchanged; the parsing of the OrchardZSA components depends on the carrier, which is open (Remark 8.1) (ZIP 226, “Circuit Statement”, “Additional Consensus Rules for the assetBurn set” and “Value Balance Verification”; ZIP 227, “Specification: Consensus Rule Changes” and “Addition to the Note Commitment Tree”; protocol specification, §“Transactions and Treestates”, §“Nullifier Sets” and §“Note Commitment Trees”). Class: every rule specified; the classes of the carrier, the circuit, the signed message and the pool are as tagged.

The division of the checks follows the *Ironwood Guide*’s Remark “Division of the checks”. Under Assumption 4.9 the proof of step (1) yields, for each Action, a witness of Definition 4.6. Whether rt is a root that the chain recorded, whether a nullifier is new, and whether a burn or an issuance is admitted by the issuance state are facts about the chain’s history, which no proof over one Action’s data attests; the verifier checks them against its state, in steps (4), (5) and (9). Issuance is checked entirely outside the proof, from public data (Remark 6.6). Table 6 maps each requirement to the conditions of the statement, the issuance and state rules, the checks outside the proof and the result that discharges it, the last as in Table 2.

8 The transaction carrier

This section states the status of every text that would carry OrchardZSA in a transaction: the transaction formats (§8.1), the digests and the message that the three kinds of signature sign (§8.2), and the fee terms (§8.3). No carrier is constructed here. The withdrawn design is described abstractly and classified as designed but unspecified; the absence of a live carrier is classified as an open problem (Definition 1.2).

8.1 Transaction formats

The carrier of a construction is the set of transaction fields that encode it together with the digests that commit to it (Definition 1.4).

Table 6. Checks against the requirements they serve. For each requirement of Definition 1.8: the conditions of the OrchardZSA Action statement (Definition 4.6), the rules of Construction 7.3 and Definition 6.12, the numbered checks of Construction 7.10 outside the proof, and the result of “Security” that discharges it.

Requirement	Conditions	Issuance and state rules	Checks	Result
Z1, per-Asset conservation	A1', A2', A4', C1	none	(6), (7)	Theorem 9.5, “No counterfeiting across Assets”
Z2, no counterfeiting across Assets	A1', A2', A3', A4', C1, C2, C3	(I4), the derived base; (T4) with Lemma 7.7; Definition 7.4	(3), (4), (7), (10)	Theorem 9.5, “No counterfeiting across Assets”
Z3, supply integrity	A4', value 0 of Split Inputs; A5', split nullifier	(T4), (I4) to (I7)	(5), (6), (9) to (12)	Theorem 9.7, “Supply integrity”; Proposition 7.8, “Finalised supply never grows”
Z4, issuance authority	none	(I1) to (I3); (I4), the base derived from issuer	(8); (5), through $\text{nf}_{0,0}$	Theorem 9.8, “Issuance authority”
Z5, hiding of the Asset type	the statement as a whole under Assumption 4.10; A4', A5'	none, issuance being public (Remark 6.6)	none beyond the published burn set	Theorem 9.11, “Privacy of the Asset type”

Deferral of the encodings. ZIP 226, “OrchardZSA Transaction Structure”, states that the transaction format that carries OrchardZSA “is described in ZIP 230”, and defers to it the transaction structure and the note plaintexts. ZIP 227, “Issue Note”, “Issuance Action” and “Issuance Bundle”, defers to ZIP 230 the encodings of Issue Notes, of Issuance Actions and of the Issuance Bundle. ZIP 230, now titled “Withdrawn Version 6 Transaction Format”, has status Withdrawn (Definition 1.1): it “has been obsoleted by ZIP 229, and will not be deployed”, and transaction version 6 “is now defined by ZIP 229” (ZIP 230, “Abstract”). The consensus rules of ZIP 227 nonetheless name fields that exist only in ZIP 230, the Action-Group count and the per-group expiry field of rule (T1) of Construction 7.3 (“State transition of a transaction”, §7.2), and require every Issue Note description to be “a valid field encoding as defined in ZIP 230” (ZIP 227, “Specification: Consensus Rule Changes”). A Draft consensus rule thus delegates to withdrawn text. ZIP 226, “Privacy Implications” and “Implications for Wallets”, still identifies OrchardZSA with version-6 transactions.

Version 6. Version 6 is the transaction format of ZIP 229, “Version 6 Transaction Format” (Draft), which the active NU6.3 deploys (ZIP 258, Draft). It carries the Ironwood component beside the Orchard component and no bundle of OrchardZSA or of issuance (*Ironwood Guide*, §“The transaction format”, Construction “Version-6 transaction”). By ZIP 229, “Non-requirements”, the format “need not support ZSAs”. The withdrawn format and the format of ZIP 229 share the version number 6 and are distinct and incompatible.

Version 7. ZIP 248, “Extensible Transaction Format” (Draft), defines transaction version 7, whose version group identifier is not yet assigned and whose “Deployment” section is empty. Its bundle registry allocates no bundle to ZSA. OrchardZSA, proposed as a provisional variant of the Orchard bundle type (ZIP 226), and ZSA Issuance, with no bundle type (ZIP 227), appear only under “Potential Future Bundle Types”, kinds that “have no allocated (bundleType, bundleVariant) pair”. ZIP 248 specifies no encoding and no digest for either (ZIP 248, “Terminology”, “Abstract”, “Potential Future Bundle Types” and “Deployment”).

Content and withdrawn design. The abstract content that the rules of this volume need is fixed earlier and does not depend on a carrier: the OrchardZSA bundle of Definition 5.2, with at most one Action Group, one burn set, the balancing value and the binding signature, and the Issuance Bundle of Definition 6.5. The withdrawn design of ZIP 230 encodes this content as follows; no part of it is a live layout.

- (a) The burn set is encoded inside the Action Group as a list of 40-byte entries, each the 32-byte encoding `asset_base` of Definition 2.14 followed by the amount as an unsigned 64-bit little-endian integer. ZIP 230 thus places one burn set in each Action Group, which under the limit of ZIP 227 of one Action Group per transaction is one burn set per transaction. Its field description of the amount, “non-zero, and less than `MAX_BURN_VALUE`”, is superseded by rule (B2) of Definition 5.1, $0 < v \leq \text{MAX_BURN_VALUE}$, which is ZIP 226’s (ZIP 230, “OrchardZSA Action Group Description” and “OrchardZSA Asset Burn Description”; ZIP 226, “Additional Consensus Rules for the `assetBurn` set”).
- (b) The flags byte of the Action Group assigns bit 2 to `enableZSA`, an assignment without live counterpart (“Action Groups”, §4.5).
- (c) Each Action carries the public fields of an Orchard Action description; the Action Group carries, in Action order, one spend-authorisation signature per Action, and the aggregate proof. The byte lengths of an Action depend on a plaintext encoding that no current ZIP defines (“Action Groups”, §4.5).

The pool into which a carrier would place OrchardZSA notes is the subject of Remark 1.7.

Remark 8.1 (Status of the carrier). Under Definition 1.2:

- (i) the transfer, burn and issuance constructions are specified, in ZIPs 226 and 227, both Draft;
- (ii) the field sets and digests of withdrawn ZIPs 230 and 246, read as a composition of those constructions into one transaction and abstracted from the version number, the version group and the positions of the flag bits, are designed but unspecified: a complete design exists in withdrawn text, and no live text fixes it. The exception is the authorising-data digest, which ZIP 246 itself marks as not updated for `sighashInfo` (ZIP 246, “v0 Digests”), and whose issuance child states its empty case for a transaction with no Orchard Actions (ZIP 246, “A.4: `issuance_auth_digest`”); no result of this volume uses that digest;
- (iii) which live transaction format carries OrchardZSA bundles and Issuance Bundles is an open problem. The only design assigns the version number 6 and bit 2 of the Orchard flags, both of which live text assigns otherwise: version 6 is the format of ZIP 229, in which that bit is reserved. ZIP 229 need not support ZSAs, and ZIP 248 allocates no bundle to them.

The texts that leave (iii) open and the obligation that closes it are enumerated in “Open problems” (§10.2) (ZIP 229, “Non-requirements”; ZIP 230 and ZIP 246, “Abstract”; ZIP 248, “Potential Future Bundle Types”).

8.2 Transaction digests and the signature message

The signed message. Every signature that OrchardZSA uses signs one message, SigHash: the issuance authorisation signature, by rule (I3) of Definition 6.12, the spend-authorisation signature of each Action, and the binding signature of the bundle (Definition 5.4; ZIP 226, “Value Balance Verification”; *Ironwood Guide*, Definition “Signature digest”). ZIP 227 defines SigHash as the digest of the protocol specification, §“SIGHASH Transaction Hashing”, under SIGHASH_ALL, “with the modifications described in ZIP 226”, and cites for them a ZIP 226 section “TxId Digest” that does not exist. ZIP 226, “Sighash modifications relative to ZIP 244”, and ZIP 227, “Modifications relative to ZIP 244”, defer those modifications to ZIP 246, “Digests for the Withdrawn Version 6 Transaction Format”, whose status is Withdrawn and whose digests “are now specified by ZIP 229” (ZIP 227, “Specification: Consensus Rule Changes”; ZIP 246, “Abstract”).

Withdrawn design of the transaction identifier. ZIP 246 extends the digest tree of ZIP 244 (*Ironwood Guide*, Definition “Transaction identifier”). Each node is the function $h = \text{BLAKE2b-256}$ under a personalisation of its own, applied to the concatenation of fixed-length digests and field encodings; a node that ZIP 246 does not redefine keeps its definition in ZIP 244. The root has six children:

$$\text{txid} := h(\text{ZcashTxHash}_- \parallel \beta, d_{\text{hdr}} \parallel d_{\text{tr}} \parallel d_{\text{Sap}} \parallel d_{\text{Orc}} \parallel d_{\text{Iss}} \parallel d_{\text{memo}}),$$

with β the encoding of the consensus branch identifier as in the *Ironwood Guide*. The header digest d_{hdr} , redefined with fields of ZIPs outside this volume, the transparent digest d_{tr} and the Sapling digest d_{Sap} carry no ZSA content. The other three children do.

- (a) The *Orchard branch* d_{Orc} hashes one digest over the Action Groups and the balancing value. For each Action Group, the digest over the Action Groups commits to every Action’s nullifier nf , note commitment cmx , ephemeral key epk and ciphertexts C_{enc} and C_{out} , value commitment cv^{net} and randomised validating key rk , in Action order, and to the flags byte, the anchor, the per-group expiry field and a *burn digest*. The burn digest hashes every burn pair ($\text{asset_base}, v$) in encoding order, and is the hash of the empty string when the burn set is empty. Without an Action Group the Orchard branch is the hash of the empty string; since the burn set is encoded inside the Action Group, a transaction then has no burn set. With at most one Action Group per transaction (ZIP 227), the digest over the Action Groups is one hash over the elements of that group.
- (b) The *issuance branch* d_{Iss} hashes the length-prefixed issuer and an *actions digest*, which commits, per Issuance Action and in order, to assetDescHash , to a *notes digest* over every Issue Note’s recipient, value, ρ and rseed , and to flagsIssuance . ZIP 246 states empty cases for the issuance digest, “no issuance components”, and for the notes digest, and none for the actions digest.
- (c) The *memo branch* d_{memo} hashes a nonce and a digest over memo chunks, and presupposes the memo bundles of ZIP 231.

The personalisation strings of the nodes below the root are not listed, since no proof or rule of this volume uses them (ZIP 246, “TxId Digest”, nodes T.4, T.4a, T.4a.vi, T.5, T.5a, T.5a.i and T.6; ZIP 244, “TxId Digest”).

Withdrawn design of the signature digest and the authorising-data digest. The signature digest has the same six children as the transaction identifier. Its Orchard, issuance and memo children are those of txid, and its transparent child is specific to the input, as in ZIP 244 (*Ironwood Guide*, Definition “Signature digest”). ZIP 246 produces a signature digest for each transparent input, each Sapling input, each Action and each Issuance Action; all of these except those of transparent inputs equal the SIGHASH_ALL digest SigHash, since no other child depends on the signer. The authorising-data digest (*Ironwood Guide*, Definition “Authorising-data digest; effecting and authorising data”) has transparent, Sapling, Orchard and issuance children. The Orchard child hashes, per Action Group, the proof and the spend-authorisation signatures, and then the binding signature; the issuance child hashes the encoding of issueAuthSig. Proofs and signatures enter this digest only. ZIP 246 marks this digest as not yet updated for the sighashInfo fields of its format (ZIP 246, “v0 Digests”). ZIP 246 states that the pair of the transaction identifier and the authorising-data digest commits to all data of the transaction (ZIP 246, “Signature Digest” and “Authorizing Data Commitment”, nodes A.3, A.3a and A.4; ZIP 244, “Signature Digest” and “Authorizing Data Commitment”).

Remark 8.2 (Commitments required of a digest). Under the withdrawn design, the digest SigHash commits

- (i) to the effecting data of the Issuance Bundle: issuer and, per Issuance Action and in order, assetDescHash, finalize and every field of every Issue Note, namely the recipient (d, pk_d) , v , ρ and rseed; the Asset Base is derived from issuer and assetDescHash (Remark 6.14);
- (ii) to the burn set;
- (iii) to the published data of the Actions of the Action Group, hence to $nf_{0,0}$;

and to no signature. The argument runs node by node. The signature digest’s Orchard child is the Orchard branch (a), and its issuance child is the issuance branch (b); their inputs include the data listed in (i) to (iii), the flag finalize as a bit of flagsIssuance. Signatures and proofs enter the authorising-data digest only. In the withdrawn design every node input is a concatenation of fixed-length digests and of fixed-length or length-prefixed field encodings, so that it parses uniquely, and the number of records of each list is determined by the length of the input. Let two transactions with equal SigHash differ in a listed datum. If their consensus branches differ, the root personalisations differ and equal outputs are already a collision in the sense of the *Ironwood Guide*’s Assumption “Collision resistance of BLAKE2b”; otherwise the differing datum yields two distinct inputs with equal outputs at some node, a collision of h. The withdrawn design thus meets Assumption 6.13 under the *Ironwood Guide*’s Assumption “Collision resistance of BLAKE2b”. The three commitments (i) to (iii), together with the exclusion of every signature, are the obligation on any future signature digest that carries OrchardZSA: Proposition 6.16 and Theorem 9.8 assume exactly them and nothing more (ZIP 246, “TxId Digest” and “Signature Digest”).

Current text. The version-6 digests of ZIP 229 have header, transparent, Sapling, Orchard and Ironwood children and no issuance, burn or memo-bundle data (no counterpart of the branch d_{memo} of ZIP 246); their Orchard child carries Orchard-pool Actions, which have no Asset Bases (*Ironwood Guide*, §“Transaction digests and signatures”; ZIP 229, “Transaction Identifiers, Auth

Digests, and Signature Digests” and “Summary of the resulting digest structure”). ZIP 248 lets each bundle type define its own versions of the signature digest, carried with each signature as `sighashInfo`, and defines none for a ZSA bundle (ZIP 248, “Sighash Versioning” and “Potential Future Bundle Types”). No current text specifies a digest that commits to OrchardZSA or issuance data.

Remark 8.3 (Status of the signature message). The rules by which the three kinds of signature sign SigHash are specified, in ZIPs 226 and 227, both Draft. The message SigHash itself is designed but unspecified: its only definition is withdrawn ZIP 246, a complete design. Its live definition, as part of a live carrier, is an open problem (Remark 8.1). Every result of this volume that uses the message is conditional on Assumption 6.13 (Definition 1.3) (ZIP 226, “Sighash modifications relative to ZIP 244”; ZIP 227, “Modifications relative to ZIP 244”; ZIP 246).

8.3 Fees

The conventional fee. The transaction fee is the value remaining in the transparent transaction value pool, and ZIP 317, “Fee calculation”, specifies the conventional fee

$$\begin{aligned} & \text{marginal_fee} \cdot \max(\text{grace_actions}, \text{logical_actions}) \text{ zatoshi,} \\ & \text{marginal_fee} = 5000, \quad \text{grace_actions} = 2, \end{aligned}$$

where *logical_actions* is a sum of contributions per component, fixed by the applicable revision of ZIP 317. The fee and the formula are those of the *Ironwood Guide*, §“The balancing value”, and are cited, not re-derived. Paying the conventional fee is not a consensus requirement: “It is not a consensus requirement that fees follow this formula; however, wallets SHOULD create transactions that pay this fee” (ZIP 317, “Fee calculation”).

Fees in ZEC. Fees are paid in ZEC only. The balancing value is an amount of ZEC and enters the binding validating key at the base V^{Orchard} only (Definition 5.4), and a Custom Asset has no transparent value pool (Remark 5.3), so no Custom Asset reaches the pool whose remainder is the fee. ZIP 227 records the alternative, a fee denominated in the Custom Asset, as not adopted, since for shielded transactions any design that lifts value from the transaction leaks information about it (ZIP 226, “Value Balance Verification”; ZIP 227, “Rationale for paying fees in ZEC”).

Missing ZSA terms. ZIP 227, “Changes to ZIP 317”, states that the conventional fee in ZEC is altered for the Issuance Actions of a transaction and for Custom Assets newly created in the global issuance state, and refers to ZIP 317, “Fee calculation”. No revision of ZIP 317 has such a term; Revision 0 is Active and Revisions 1 and 2 are Draft. The Change History of ZIP 317 records “Removed the ZSA issuance and asset-creation fee contributions (ZIP 227)”. ZIP 226, “Transaction Fees”, links a ZIP 227 section “OrchardZSA Fee Calculation” that does not exist. Both references dangle. The removed design had one contribution per Issue Note and one per newly created Custom Asset; their values are not quoted (ZIP 317, “Revisions”, “Fee calculation” and “Change History”).

Remark 8.4 (Status of the fee terms). The ZSA fee terms have one class, designed but unspecified (Definition 1.2): a complete design existed as text of ZIP 317 and was removed, and it conflicts with no live text. Live ZIP 227, “Changes to ZIP 317”, states that the conventional fee is altered,

and live ZIP 317, “Fee calculation”, has no such term. The conventional fee is not a consensus rule, so the protocol of this volume does not need the fee terms in the sense of Definition 1.2(iii), and they are not an open problem. That fees are paid in ZEC is specified (ZIP 317, “Change History”; ZIP 227, “Changes to ZIP 317”).

9 Security

This section composes the propositions stated at their objects into the five requirements of Definition 1.8, as Table 2 names them: Theorem 9.5 discharges Z1 and Z2; Theorem 9.7, with Proposition 7.8, discharges Z3; Theorem 9.8 discharges Z4; and Theorem 9.11 discharges Z5. The model and the assumptions are fixed in §9.1, and the four theorems are proved in §9.2 to §9.5.

The conventions are those of the *Ironwood Guide*, §“Security”, cited, not restated. Efficient adversaries and negligible functions are those of the *Crypto Guide*, §“Adversaries and the security parameter”, and computational indistinguishability is that of its Definition “Perfect, statistical, and computational indistinguishability” (§“Distribution ensembles and indistinguishability”). Every reduction simulates the honest parties and the random oracles, which are independent of one another. Every result holds except with negligible probability, and every result that uses a property of key generation is claimed for keys generated as the *Ironwood Guide*, §“Security”, claims them, with `use_ysk = false`.

9.1 Accepted transactions and assumptions

Definition 9.1 (Accepted OrchardZSA transactions and witnesses). The definition extends the *Ironwood Guide*’s Definition “Pools, accepted Actions and witnesses” to OrchardZSA.

- (a) *Pool*. The pool is the pool that carries OrchardZSA notes, which Remark 1.7 leaves open. It has one note commitment tree, one nullifier set and one issuance state `issued_assets` (Definition 7.1); every statement below uses these.
- (b) *Valid chain*. An adversary outputs a *valid chain*: a sequence of blocks each of whose transactions passes Construction 7.10 against the tree, the nullifier set and the issuance state that the transactions before it produce, the tree being extended in the order of Definition 7.4 and the issuance state by Construction 7.3. An Action, an Action Group (Definition 4.13), an OrchardZSA bundle (Definition 5.2) or an Issuance Bundle (Definition 6.5) of a transaction of a valid chain is *accepted*.
- (c) *Witnesses*. The *witness* of an accepted OrchardZSA Action is the auxiliary input that the extractor of Assumption 4.9 outputs for the proof of its Action Group, the assumption being applied to each Action Group as the *Ironwood Guide* applies its knowledge-soundness assumption to each bundle (the paragraph after its Definition “Pools, accepted Actions and witnesses”). It contains `AssetBase`, `is_native_asset`, `split_flag` and ψ^{nf} besides the Orchard witness, and satisfies Definition 4.6. The Action *consumes* the note

$$(g_d^{\text{old}}, pk_d^{\text{old}}, v^{\text{old}}, \text{AssetBase}, \rho^{\text{old}}, \psi^{\text{old}}, rcm^{\text{old}})$$

of its witness, with value $v' := 0$ if $\text{split_flag} = 1$ and $v' := v^{\text{old}}$ otherwise (the notation of $A4'$), and *creates* the note

$$(g_d^{\text{new}}, pk_d^{\text{new}}, v^{\text{new}}, \text{AssetBase}, \rho^{\text{new}}, \psi^{\text{new}}, \text{rcm}^{\text{new}}),$$

with ρ^{new} the published nullifier. Notes are given by expanded receivers as in item (d) of the cited definition. An accepted Action with $\text{split_flag} = 0$ is a *real spend* of its consumed note; one with $\text{split_flag} = 1$ is a Split Action (Definition 4.2). The shared pool also admits Orchard-protocol Actions (ZIP 226, “Privacy Implications”); the witness of such an Action is that of the *Ironwood Guide*’s Assumption “Knowledge soundness of the Action proof”, and every such Action is a real spend.

- (d) *Issue Notes*. An Issue Note of an accepted Issuance Bundle is given by its published fields and the Asset Base that rule (I4) of Construction 7.3 derives. Its note commitment is the one that the validator computes by rule (I6), which presupposes a fixed deterministic derivation of rcm (Remark 6.2, Definition 1.3).
- (e) *Committed notes and openings*. A *committed note* is a note whose extracted commitment is a leaf of the tree. An *opening* of a leaf is a note whose extracted commitment equals it.
- (f) *Adversary*. The adversary controls every party except the honest ones that it attacks. It may generate issuance keys, issue Assets under them with any descriptions, choose every field of its own transactions, copy any leaf as a Split Input, and obtain from each honest issuer the signed transactions that the issuer composes, in an order that it schedules.

(ZIP 226, “Specification”; ZIP 227, “Specification: Consensus Rule Changes”).

By the union bound over the polynomially many Action Groups of a valid chain, every witness satisfies Definition 4.6, except with negligible probability. Conditions $A1'$, $A2'$, $A3'$ and $A7$ then hold in their unweakened forms, without the $\perp$ case, except with negligible probability, by the *Ironwood Guide*’s Remark “ $\perp$ -weakened conditions” (§“The Action statement”) and, for the custom branch of $A1'$ and $A2'$, by Proposition 3.3(c), under the *Ironwood Guide*’s Assumptions “Discrete logarithms on Pallas” and “GroupHash as a random oracle”. The results below include this event in their negligible terms.

Remark 9.2 (Standing assumptions). The results of this section rest on the following assumptions, each stated in the text named.

1. The *Ironwood Guide*’s Assumptions “GroupHash as a random oracle” (§“GroupHash, domain separation, and nothing-up-my-sleeve generators”) and “Discrete logarithms on Pallas” (§“Sinsemilla: hash, commitment, and short forms”).
2. The *Ironwood Guide*’s Assumption “Collision resistance of BLAKE2b” (§“Transaction digests and signatures”), and Assumption 2.9 (“Asset Digests and Asset Bases”).
3. Assumption 6.8 (“Computation of ρ for Issue Notes”).
4. Assumption 2.3 (“Issuance keys and the issuer identifier”).

5. Assumptions 4.9 and 4.10 (“The OrchardZSA Action proof”), and the *Ironwood Guide*’s Assumption “Knowledge soundness of the Action proof” for Orchard-protocol Actions.
6. The *Ironwood Guide*’s Assumption “The RedPallas hash as a random oracle” (§“The Red-Pallas signature scheme”).
7. The assumptions of the *Ironwood Guide*’s Proposition “Confidentiality and key privacy of note encryption” (§“Encryption to the recipient”), and its Assumptions “Poseidon is a PRF” (§“The nullifier”), “Pseudorandomness of PRF expansion” (§“Pseudorandom functions and field reductions”) and “Joint hiding of the viewing keys” (§“Viewing keys”).
8. The hypothesis H-SIG, Assumption 6.13 (“Issuance authorisation”), in the form that Remark 8.2 states as the obligation on a digest.

Following Definition 1.3, three of them concern designed but unspecified objects: Assumptions 4.9 and 4.10, the circuit and its verifying key, and Assumption 6.13, the signed message. Each result below names which of these it is conditional on, and the results about Issue Notes add the fixed derivation of rcm . The *Ironwood Guide*’s results on its own statement, for example its Theorems “Balance” and “Privacy of an Action”, are not cited as proved for OrchardZSA; only their proof steps are reused, each cited at its use.

9.2 Asset Bases and balance of accepted bundles

Lemma 9.3 (Provenance of Asset Bases). *Under Assumption 4.9, the Ironwood Guide’s Assumptions “Knowledge soundness of the Action proof”, “GroupHash as a random oracle” and “Discrete logarithms on Pallas”, in every valid chain, except with negligible probability:*

- (i) *every leaf of the tree has at most one opening that an efficient algorithm finds;*
- (ii) *every leaf whose opening has $\text{AssetBase} \neq V^{\text{Orchard}}$ opens to a note whose Asset Base is*

$$\text{ZSAValueBase}(\text{AssetDigest}_{(\text{issuer}, \text{assetDescHash})})$$

(Construction 2.10) for the issuer of an accepted Issuance Bundle and the assetDescHash of one of its Issuance Actions, whose Issue Notes precede the leaf, or include it, in the order of Definition 7.4;

- (iii) *every leaf appended by an Orchard-protocol Action, before or after OrchardZSA, opens only with $\text{AssetBase} = V^{\text{Orchard}}$, and the empty leaf opens to nothing.*

Status (Definition 1.3): proved here for the specified statement and rules; conditional on Assumption 4.9, which concerns the designed but unspecified circuit, and, for the leaves of Issue Notes, on a fixed derivation of rcm , any deterministic one.

Proof. Every step is carried out by the efficient algorithm that runs the adversary with the extractors and inspects the chain; the chain has polynomially many leaves, and the union bound collects the negligible events.

Uniqueness of openings. By Proposition 3.3(b), no efficient algorithm outputs two distinct OrchardZSA notes with equal extracted commitments other than $\perp$, within one branch of Definition 3.2 or across the two; the sign ambiguity of extraction is handled in its proof by the *Ironwood Guide*’s Lemma “Coordinate extraction identifies opposite points”. An Orchard note is the native-branch OrchardZSA note with the same components, by the case split of that definition. Hence (i), and an opening found in the native branch excludes one in the custom branch.

Induction. The claim is proved for the leaves in the order of Definition 7.4, the hypothesis being (ii) and (iii) for all earlier leaves. A position beyond the appended leaves holds $\text{Uncommitted}^{\text{Orchard}} = 2$, which by the *Ironwood Guide*’s Lemma “Uncommitted leaves” is the extracted commitment of no note, since in both branches cmx is $\perp$ or the extraction of a point. Every appended leaf is of one of three kinds.

Orchard-protocol Action. The witness of the Action that appended the leaf creates a note whose commitment $\text{NoteCommit}^{\text{Orchard}}$, by the unweakened condition A2, extracts to the leaf. That commitment is the native branch of $\text{NoteCommit}^{\text{OrchardZSA}}$ (Definition 3.2), so the leaf has an opening with $\text{AssetBase} = V^{\text{Orchard}}$, and by (i) no other. This holds for leaves appended before OrchardZSA and after it.

OrchardZSA Action. Let the witness have $\text{AssetBase} = B$. If $B = V^{\text{Orchard}}$ there is nothing to show. Otherwise $\text{is_native_asset} = 0$ by C1, so the first disjunct of A3’ fails, and the unweakened A3’ gives a valid path from $\text{Extract}_p(\text{cm}^{\text{old}})$ to the anchor rt of the Action Group. By check (4) of Construction 7.10, the anchor is the root of the tree in the final treestate of an earlier block. By the *Ironwood Guide*’s Proposition “Membership soundness”, and since $\text{cm}^{\text{old}} \neq \perp$ by the unweakened A1’, the value $\text{Extract}_p(\text{cm}^{\text{old}})$ is a leaf appended by an earlier transaction. By A1’, the consumed note, of Asset Base B , opens that leaf, and by (i) it is its opening. By the hypothesis, B is the Asset Base derived from the issuer and an assetDescHash of an earlier accepted Issuance Action whose Issue Notes precede that leaf. By A2’, the created note carries the same B and opens the new leaf, and by (i) it is its opening. The case covers Split Actions: A3’ demands the path for every consumed note with $\text{is_native_asset} = 0$, whatever its value and split_flag , and C3 forbids $\text{split_flag} = 1$ with $\text{is_native_asset} = 1$.

Issue Note. Rule (I4) derives the Asset Base from the bundle’s issuer and the action’s assetDescHash by Construction 2.10; rule (I6) makes the validator compute the commitment from that base and the published fields (Remark 6.14), with the fixed derivation of rcm ; its commitment is not $\perp$ except with negligible probability, by Proposition 3.3(c), its Asset Base being a custom one (Proposition 2.13). The Issue Note therefore opens the leaf, and by (i) it is its opening; its Asset Base is the derived one, which differs from V^{Orchard} except with negligible probability (Proposition 2.13), and its own Issuance Action includes the leaf. $\square$

ZIP 226 states the conclusion in rationale: “This ensures that the value base points of all output notes of a transfer are actual outputs of a GroupHash, as they originate in the Issuance protocol which is publicly verified” (“Rationale for Split Notes”), and the output notes “receive an Asset Base that exists on the global state” (“Rationale for Asset Identifiers”). In Lemma 9.3, existence on the global state is realised by membership in the tree; the proof never consults issued_assets (ZIP 227, “Addition to the Note Commitment Tree”). Figure 3 draws the induction.

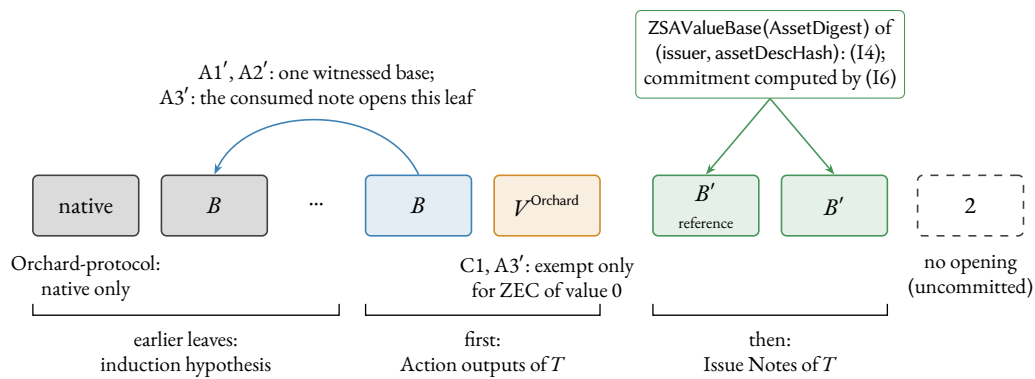


Figure 3. Provenance of Asset Bases in the tree, the induction of Lemma 9.3. One transaction T appends its leaves in the order of Definition 7.4: the outputs of its Actions, then its Issue Notes. Earlier leaves (grey) satisfy the induction hypothesis; a leaf appended by an Orchard-protocol Action opens only with V^{Orchard} . An Action output of a Custom Asset B (blue) inherits B from an earlier leaf through $A1'$, $A2'$ and $A3'$; $C1$ confines the exemption from membership to ZEC (amber). The Asset Base B' of an Issue Note (green) is derived by the validator from issuer and assetDescHash by (I4), and its commitment computed by (I6). The empty leaf 2 opens to no note.

Corollary 9.4 (Asset Bases of accepted Actions). *Under the assumptions of Lemma 9.3, in every valid chain, except with negligible probability, every Asset Base witnessed by an accepted OrchardZSA Action is V^{Orchard} or $\text{ZSAValueBase}(\text{AssetDigest})$ of the Asset Identifier of an Issuance Action of an accepted Issuance Bundle of an earlier transaction. No transactor chooses an Asset Base: every Custom Asset Base in an accepted Action is one that a validator derived. Status (Definition 1.3): proved here; conditional on Assumption 4.9 and, through Lemma 9.3, on a fixed derivation of rcm for Issue Notes.*

Proof. For a witness with $\text{AssetBase} = \mathcal{V}^{\text{Orchard}}$ there is nothing to show. The exemption of $A3'$ applies only to such consumed notes, since C1 makes $\text{is_native_asset} = 1$ exactly when $\text{AssetBase} = \mathcal{V}^{\text{Orchard}}$. Otherwise, as in the proof of Lemma 9.3, $A3'$ and the *Ironwood Guide*'s Proposition "Membership soundness" make the consumed note an opening of a leaf of the treestate at the anchor, appended by an earlier transaction; $A1'$ fixes its Asset Base as the witnessed one; and Lemma 9.3(ii) applies to that leaf. The only place where an Asset Base not derived by a validator could enter an Action is a consumed note without a membership proof, which C1 and $A3'$ confine to $\mathcal{V}^{\text{Orchard}}$; Proposition 4.1 shows that the confinement is necessary (ZIP 226, "Circuit Statement", "Asset Base Equality" and the modified Merkle path condition). $\square$

Theorem 9.5 (No counterfeiting across Assets). *Assume Assumption 4.9, Assumption 2.9, and the Ironwood Guide’s Assumptions “Knowledge soundness of the Action proof”, “GroupHash as a random oracle”, “Discrete logarithms on Pallas” and “The RedPallas hash as a random oracle”; the collision resistance of the Sinsemilla instances used by that volume’s Proposition “Membership soundness” follows from “GroupHash as a random oracle” and “Discrete logarithms on Pallas”. In every valid chain, except with negligible probability, every accepted OrchardZSA bundle, with Actions i , burn set assetBurn and balancing value v^{balance} , satisfies the following, as equalities of*

integers, the values being read from the witnesses:

(i) for every point $B \neq V^{\text{Orchard}}$,

$$\sum_{i: \text{AssetBase}_i = B} (v'_i - v_i^{\text{new}}) = v_B,$$

where v_B is the amount of the pair $(B, v_B) \in \text{assetBurn}$, or 0 if there is none;

(ii) $\sum_{i: \text{AssetBase}_i = V^{\text{Orchard}}} (v'_i - v_i^{\text{new}}) = v^{\text{balance}}$;

(iii) distinct Assets have distinct Asset Bases, and no Custom Asset has the Asset Base V^{Orchard} , so that (i) and (ii) hold per Asset.

Hence no accepted bundle creates value of one Asset from value of another Asset or from nothing (requirements Z1 and Z2). The statement concerns the accepted bundles of a valid chain, not single bundles: the Asset Bases of burn pairs are constrained only by rule (T4) of Construction 7.3 against the issuance state. Status (Definition 1.3): proved here for the specified constructions; conditional on Assumption 4.9, which concerns the designed but unspecified circuit, and on a fixed derivation of rcm for Issue Notes, and not on Assumption 6.13. ZIP 226 argues the property in rationale only.

Proof. Witnessed Asset Bases. By Corollary 9.4, each is V^{Orchard} or

$$\text{GroupHash}(\text{z.cash:OrchardZSA}, D)$$

for the Asset Digest D of an accepted Issuance Action.

Asset Bases of burn pairs. By rule (T4) every burn pair of an accepted transaction has an Asset Base in the domain of its input state, and by Lemma 7.7 every such Asset Base is $\text{GroupHash}(\text{z.cash:OrchardZSA}, D)$ for the Asset Digest D of an accepted Issuance Action. Rule (B1) alone does not give this.

Premise and balance. Every input $(\text{z.cash:OrchardZSA}, D)$ differs from the inputs of V^{Orchard} and R^{Orchard} (Lemma 2.8). Each such input is computable from the chain, by searching its Issuance Actions for one whose derived Asset Base is the point. So the algorithm that runs the adversary with the extractors, and outputs the accepted bundles together with these inputs whenever it finds them, outputs bundles whose set $\mathfrak{B}$ meets the premise on the bases of §5.3; by the two preceding steps it fails to find an input only with negligible probability. Parts (a) and (b) of Proposition 5.6 then give (i) and (ii), the congruences modulo p_{Vesta} being lifted to integers by Lemma 5.5, whose hypotheses hold for every bundle of a valid block. A Split Action contributes $v' = 0$ by A4', so the value of a copied note never enters these sums. No other term of bvk carries a Custom Asset (Remark 5.3).

Part (iii). Claims 2 and 3 of Proposition 2.13: distinct Asset Identifiers have distinct Asset Bases, and no Asset Base of a Custom Asset equals V^{Orchard} , except with negligible probability, under Assumption 2.9 and the *Ironwood Guide's* Assumption “GroupHash as a random oracle”. $\square$

Each premise of the proof is necessary. For witnessed Asset Bases, Proposition 4.1 creates ZEC from a witnessed base $[-1] V^{\text{Orchard}}$ when the exemption of A3' is not confined to ZEC. For

burn pairs, the example after Proposition 5.6 creates $2v$ units of an issued Asset B with the pair $([-2] B, v)$, which rules (B1) to (B3) admit and rule (T4) rejects. ZIP 226, “Rationale for Split Notes”, “Value Balance Verification” and “Additional Consensus Rules for the assetBurn set”, gives the argument informally; the proof here does not change the class of that rationale, and ZIP 227, “Specification: Consensus Rule Changes”, supplies rule (T4).

9.3 Circulating supply

Lemma 9.6 (One consumption per OrchardZSA note). *Under Assumption 4.9, Assumption 6.8, and the Ironwood Guide’s Assumptions “Knowledge soundness of the Action proof”, “GroupHash as a random oracle” and “Discrete logarithms on Pallas”, in every valid chain, except with negligible probability:*

- (a) *the openings of the leaves of the tree have pairwise distinct elements ρ , and are therefore pairwise distinct notes;*
- (b) *every committed note is consumed by at most one accepted real spend;*
- (c) *a Split Action contributes $v' = 0$ and publishes a nullifier that is not the nullifier of its copied note.*

Status (Definition 1.3): proved here; conditional on Assumption 4.9, which concerns the designed but unspecified circuit, and, for the leaves of Issue Notes, on a fixed derivation of rcm , any deterministic one.

Proof. By Lemma 9.3(i), the opening of a leaf appended by an Action is the note that the Action’s witness creates, and the opening of a leaf appended by issuance is the Issue Note.

(a) Three cases cover every pair of leaves.

Two Action outputs. The note created by an accepted Action, of either protocol, has ρ equal to that Action’s published nullifier (A2’ with $\rho^{\text{new}} := \text{nf}^{\text{old}}$, or A2), and the published nullifiers, real and split, are pairwise distinct by the nullifier-set rule, check (5) of Construction 7.10.

Two Issue Notes. Their elements ρ are pairwise distinct by Proposition 6.9, under Assumption 6.8.

An Action output and an Issue Note. It suffices that no accepted Action publishes a nullifier equal to the ρ of an Issue Note of the chain, in whichever order the two occur; Proposition 6.9 covers only the Action outputs fixed before the issued value. Let an efficient algorithm $\mathcal{A}'$, running the adversary with the extractors, find such a pair with probability ϵ . The published nullifier is $\text{Extract}_p(Q)$, where by A5’ (or A5)

$$Q = [s] K^{\text{Orchard}} + \text{cm}^{\text{old}} \quad \text{or} \quad Q = [s] K^{\text{Orchard}} + \text{cm}^{\text{old}} + L^{\text{Orchard}},$$

the second for a split nullifier, with s computed from the witness. By the unweakened A1’ (or A1), $\text{cm}^{\text{old}} \neq \perp$ is, by the unfolding of the *Ironwood Guide*’s Construction “Sinsemilla hash on Pallas” used in the proof of Proposition 4.5(i), an integer combination of values of GroupHash with coefficients computable from the witness; so is Q . A reduction to the *Ironwood Guide*’s Assumption “Discrete logarithms on Pallas” receives a challenge Y to a base G , programs every value

of GroupHash as $[a]G$ for a fresh uniform scalar a , as in the proof of that volume’s Lemma “Relations among fixed generators”, chooses uniformly one of the at most q queries to the oracle of personalisation `Zcash_ExpandSeed`, and answers it with a uniform 64-byte string u conditioned on $\text{ToBase}(u) = \text{Extract}_p(Y + [b]G)$, for a uniform scalar b . If $\mathcal{A}'$ succeeds at that query, then $\text{Extract}_p(Q) = \text{Extract}_p(Y + [b]G)$, so $Q = \pm(Y + [b]G)$ by the *Ironwood Guide*’s Lemma “Coordinate extraction identifies opposite points”; with $Q = [c]G$ for the computed c , the reduction outputs $\pm c - b$, the discrete logarithm of Y . The programmed answer differs from a uniform one in that its reduction is the x -coordinate of a point, up to the statistical distance of `ToBase` from uniform, below 2^{-257} (Proposition 4.5); success requires the issued ρ to be such a coordinate in any case, and, given that it is one, the two distributions of ρ are within negligible statistical distance. The reduction therefore succeeds with probability at least ϵ/q less a negligible term, and ϵ is negligible.

Notes with distinct ρ are distinct.

(b) The steps of the proof of the *Ironwood Guide*’s Lemma “One nullifier per note” apply, with $A1'$ in place of $A1$ and $A5'$ with `split_flag` = 0, which sets $\psi^{\text{nf}} = \psi^{\text{old}}$ and publishes $\text{DeriveNullifier}_{\text{nk}}(\rho^{\text{old}}, \psi^{\text{old}}, \text{cm}^{\text{old}})$, in place of $A5$; $A7$ is unchanged. Two real spends, of either protocol, whose consumed notes have the same extracted commitment consume the same note, by Proposition 3.3(b), with the same nk , by the *Ironwood Guide*’s Proposition “Binding of ivk to (ak, nk) ”, and publish the same nullifier. The nullifier-set rule rejects a repeated nullifier within a transaction and across the chain.

(c) The value $v' = 0$ is the notation of $A4'$ for `split_flag` = 1. The nullifier is the split nullifier of $A5'$, and by Proposition 4.5(i), which holds for every ψ^{nf} , including one that the adversary chooses, it is not the nullifier of the copied note, except with negligible probability. A split nullifier that equals the nullifier of some other note only prevents the later spend of that note; it creates no value, and the note is not thereby consumed in the sense of Definition 9.1(c). (ZIP 226, “Split Notes”; ZIP 227, “Computation of ρ ”; protocol specification, §“Nullifier Sets”.) $\square$

Theorem 9.7 (Supply integrity). *Under the assumptions of Theorem 9.5 and Lemma 9.6, for every valid chain, every point $B \neq V^{\text{Orchard}}$ and every state of the chain after a transaction, except with negligible probability,*

$$\sum_n v(n) = \text{issued_assets}(B).\text{balance},$$

the sum being over the committed notes n with Asset Base B that no accepted real spend has consumed, and $v(n)$ being the value of n . By Proposition 7.9, the right-hand side is the amount of B issued less the amount burnt. Consequently the amount of B in circulation is never negative and is at most $\text{MAX_ISSUE} = 2^{64} - 1$, and it is constant or falling once B is finalised (Proposition 7.8). The invariant holds per branch, since the issuance state is a function of the branch (Remark 7.2). Status (Definition 1.3): proved here for the specified constructions; conditional on Assumption 4.9, which concerns the designed but unspecified circuit, and on a fixed derivation of `rcm` for Issue Notes, any deterministic one. ZIP 227 argues the property in rationale only.

Proof. Induction over the transactions of the chain. Write U_B for the left-hand side.

Base. Before the first transaction of the OrchardZSA activation block the issuance state is the empty map (Definition 7.1), whose balances read 0, and every leaf was appended by an Orchard-protocol Action, so by Lemma 9.3(iii) no committed note has a Custom Asset Base: $U_B = 0$.

Step. Let T be a transaction of the chain, $B \neq V^{\text{Orchard}}$, and let the claim hold before T . The leaves that T appends open uniquely (Lemma 9.3(i)) to notes distinct from the openings of all other leaves (Lemma 9.6(a)), and each is unconsumed when appended.

Orchard-protocol Actions of T . They create native notes only (Lemma 9.3(iii)). A note that such an Action consumes is a native note; by Lemma 9.3(i) it is not the opening of a leaf of Asset Base B . They leave U_B unchanged.

The Action Group of T and its burn set. Let R_B be its real spends witnessing B , and A_B all its Actions witnessing B . A note of Asset Base B consumed by an Action of T is consumed by an Action witnessing B , by A1'. The consumed notes of R_B are committed, by A3' with C1 and the *Ironwood Guide's* Proposition "Membership soundness", as in the proof of Lemma 9.3; and they were unconsumed before T and are pairwise distinct, by Lemma 9.6(b). Split Actions are not real spends and remove no note from U_B ; each contributes $v' = 0$ (Definition 9.1(c), Lemma 9.6(c)). Every Action of A_B creates one note of Asset Base B whose leaf T appends (Definition 7.4). Hence U_B changes by

$$\sum_{i \in A_B} v_i^{\text{new}} - \sum_{i \in R_B} v_i^{\text{old}} = - \sum_{i \in A_B} (v'_i - v_i^{\text{new}}) = -v_B,$$

the first equality because $v'_i = v_i^{\text{old}}$ on R_B and $v'_i = 0$ on $A_B \setminus R_B$, and the second by Theorem 9.5(i), whose premise on burn-pair bases rests on (T4) and Lemma 7.7. Rule (T4) lowers the balance of B by the same v_B . No transparent term carries B (Remark 5.3).

The Issuance Bundle of T . Each Issue Note of Asset Base B , that is of an Issuance Action whose derived base is B by (I4), enters the tree after the Action outputs (Definition 7.4) with its public value and unconsumed, and no Action of T consumes it: a consumed note of T opens a leaf of an earlier block, and by Lemma 9.6(a) that opening is not the Issue Note. Rule (I6) raises the balance of B by exactly that value; a reference note adds 0 to both sides. Issuance Actions of other Asset Bases write other entries.

Both sides therefore change equally, and the claim holds after T . The bounds follow from $U_B \geq 0$ and Proposition 7.9, and the behaviour after finalisation from Proposition 7.8. $\square$

The theorem, not the issuance state, excludes counterfeit value. ZIP 227, "Rationale for Global Issuance State", keeps the state along the lines of ZIP 209 ("Specification"), and ZIP 227, "Global Issuance State" and "Management of the Global Issuance State", maintain it. The state detects counterfeit value directly only when a burn exceeds the recorded balance, by rule (T4). A bundle that created unbacked value of B and did not burn it would pass every check on the map; Theorem 9.5 excludes such a bundle, and Theorem 9.7 inherits the exclusion.

9.4 Authority over issuance

Theorem 9.8 (Issuance authority). *Let an honest issuer generate isk as in Definition 2.1 and Assumption 2.3 and publish issuer = ik_encoding (Definition 2.4), and let the adversary of Definition 9.1 obtain the transactions that the issuer signs. An Asset of the honest issuer is an Asset*

whose Asset Identifier has that issuer. Under Assumption 2.3, Assumption 6.13, Assumption 2.9 and the Ironwood Guide’s Assumptions “Collision resistance of BLAKE2b” and “GroupHash as a random oracle”, in every valid chain that the adversary outputs, except with negligible probability:

- (i) every accepted Issuance Action whose Asset Base is that of an Asset of the honest issuer, and which raises its balance by rule (I6), sets its finalisation bit by rule (I7) or creates its entry with the reference note by rule (I5), belongs to a transaction whose Issuance Bundle has the effecting data, and whose transaction has the burn set and the nullifier $\text{nf}_{0,0}$, of a transaction that the honest issuer signed;
- (ii) the Issuance Bundle of each signed transaction is accepted at most once.

Hence only the holder of isk issues or finalises the issuer’s Assets (requirement Z4). Status (Definition 1.3): proved here for the specified rules; conditional on Assumption 6.13, which concerns the designed but unspecified signed message.

Proof. Reduction. Let an accepted Issuance Action change the entry at an Asset Base B of an Asset of the honest issuer, with Asset Identifier (issuer_b, b) . Rule (I4) derives B from the bundle’s issuer and the action’s assetDescHash . If $(\text{issuer}, \text{assetDescHash}) \neq (\text{issuer}_b, b)$, two distinct Asset Identifiers have the Asset Base B , which claim 2 of Proposition 2.13 excludes, under Assumption 2.9 and the Ironwood Guide’s Assumption “GroupHash as a random oracle”, except with negligible probability. So the bundle’s issuer is the honest ik_encoding . Rules (I2) and (I3) then require a signature valid under the honest ik on the transaction’s SigHash .

If the honest issuer made no signature on that digest, the reduction, which answers the honest issuer’s signing through the signing oracle of the EUF-CMA experiment, outputs the digest and the signature of the chain, a forgery against Assumption 2.3. If it made one, on a transaction T_b , then the two transactions have equal SigHash , and Assumption 6.13, which for the withdrawn digest rests on the Ironwood Guide’s Assumption “Collision resistance of BLAKE2b” (Remark 8.2), makes their Issuance Bundles’ effecting data equal (issuer and, per Issuance Action, assetDescHash , finalize and every field of every Issue Note), and their burn sets and nullifiers $\text{nf}_{0,0}$ equal. Every change of the entry, the finalisation bit included, is therefore one that the issuer authorised; this is (i).

Part (ii) is Proposition 6.16, by rule (I1) and the nullifier set on $\text{nf}_{0,0}$. Burns of the issuer’s Assets by note holders are outside the claim: rule (T4) only lowers the balance (ZIP 227, “Specification: Consensus Rule Changes” and “Issuance Authorization Signature Scheme”). $\square$

Remark 9.9 (Issuance key compromise). No rotation of issuance keys exists, and issuer is the encoding ik_encoding of the key; ZIP 227 notes that this equality “might not hold in future when key rotation is specified for issuance key pairs” (“Issuer Identifier”). The Asset Identifier of an Asset, and hence its Asset Base, stays bound to the key under which it was issued. On compromise of isk , ZIP 227 recommends, without a requirement keyword, setting $\text{finalize} = 1$ for every Asset issued with that key, changing to a new issuance authorising key, hence a new issuer, and issuing new Assets, each with a new Asset Identifier (“Issuance Key Compromise”). By rule (I4), finalisation bounds only the future issuance of the finalised Asset. It does not undo issuance by the attacker

before the finalising transaction is accepted. An attacker that holds isk can issue up to MAX_ISSUE less the recorded balance of each unfinalised Asset, can finalise first, and can issue Assets under new descriptions, which no finalisation prevents; by consensus these are indistinguishable from the issuer's own, and ZIP 227 treats their presentation only as a matter of display ("Security and Privacy Considerations", "Displaying Asset Identifier information to users"). Theorem 9.8 assumes the secrecy of isk and says nothing after its compromise. Key rotation has no design in any text. The specified protocol needs it for no result of this volume, so its absence is a stated limitation, not an open problem in the sense of Definition 1.2 (ZIP 227, "Issuance Keys").

9.5 Hiding of the Asset type

The *Ironwood Guide*'s Theorem "Privacy of an Action" is confined to Ironwood-pool bundles by precondition (P2) of its Definition "Public leakage and privacy preconditions". It is not cited as proved here; the definition below has the form of that definition.

Definition 9.10 (Public leakage of an OrchardZSA bundle). The *public leakage* of an OrchardZSA bundle is its number of Actions, its anchor, its flags `enableSpend`s, `enableOutputs` and `enableZSA`, its balancing value, and its burn set, which publishes each burnt Asset Base and amount. The rest of the transaction is public as well, including any Issuance Bundle with every field of its Issue Notes (Remark 6.6) and the carrier that holds the bundle. The *public fields* are, per Action, cv^{net} , nf , rk , cmx , epk , C_{enc} , C_{out} and the spend-authorisation signature, and, per bundle, the proof and the binding signature. The *private inputs* are, per Action, its witness (Definition 4.6, with `AssetBase`, `is_native_asset`, `split_flag` and ψ^{nf}), the plaintext of its created note (Definition 3.7), the outgoing viewing key under which its C_{out} is formed, or $\perp$, and whether its consumed side is a real note, a Split Input or a dummy. The *privacy preconditions* are:

- (Q1) every key involved is generated as in (P1) of the cited definition, except the all-zero spending key of a reference note copied as a Split Input, which is public (Definition 6.10);
- (Q2) honest construction: each created note has a fresh uniform `rseed`, and its `rcm` is derived from `rseed` by the designed `Derive_rcm` under an assigned lead byte (Remark 3.6); `rcv` is uniform; α is drawn by `GenRandom`; each Split Input has a fresh uniform `rseed_nf` (Construction 4.4); dummy consumed notes occur only for ZEC and are formed as in the protocol specification, §"Dummy Notes (Orchard)"; each Split Input copies a leaf with the Action's Asset Base whose recipient key is a key of (Q1) held by the sender, or the reference note; consumed notes are padded per Asset by the padding rule after Definition 4.2; each real consumed note is a note of a key of (Q1) that the sender accepted by the *Ironwood Guide*'s §"Trial decryption and note acceptance"; and the plaintext encoding has a length independent of the Asset;
- (Q3) authentication paths are computed from public data (the *Ironwood Guide*'s Lemma "Authentication paths from public data");
- (Q4) the adversary holds no viewing key of a key of (Q1).

Within these preconditions the adversary may choose the consumed notes, the recipients, the Assets and the values, and may know the contents of the consumed notes (ZIP 226, “Burn Mechanism”, “Split Notes” and “Backward Compatibility”; protocol specification, §“Dummy Notes (Orchard)”).

Theorem 9.11 (Privacy of the Asset type). *Assume (Q1) to (Q4) of Definition 9.10, and let two sequences of private inputs of one OrchardZSA bundle have equal public leakage, both satisfy Definition 4.6 with the common anchor and flags, and each balance per Asset against the common burn set and balancing value. Assume Assumption 4.10 and the Ironwood Guide’s Assumptions “Pseudorandomness of PRF expansion”, “Joint hiding of the viewing keys”, “Poseidon is a PRF”, “DDH on Pallas”, “GroupHash as a random oracle”, “The note-encryption hashes as random oracles”, “Confidentiality and integrity of ChaCha20-Poly1305”, “The RedPallas hash as a random oracle” and “Discrete logarithms on Pallas”. Then the public fields of the two bundles are computationally indistinguishable.*

Precisely, every efficient adversary has negligible advantage in the experiment of the Ironwood Guide’s Theorem “Privacy of an Action”, with the following changes: the leaves that the adversary outputs are those of the tree of Definition 9.1(a); it outputs in addition the common burn set and the flag enableZSA; and a specification gives, per Action, its Asset Base, a consumed side that is a dummy (for V^{Orchard} only), a note of a key of (Q1) whose extracted commitment is a leaf, or a Split Input as in (Q2), and a created side as in that experiment. Each specification balances per Asset, the consumed notes of one key have pairwise distinct ρ , and the construction of (Q2) yields witnesses of Definition 4.6.

In particular a bundle reveals, beyond its leakage, neither which Assets its Actions move, nor how many distinct Assets, nor their values, nor which consumed sides are real, split or dummies; Remark 9.12 states what the leakage itself reveals (requirement Z5). Status (Definition 1.3): proved here; conditional on Assumption 4.10, which concerns the designed but unspecified circuit, on the unassigned lead byte for the uniformity of rcm, and on the open plaintext encoding. ZIP 226 argues the property in rationale only (“Overview”, “Privacy Implications”).

Proof. For each of the two specifications, the hybrids H_0 to H_{10} below change the probability that the adversary outputs 1 by a negligible amount (*Crypto Guide*, §“The hybrid argument”, Lemma “Hybrid lemma”; each hybrid is reached in polynomially many steps, one key or one Action at a time), and the two hybrids H_{10} are identically distributed. Every reduction runs the experiment with the adversary and samples every value that its hop does not concern. The keys involved are those of (Q1), the keys of the dummy consumed notes and the keys of dummy created notes.

Hybrid H_0 is the real bundle.

Hybrid H_1 (*proof*). The aggregate proof is the output of the simulator of Assumption 4.10 on the primary inputs, which are public, enableZSA included; the witnesses satisfy Definition 4.6 by hypothesis. The cost is the statistical distance of that assumption. From H_1 on, a witness enters the game only through the public fields computed from it.

Hybrid H_2 (*keys*). Each key of (Q1) and each dummy key is replaced as in game G_2 of the proof of the *Ironwood Guide*’s Theorem “Privacy of an Action” (its Lemmas “Rejection in key generation” and “Independence of domain-separated expansions”, and Assumption “Joint hiding of

the viewing keys”), so that ask , nk , ivk , dk and ovk of each key are independent and dk and ovk uniform. The public all-zero key is left unchanged.

Hybrid H_3 (*randomisers*). Each randomiser α_i is replaced by a uniform scalar, as in game G_4 of that proof (*Ironwood Guide*, Lemma “Distance of GenRandom from uniform”, and Assumption “The RedPallas hash as a random oracle”). Then $\text{rsk}_i = \text{ask} + \alpha_i$ is uniform and independent of ask , the public ask of the all-zero key included, and each rk_i is a uniform point independent of ak .

Hybrid H_4 (*signatures*). Every spend-authorisation signature and the binding signature are replaced by simulated signatures under rk_i and bvk , programming the RedPallas hash, by the *Crypto Guide*’s Lemma “Signature simulation” (§“Security in the random oracle model and the forking lemma”). Its simulator applies to RedPallas, a Schnorr signature whose challenge is the RedPallas hash of the nonce commitment, the validating key and the message (*Ironwood Guide*, Construction “RedPallas”); the costs are the abort probability of the lemma and the statistical distance of the reduced hash output from a uniform scalar. No signing key enters the game from H_4 on: neither rsk_i nor bsk .

Hybrid H_5 (*value commitments*). After H_4 , each trapdoor rcv_i enters the game only through cv_i^{net} . Each cv_i^{net} is replaced by an independent uniform point, by perfect hiding for every Asset Base (Lemma 3.9(i)); the key bvk is computed from these points, the public burn set and the balancing value by Definition 5.4. The hop is perfect.

Hybrid H_6 (*split nullifiers*). Each split nullifier is replaced by the x -coordinate of a uniform point, by Proposition 4.5(ii), at cost $\epsilon_{\text{expand}} + \epsilon_{\text{ToBase}} + \delta$ each. That proposition holds against an adversary that knows the copied note and nk , so the reduction samples every key itself; it holds in particular for the public key of a reference note, and for a copied note that the same bundle spends.

Hybrid H_7 (*nullifiers of consumed notes*). For each key that consumes a real or dummy note, nk enters the game after H_6 only through the nullifiers of those notes, $\text{Extract}_P([\text{PRF}_{\text{nk}}^{\text{nfOrchard}}(\rho) + \psi] K^{\text{Orchard}} + \text{cm}^{\text{old}})$, where cm^{old} depends on the Asset Base. As in hybrids H_4 and H_5 of the proof of the *Ironwood Guide*’s Proposition “Nullifier unlinkability” and in game G_5 , the PRF is replaced by a random function (Assumption “Poseidon is a PRF”) and each multiplier of K^{Orchard} by a uniform scalar, so each nullifier is the x -coordinate of a uniform point whatever cm^{old} is. The real consumed notes of one key have pairwise distinct ρ by the definition of the experiment, which in a valid chain holds by Lemma 9.6(a), and a dummy consumed note has a key of its own.

Hybrid H_8 (*outgoing ciphertexts*). Each C_{out} is replaced by the encryption of a fixed string under an independent uniform key, as in game G_3 (*Ironwood Guide*, Proposition “Confidentiality of the outgoing ciphertext”).

Hybrid H_9 (*ephemeral keys and note ciphertexts*). For each created note, epk is replaced by a uniform non-identity point and C_{enc} by the encryption of a fixed plaintext of the common length, keeping every note commitment, by steps (1) to (4) of the proof of the *Ironwood Guide*’s Proposition “Confidentiality and key privacy of note encryption”, as in game G_6 . The common length is the hypothesis of (Q2) on the plaintext encoding.

After H_9 , the seed rseed of each created note enters the game only through ψ and rcm , that is through cm . With rcm a PRF-expansion output of rseed under an assigned lead byte, as for either derivation of Definition 3.5, the argument of the proof of the *Ironwood Guide*’s Proposition “Hiding under the derived trapdoor” places rcm within negligible distance of a uniform scalar

independent of the rest of the game.

Hybrid H_{10} (*note commitments*). Each cmx is replaced by the x -coordinate of a uniform point, by hiding with one output distribution for both branches (Proposition 3.3(a)), as in game G_7 .

In H_{10} every public field is a uniform value independent of the specification, a simulated proof or signature on public values, or a function of the leakage of Definition 9.10: the value commitments, nullifiers and extracted commitments are uniform, each epk is a uniform non-identity point, the ciphertexts encrypt fixed strings under fresh keys, each rk_i is uniform, and bvk is a function of these and of the common burn set and balancing value. The two hybrids H_{10} are therefore identically distributed, and the advantage is at most the sum of the costs for both specifications, which is negligible. $\square$

Remark 9.12 (Limits of Asset-type privacy).

- (a) *Leakage*. Each burn publishes its Asset Base and amount, “the amount and identifier of the Asset being burnt” (ZIP 226, “Rationale for Value Balance Verification”). The flag $\text{enableZSA} = 0$ shows that a bundle moves ZEC only (C2). The balancing value is an amount of ZEC. Issuance is public (Remark 6.6), although a later spend of an Issue Note is unlinkable to it (Proposition 6.4), except for a reference note.
- (b) *Action count*. Every Action has one Asset Base ($A1', A2'$), consumes at most one real note and creates one note. A bundle whose Actions move k Assets, Asset j with i_j real consumed notes and o_j created notes, therefore has at least $\sum_{j=1}^k \max(i_j, o_j) \geq k$ Actions, and the public number of Actions bounds k . Theorem 9.11 hides the Assets only among bundles with equal Action counts.
- (c) *Order*. ZIP 226 fixes no Action order. Theorem 9.11 holds for every order against an adversary bound by (Q4); it claims nothing against a recipient, which by (e) is outside its scope.
- (d) *Anonymity set*. ZIP 226, “Privacy Implications”, separates Orchard and OrchardZSA note commitments by transaction version, version 5 for Orchard and version 6 for OrchardZSA (Remark 3.4); no current format assigns these versions so (Remark 8.1). The anonymity set of an Action is the set of OrchardZSA Actions of whatever carrier is specified, and an Orchard-protocol bundle reveals that its notes are ZEC. The pool that carries OrchardZSA notes is the open problem of Remark 1.7.
- (e) *Non-claims*. The theorem claims nothing against a holder of a viewing key of a key involved; nothing for the copied note of a Split Input beyond the unlinkability of its split nullifier; and nothing about timing, network metadata or information outside the transaction.

(ZIP 226, “Privacy Implications”, “Rationale for Value Balance Verification” and “Asset Base Equality”.)

10 Classification of the components

This section collects the classes that the preceding sections assign. It tabulates the class of every construction of the volume under Definition 1.2 and the unspecified objects on which every result

is conditional under Definition 1.3 (§10.1), and it enumerates the open problems, each with the texts that leave it open and the obligation that closes it (§10.2). No class is derived here; each is the one stated where the construction or result is made.

10.1 Classes of the constructions and results

Remark 10.1 (Reading of the table). Each row of the first part of Table 7 is a construction, or a named part of one, with the subsection that states it; the rows are grouped by their class under Definition 1.2. “Specified” means specified by Draft text: no ZIP of OrchardZSA is Final and no network upgrade includes one (item (1) of “Open problems”, §10.2). The second part lists every theorem, proposition, lemma and corollary of the volume. A result carries no class (Definition 1.3); its row states only the unspecified objects that its hypotheses concern, and the rows are grouped by those objects. The standing assumptions of “Accepted transactions and assumptions” (§9.1) on specified primitives, namely the group hash, discrete logarithms on Pallas, BLAKE2b, RedPallas, IssueAuthSig, note encryption, Poseidon as a PRF, PRF expansion and the joint hiding of the viewing keys, are not repeated per row.

The reading of condition A8 has a group of its own. ZIP 226 leaves conditions A8 and A9 unchanged by its delta, so A8 reads v^{old} and A9 reads v^{new} , and a Split Action that copies a note of non-zero value needs `enableSpend` = 1; this is specified by the delta. The confirmation of that reading in the circuit document outside the ZIPs is designed but unspecified (Remark 4.7).

The groups of the second part read as follows. A fixed derivation of `rcm` for Issue Notes is needed only so that the validator’s commitment is computable; any deterministic derivation from the published fields suffices (Definition 1.3), and the corollary and the theorem that use Lemma 9.3 carry that qualification only through it. Theorem 9.11 needs the lead byte for `rcm` to be uniform as a pseudorandom output, and the plaintext encoding to have a length independent of the Asset; both are open, items (2) and (8). No result is conditional on the carrier otherwise than through Assumption 6.13.

Table 7. Classification of the OrchardZSA components, first part: every construction with its class under Definition 1.2 and the subsection that states it. “Specified” means specified by Draft ZIP text.

Construction	Subsection
<i>Specified (Draft text)</i>	
Issuance key pair, IssueAuthSig, issuer and signature encodings	§2.1
Asset description, Asset Identifier and its encoding	§2.2
Asset Digest, Asset Base and its byte encoding	§2.3
Rejection of an issuance whose derived Asset Base is the identity, by typing (Remark 2.12)	§2.3
Native Asset Base	§2.4
OrchardZSA note	§3.1
OrchardZSA note commitment	§3.2
Derivation of ψ	§3.3
Field set of the OrchardZSA note plaintext	§3.4
Per-Asset value commitment	§3.5
Split Inputs, Split Actions and the padding rule	§4.1
Split-nullifier formula, a sender rule that no condition of the statement enforces (Remark 4.8)	§4.2
Statement delta: conditions A1' to A5' and C1 to C3 against A1 to A9, with A8 and A9 unchanged and reading v^{old} and v^{new}	§4.3
Structure of the Action Group	§4.5
Burn set, rules (B1) to (B3), structure of the OrchardZSA bundle	§5.1
Binding validating key; its signed message excepted	§5.2
Issue Note; the rule that validators compute its commitment	§6.1
Issuance Action and Issuance Bundle	§6.2
Derivation of issued ρ values	§6.3
Reference note	§6.4
Rules (I1) to (I3); the signed message excepted	§6.5
Global issuance state and its chaining	§7.1
Transition rules (T1) to (T4) and (I4) to (I7); insertion order of note commitments	§7.2
Finalisation	§7.3
Checks of the verification procedure, each carrier tagged with its own class	§7.4
<i>Specified by the delta; realisation designed but unspecified</i>	
Reading of A8 on v^{old} : specified; its confirmation in the circuit document outside the ZIPs designed but unspecified	§4.3
<i>Designed but unspecified</i>	
OrchardZSA circuit, its verifying key and the proof length it determines	§4.4
Derivation of rcm, up to its lead byte	§3.3
Withdrawn transaction layouts, read as a composition of the constructions	§8.1
Withdrawn transaction-identifier and signature digests, and the signature message SigHash they define (the authorising-data digest excepted, Remark 8.1(ii))	§8.2, §6.5
ZSA fee terms	§8.3
Rationale arguments of the ZIPs for the properties proved in “Security”	§9
<i>Open problem (item of “Open problems”)</i>	
(1) Network upgrade; activation block of the issuance state	§7.1
(2) Value of the lead byte	§3.3
(3) Live carrier: encodings, digests, signature message, and the bit and instance position of enableZSA	§8.1, §8.2
(4) Carrying pool (Remark 1.7)	§1.3
(5) Composition of the delta with A10 and disableCrossAddress	§4.3
(6) Empty Issuance Action for a new Asset	§7.2
(7) Issue Note whose commitment is $\perp$	§6.1
(8) Encoding of the note plaintext	§3.4

Table 7. Classification of the OrchardZSA components, second part: every result of the volume, grouped by the unspecified objects that its hypotheses concern (Definition 1.3). Standing assumptions on specified primitives are omitted.

Result	Number
<i>Conditional on no unspecified object</i>	
Proposition “Uniqueness of Asset Identifiers”	2.7
Lemma “Distinctness of the OrchardZSA group-hash inputs”	2.8
Lemma “Non-identity of derived Asset Bases”	2.11
Proposition “Separation of Asset Bases”	2.13
Proposition “Hiding and binding of OrchardZSA note commitments”, stated for uniform rcm	3.3
Lemma “Hiding and binding of per-Asset value commitments”	3.9
Proposition “Unsoundness of the dummy exemption under a witnessed base”	4.1
Proposition “Independence of split nullifiers”; part (ii) for senders that follow the sender rule	4.5
Lemma “Modular per-Asset balance is integer balance”	5.5
Proposition “Unlinkability of spends of Issue Notes”	6.4
Proposition “Uniqueness of issued rho values”	6.9
Lemma “Keys of the issuance state”	7.7
Proposition “Finalised supply never grows”	7.8
Proposition “The recorded balance”	7.9
<i>Conditional on an open case</i>	
Lemma “First-issuance detection”: on item (6)	7.6
Proposition “Reference notes as universal Split Inputs”, part (a): on item (2), the lead byte	6.11
<i>Conditional on the circuit, through Assumption 4.9</i>	
Proposition “Ownership of Split Inputs”	4.12
Proposition “Per-Asset balance and key knowledge”	5.6
Proposition “Reference notes as universal Split Inputs”, part (b), on zero-valued outputs, through Proposition 5.6	6.11
<i>Conditional on the circuit, through Assumption 4.9, and, for leaves of Issue Notes, on a fixed derivation of rcm</i>	
Lemma “Provenance of Asset Bases”	9.3
Lemma “One consumption per OrchardZSA note”	9.6
Theorem “Supply integrity”	9.7
Corollary “Asset Bases of accepted Actions”, through Lemma 9.3	9.4
Theorem “No counterfeiting across Assets”, through Lemma 9.3	9.5
<i>Conditional on the signed message, through Assumption 6.13</i>	
Proposition “Replay protection of issuance”	6.16
Theorem “Issuance authority”	9.8
<i>Conditional on the circuit, through Assumption 4.10, on the lead byte and on the plaintext encoding</i>	
Theorem “Privacy of the Asset type”	9.11

10.2 Open problems

Each item below is an open problem in the sense of Definition 1.2: no text supplies a design, every existing design conflicts with live text, or live texts contradict one another. Each item names the texts that leave it open and the obligation that closes it.

- (1) *Network upgrade.* ZIP 226, “Deployment”, names NU7; ZIP 227, “Deployment”, reads “TBD”. ZIP 259 (Draft), which specifies NU7, lists its consensus changes without either ZIP (“NU7 consensus changes”) and introduces no transaction format (“Rationale for no new transaction format”); ZIP 258, which specifies NU6.3, mentions neither. Consequently the first block of OrchardZSA, whose input issuance state is the empty map (Definition 7.1), is not assigned. Closed by a deployment ZIP that includes ZIPs 226 and 227.
- (2) *Value of the lead byte of OrchardZSA notes and Issue Notes.* ZIP 226, “Note Structure and Commitment”, fixes only the placeholder `{{ZSALEADBYTE}}`; ZIP 2005, “Usage with other proposals requiring note plaintext format changes”, requires a new lead byte for plaintext changes; and the removed design’s `0x03` is the Ironwood lead byte in live text (protocol specification, §“Note Plaintexts and Memo Fields”; Remark 3.6). Consequently the commitment of an Issue Note is not computable from live text (Remark 6.2). Closed by an assignment of the value and an extension of `Derive_rcm` to it.
- (3) *Live carrier.* ZIP 226, “OrchardZSA Transaction Structure”, and ZIP 227 defer the encodings to ZIP 230, and ZIP 226, “Sighash modifications relative to ZIP 244”, and ZIP 227, “Modifications relative to ZIP 244”, defer the signature message to ZIP 246; both are Withdrawn. Version 6 is the format of ZIP 229, which carries no ZSA bundle, and ZIP 248 lists OrchardZSA and ZSA Issuance only as potential bundle types without an allocated type (Remarks 8.1 and 8.3). The flag `enableZSA` has no bit and no instance position in any current format: the withdrawn design places it at bit 2 of `flagsOrchard`, which the consensus rules of ZIP 229 reserve and require to be 0, the format table of that ZIP naming the bit `enableCrossAddress`. Closed by a live format that encodes the OrchardZSA bundle, the Issuance Bundle and `enableZSA`, with a signature digest that meets Remark 8.2.
- (4) *Carrying pool.* ZIP 226, “Backward Compatibility”, places OrchardZSA notes in the Orchard pool, which ZIP 258, “Consensus rules from NU6.3 activation”, closes to new value and to cross-address transfers from NU6.3 (Remark 1.7). The rules of that pool then forbid transfers of Custom Assets between distinct addresses, leave issuance into it unaddressed, and leave undetermined the tree that Definition 7.4 extends. Closed by a reconciliation of ZIP 226 with ZIP 258 that names the pool and its rules.
- (5) *Composition of the delta with condition A10.* ZIP 226, “Circuit Statement”, states its delta against A1 to A9; the current statement adds condition A10 and the primary input `disableCrossAddress` (*Ironwood Guide*, Definition “Orchard Action statement”; Remark 4.7). Closed by a statement text that includes both, which depends on the resolution of item (4).
- (6) *Empty Issuance Action for a new Asset.* For an Issuance Action without Issue Notes on an Asset Base without a reference note, rule (I5) refers to a first Issue Note that does not exist,

so its validity, finalising or not, is undetermined (ZIP 227, “Specification: Consensus Rule Changes”; Remark 7.5). Lemma 7.6 is conditional on this case. Closed by a rule that rejects such an action or fixes its effect on the issuance state.

- (7) *Issue Note whose commitment is $\perp$* . The protocol specification, §“Sending Notes (Orchard)”, makes a sent note whose commitment is $\perp$ invalid and resamples `rseed`, which covers sent notes only; ZIP 227, “Issue Note”, states no rule (Remark “Commitment $\perp$ for an Issue Note” in §6.1). The event has negligible probability. Closed by a rule: resampling by the issuer or rejection by validators.
- (8) *Encoding of the OrchardZSA note plaintext*. The field set is specified (Definition 3.7); ZIP 226, “Note Structure and Commitment”, delegates the encoding to ZIP 230, which is Withdrawn. Theorem 9.11 is conditional on it. Closed by a live encoding whose length does not depend on the Asset.

Remark 10.2 (Items not open). Four items are not open problems. The bound on the number of Actions that Lemma 5.5 needs, at most $62\,500 < 2^{16}$ Actions per transaction, follows from the block-size rule (protocol specification, §“Block Header Encoding and Consensus”) whatever the carrier. The ZSA fee terms (Remark 8.4), the derivation of `rcm` (Remark 3.6) and the OrchardZSA proof length (Remark 4.11) are designed but unspecified, as the first part of Table 7 records: each has a design that no live text contradicts. ZIP 227 and ZIP 317 disagree about the fee terms, which the protocol does not need (Remark 8.4).